

الجمهورية العربية السورية
المعهد العالي للعلوم التطبيقية والتكنولوجيا
قسم المعلومات
العام الدراسي 2025/2026

مشروع سنة رابعة

بناء نظام مراسلة موزع يعتمد على نموذج النشر والاشتراك

تقديم
عبد المجيد علي العوض

إشراف
د. حسين خيو

2025/2026

الخلاصة

يهدف هذا المشروع إلى تصميم وتنفيذ نظام مراسلة موزع يعتمد على نموذج النشر والاشتراك (Publish/Subscribe)، بحيث يتم فصل عملية نشر الرسائل عن توزيعها على المستخدمين المشتركين، مع ضمان التخزين الدائم وإمكانية استعادة الرسائل بعد انقطاع الاتصال.

تم بناء النظام باستخدام FastAPI و PostgreSQL و RabbitMQ و Redis و WebSocket و Next.js، مع دعم القنوات والعضويات والصلاحيات والرسائل والمرفقات. كما تم تطبيق آليات للمصادقة وإدارة الجلسات وحماية البيانات، بالإضافة إلى سجل أحداث قابل للتحقق من سلامته وكشف محاولات العبث.

أظهرت الاختبارات نجاح تكامل مكونات النظام وتنفيذ مسار نشر الرسائل وتوزيعها واستعادتها، مما حقق الهدف الأساسي للمشروع في بناء نظام مراسلة موزع قابل للتشغيل يجمع بين الموثوقية والأمان وسهولة الاستخدام.

Abstract

This project aims to design and implement a distributed messaging system based on the Publish/Subscribe model, where message publishing is separated from message distribution to subscribed users while providing persistent storage and message recovery after connection interruptions.

The system was developed using FastAPI, PostgreSQL, RabbitMQ, Redis, WebSocket, and Next.js. It supports channels, memberships, access permissions, messages, and attachments. Authentication, session management, data protection, and a tamper-evident event log were also implemented.

The conducted tests demonstrated successful integration between the system components and verified the complete message publishing, distribution, and recovery flow. The project therefore achieved its main objective of building a functional distributed messaging system that combines reliability, security, and ease of use.

المحتويات

1.....	الخلاصة	1.....
1.....	Abstract	1.....
12.....	مقدمة عامة	12.....
13.....	الفصل الأول	13.....
13.....	التعريف بالمشروع	13.....
13.....	1.1 - مقدمة	13.....
13.....	2.1 - مشكلة المشروع	13.....
13.....	3.1 - هدف المشروع	13.....
14.....	4.1 - متطلبات وظيفية	14.....
15.....	5.1 - متطلبات غير وظيفية	15.....
16.....	6.1 - خلاصة الفصل	16.....
17.....	الفصل الثاني	17.....
17.....	الدراسة المرجعية	17.....
18.....	الفصل الثالث	18.....
18.....	الدراسة التحليلية	18.....
18.....	1.3 - المقدمة	18.....
18.....	2.3 - الممثلون في النظام	18.....
19.....	3.3 - مخطط حالات الاستخدام	19.....
20.....	4.3 - السرد النصي لحالات الاستخدام	20.....
20.....	1.4.3- تسجيل الدخول	20.....
21.....	2.4.3- إنشاء قناة	21.....

22.....	الانضمام إلى القناة	3.4.3-
23.....	نشر رسالة في قناة	4.4.3-
24.....	استعراض رسائل القناة	5.4.3-
25.....	مخططات تسلسل النظام	5.3-
25.....	تسجيل الدخول	1.5.3-
26.....	إنشاء القناة	2.5.3-
27.....	الانضمام إلى قناة	-3.5.3
28.....	نشر رسالة في قناة	-4.5.3
29.....	استعراض رسائل القناة	-5.5.3
30.....	مخطط علاقات الكيانات ERD	6.3-
31.....	مخطط الصفوف	-7.3
31.....	خلاصة	8.3-
32.....	الفصل الرابع	
32.....	الدراسة النظرية	
33.....	الفصل الخامس	
33.....	الدراسة التصميمية	
33.....	المقدمة	-1.5
33.....	منهجية التصميم	-2.5
34.....	المعمارية العامة للنظام	-3.5
35.....	مكونات النظام ومسؤولياتها	-4.5
36.....	تصميم الواجهة الخلفية	-5.5
37.....	مخطط الواجهة الخلفية	-6.5

38.....	تصميم قاعدة البيانات	-7.5
39.....	تصميم مسار نشر الرسالة وتوزيعها	8.5-
40.....	تصميم آليات موثوقية التوزيع ومعالجة الفشل	-9.5
42.....	تصميم إدارة العضويات وربطها بتوجيه الرسائل	-10.5
44.....	تصميم الاتصال اللحظي ومزامنة الرسائل	-11.5
46.....	تصميم إدارة الملفات والمرفقات	-12.5
48.....	تصميم المصادقة وإدارة الجلسات	-13.5
50.....	تصميم التحكم في الصلاحيات والأدوار	-14.5
52.....	تصميم سجل الأحداث والتدقيق	-15.5
54.....	تصميم التحقق من سلامة سجل البحث	-16.5
56.....	تصميم الواجهة الأمامية	17.5-
58.....	تصميم بيئة التشغيل والنشر	-18.5
59.....	الخلاصة	-19.5
60.....	الفصل السادس	
60.....	الأدوات المستخدمة	
60.....	المقدمة	-1.6
60.....	لغة Python	-2.6
Uvicorn و	FastAPI	-3.6
		61
PostgreSQL.....		-4.6
		61
AsyncPG و	SQLAlchemy	-5.6
		62

Alembic	-6.6
	62
aio-pika و	RabbitMQ-7.6
	63
Redis و	WebSocket-8.6
	63
TypeScript و	Next.js-9.6
React و	
	64
Docker	Docker-10.6
Compose و	
	65
Nginx	-11.6
	65
66..... أدوات الأمان والتشفير	-12.6
66..... أدوات الاختبار والتحقق	-13.6
GitHub و	Git-14.6
	67
67..... الخلاصة	15.6-
69..... الفصل السابع	
69..... القسم العملي	
69..... المقدمة	-1.7
69..... تنظيم المشروع	-2.7
69..... البنية العامة للمشروع	1.2.7-
70..... تنظيم الواجهة الخلفية	-2.2.7
72..... فصل مكونات النظام	3.2.7-
73..... تنفيذ الواجهة الخلفية ووظائف النظام	3.7-

73.....	تنفيذ المصادقة وإدارة الجلسات	1.3.7-
74.....	تنفيذ القنوات والعضويات والصلاحيات	2.3.7-
75.....	تنفيذ الرسائل	3.3.7-
76.....	تنفيذ الملفات والمرفقات	4.3.7-
77.....	تنفيذ التوزيع غير المتزامن والاتصال اللحظي	4.7-
77.....	تنفيذ Transactional Outbox	1.4.7-
78.....	تنفيذ RabbitMQ و Worker	2.4.7-
79.....	معالجة الفشل وإعادة المحاولة	3.4.7-
80.....	الاتصال اللحظي والمزامنة	4.4.7-
82.....	حماية الحسابات والبيانات	1.5.7-
83.....	تنفيذ سجل الأحداث	2.5.7-
84.....	تنفيذ التحقق من سلامة السجل	3.5.7-
86.....	تنفيذ الواجهة الأمامية ودليل الاستخدام	6.7-
86.....	واجهة المصادقة والتنقل	1.6.7-
87.....	واجهات القنوات والعضويات	2.6.7-
88.....	واجهة المراسلة	3.6.7-
88.....	تكامل مكونات النظام وتشغيله	7.7-
89.....	سيناريو نشر رسالة متكامل	1.7.7-
90.....	انقطاع الاتصال وإعادة المزامنة	2.7.7-
92.....	تشغيل النظام	3.7.7-
93.....	الاختبارات والنتائج	8.7-
93.....	منهجية الاختبار	1.8.7-

94.....	الاختبارات الوظيفية والتكاملية	2.8.7-
97.....	اختبارات الأمان والموثوقية	3.8.7-
99.....	نتائج الاختبارات	4.8.7-

قائمة الأشكال

19.....	الشكل رقم 1 مخطط حالات الاستخدام الرئيسة في النظام
25.....	الشكل رقم 2 مخطط تسلسل عملية تسجيل الدخول
26.....	الشكل رقم 3 مخطط تسلسل عملية إنشاء قناة جديدة
27.....	الشكل رقم 4 مخطط تسلسل عملية الانضمام إلى قناة
28.....	الشكل رقم 5 مخطط تسلسل عملية نشر رسالة في قناة
29.....	الشكل رقم 6 مخطط تسلسل عملية استعراض رسائل القناة
30.....	الشكل رقم 7 مخطط علاقات الكيانات الرئيسة في النظام (ERD)
31.....	الشكل رقم 8 مخطط الصفوف الرئيسة في النظام
34.....	الشكل رقم 9 المعمارية العامة لنظام المراسلة الموزع
37.....	الشكل رقم 10 البنية الداخلية للواجهة الخلفية ومكوناتها الرئيسة
39.....	الشكل رقم 11 مسار نشر الرسالة وتوزيعها على المشتركين
41.....	الشكل رقم 12 حالات معالجة سجل Outbox وإعادة المحاولة عند الفشل
43.....	الشكل رقم 13 ربط العضويات بمسار توجيه الرسائل
45.....	الشكل رقم 14 مسار الاتصال اللحظي ومزامنة الرسائل بعد انقطاع الاتصال
47.....	الشكل رقم 15 مسار رفع الملفات وربطها بالرسائل
49.....	الشكل رقم 16 مسار المصادقة وإدارة جلسات المستخدم
51.....	الشكل رقم 17 مسار التحقق من الصلاحيات والأدوار داخل القنوات
53.....	الشكل رقم 18 مسار إنشاء سجل الأحداث وتخزينه
55.....	الشكل رقم 19 مراحل التحقق من سلامة سجل الأحداث باستخدام التجزئة وشجرة Merkle والتوقيع الرقمي

الشكل رقم 20	البنية العامة للواجهة الأمامية وتكاملها مع خدمات النظام.....	57
الشكل رقم 21	بنية تشغيل ونشر مكونات النظام باستخدام Docker وNginx.....	58
الشكل رقم 22	البنية العامة لمجلدات المشروع وملفات التشغيل الرئيسة.....	70
الشكل رقم 23	التنظيم الداخلي لملفات الواجهة الخلفية.....	71
الشكل رقم 24	حالة الحاويات وهي تعمل	78
الشكل رقم 25	حالات معالجة سجل Outbox وإعادة المحاولة عند الفشل.....	79
الشكل رقم 26	مسار الاتصال اللحظي ومزامنة الرسائل بعد انقطاع الاتصال.....	81
الشكل رقم 27	آلية التحقق من سلامة سجل الأحداث باستخدام سلسلة التجزئة وشجرة Merkle....	85
الشكل رقم 28	مسار نشر الرسالة وتوزيعها من المرسل إلى المستقبل عبر مكونات النظام.....	89
الشكل رقم 29	مسار استعادة الرسائل والمزامنة بعد انقطاع الاتصال.....	91
الشكل رقم 30	نتيجة ال full Pub/Sub flow	96

قائمة الجداول

الجدول رقم 3	بيانات حالة استخدام تسجيل الدخول	20
الجدول رقم 4	سير الأحداث لحالة استخدام تسجيل الدخول	20
الجدول رقم 5	بيانات حالة استخدام إنشاء قناة	21
الجدول رقم 6	سير الأحداث لحالة استخدام إنشاء قناة	21
الجدول رقم 7	بيانات حالة استخدام الانضمام إلى قناة	22
الجدول رقم 8	سير الأحداث لحالة استخدام الانضمام إلى قناة	22
الجدول رقم 9	بيانات حالة استخدام نشر رسالة في قناة	23
الجدول رقم 10	سير الأحداث لحالة استخدام نشر رسالة في قناة	23
الجدول رقم 11	بيانات حالة استخدام استعراض رسائل القناة	24
الجدول رقم 12	سير الأحداث لحالة استخدام استعراض رسائل القناة	24
الجدول رقم 13	آليات الحماية الأمنية المطبقة في النظام	83
الجدول رقم 14	أمثلة عن الأحداث المسجلة وأغراض تسجيلها	84
الجدول رقم 15	مستويات الاختبار وعمليات التحقق	94
الجدول رقم 16	نتائج الاختبارات الوظيفية والتكاملية	95
الجدول رقم 17	نتائج اختبارات الأمان والموثوقية	97
الجدول رقم 18	النتائج النهائية للاختبارات وعمليات التحقق	99

الاختصارات

الرموز

مقدمة عامة

أصبحت أنظمة المراسلة من التطبيقات الأساسية في الأنظمة الشبكية الحديثة، حيث يتطلب تبادل المعلومات بين عدد كبير من المستخدمين توفير آليات تسمح بإيصال الرسائل بكفاءة، مع المحافظة على إمكانية الوصول إليها حتى في حال انقطاع بعض المستخدمين أو حدوث فشل مؤقت في إحدى خدمات النظام.

في أنظمة المراسلة التقليدية قد يرتبط مرسل الرسالة بصورة مباشرة بالمستقبلين، مما يزيد الترابط بين مكونات النظام ويجعل إدارة التوزيع أكثر تعقيداً مع ازدياد عدد المستخدمين. يقدم نموذج النشر والاشتراك (Publish/Subscribe) أسلوباً مختلفاً، حيث يقوم الناشر بإرسال الرسالة إلى قناة أو موضوع، بينما تتولى البنية الوسيطة مسؤولية توزيعها على المشتركين، دون الحاجة إلى وجود ارتباط مباشر بين الطرفين.

يهدف هذا المشروع إلى دراسة هذا النموذج وتطبيقه عملياً من خلال بناء نظام مراسلة موزع يدعم إنشاء القنوات وإدارة المشتركين ونشر الرسائل النصية والوسائط واستقبالها في الزمن الحقيقي، مع الاحتفاظ بالرسائل بصورة دائمة وإتاحة استرجاعها عند فقد التسليم اللحظي. كما يهتم النظام بالمصادقة والصلاحيات وحماية محتوى الرسائل ومراقبة عملية توزيعها. يعرض هذا التقرير مراحل بناء النظام بدءاً من تحديد متطلباته ودراسة الأنظمة المشابهة، ثم عرض الأسس النظرية والتحليلية والتصميمية المعتمدة، وصولاً إلى الأدوات المستخدمة والتنفيذ العملي للنظام واختباره.

الفصل الأول

التعريف بالمشروع

يتضمن هذا الفصل تعريفاً للمشروع والغاية منه، بالإضافة إلى تحديد المتطلبات الوظيفية وغير الوظيفية التي يقوم عليها النظام.

1.1 - مقدمة

تُعد أنظمة المراسلة من المكونات الأساسية في التطبيقات الشبكية الحديثة، إذ تتطلب نقل الرسائل بين المستخدمين بكفاءة مع الحفاظ على موثوقية الوصول إليها عند حدوث انقطاع أو فشل مؤقت في إحدى خدمات النظام. ويقدم نموذج النشر والاشتراك (Publish/Subscribe) أسلوباً مناسباً لهذه الأنظمة، إذ يفصل بين ناشر الرسالة والمشاركين في استقبالها. وانطلاقاً من ذلك، يهدف هذا المشروع إلى بناء نظام مراسلة موزع يعتمد على القنوات، ويدعم النشر والاستقبال اللحظي مع التخزين الدائم للرسائل وإدارة الصلاحيات وآليات الأمان.

2.1 - مشكلة المشروع

تُعاني أنظمة المراسلة التي تعتمد على الاتصال المباشر بين المرسل والمستقبلين من زيادة الترابط بين مكوناتها وصعوبة إدارة توزيع الرسائل مع ازدياد عدد المستخدمين. كما تظهر مشكلة إضافية عند انقطاع أحد المستخدمين أو فشل إحدى خدمات التوزيع، إذ يجب ضمان عدم فقدان الرسالة وإمكانية استرجاعها لاحقاً. لذلك تتمثل مشكلة المشروع في بناء آلية مراسلة تفصل بين ناشر الرسالة ومستقبلها، وتدعم التوزيع غير المتزامن مع التخزين الدائم وإدارة حالات فشل التسليم والصلاحيات.

3.1 - هدف المشروع

يهدف المشروع إلى تصميم وتنفيذ نظام مراسلة موزع وآمن يطبق نموذج النشر والاشتراك، ويفصل بين ناشر الرسالة ومستقبلها، مع توفير التسليم اللحظي والتخزين الدائم للرسائل، وإدارة القنوات والمشاركين والصلاحيات بطريقة قابلة للتوسع والصيانة.

4.1- متطلبات وظيفية

يجب أن يوفر النظام مجموعة من الوظائف الأساسية التي تحقق هدف المشروع وتسمح بإدارة عملية النشر والاشتراك بصورة آمنة وموثوقة، ويمكن تلخيصها بما يأتي:

1. إدارة المستخدمين والمصادقة

- يجب أن يتيح النظام للمستخدم إنشاء حساب وتسجيل الدخول.
- يجب أن يتحقق النظام من هوية المستخدم قبل السماح له بالوصول إلى الوظائف المحمية.
- يجب أن يتيح النظام إدارة جلسات المستخدم وتسجيل الخروج منها.

2. إدارة القنوات

- يجب أن يتمكن المستخدم من إنشاء قناة وتحديد إعداداتها.
- يجب أن يدعم النظام القنوات العامة والخاصة.
- يجب أن يتمكن مالك القناة أو المستخدم المخول من تعديل إعداداتها وإدارة أعضائها.

3. إدارة العضويات والدعوات

- يجب أن يتمكن المستخدم من الانضمام إلى القنوات وفق سياسة الانضمام الخاصة بها.
- يجب أن يدعم النظام الدعوات وطلبات الانضمام التي تتطلب موافقة.
- يجب أن يتمكن المستخدم المخول من قبول الأعضاء أو إزالتهم وتحديد أدوارهم وصلاحياتهم.

4. نشر الرسائل واستقبالها

- يجب أن يتمكن العضو المخول من نشر الرسائل النصية والوسائط ضمن القنوات.
- يجب أن يحتفظ النظام بالرسائل بصورة دائمة.
- يجب أن تصل الرسائل الجديدة إلى المشتركين المتصلين بصورة لحظية.
- يجب أن يتمكن المستخدم من استرجاع الرسائل التي فاتته استقبالها عند إعادة الاتصال.

5. إدارة الملفات والوسائط

- يجب أن يسمح النظام برفع الملفات وربطها بالرسائل.
- يجب ألا يتمكن من الوصول إلى الملفات الخاصة إلا المستخدمون الذين يمتلكون الصلاحية المناسبة.

6. التحكم في الوصول والصلاحيات

- يجب أن يتحقق النظام من عضوية المستخدم ودوره قبل تنفيذ عمليات القراءة أو النشر أو الإدارة.
- يجب رفض أي عملية لا يمتلك المستخدم الصلاحية اللازمة لتنفيذها.

7. متابعة عمليات توزيع الرسائل

- يجب أن يسجل النظام حالة عمليات توزيع الرسائل.
- يجب أن يعيد محاولة عمليات التوزيع التي تفشل بصورة مؤقتة.
- يجب أن يحتفظ بحالة العمليات التي يتعذر تسليمها بعد استنفاد محاولات إعادة الإرسال.

8. سجل الأحداث والتحقق من سلامته

- يجب أن يسجل النظام الأحداث المهمة المتعلقة بالمستخدمين والقنوات والرسائل والعمليات الإدارية.
- يجب أن يوفر آلية للتحقق من سلامة سجل الأحداث والكشف عن أي تعديل غير مشروع عليه.

5.1- متطلبات غير وظيفية

إضافة إلى الوظائف الأساسية، يجب أن يحقق النظام مجموعة من المتطلبات غير الوظيفية التي تضمن جودة التشغيل واستقراره، وأهمها:

1. الأداء

- يجب أن يستجيب النظام لطلبات المستخدمين بزمان مناسب.
- يجب أن يتيح استقبال الرسائل الجديدة بصورة لحظية قدر الإمكان دون التأثير في التخزين الدائم للرسائل.

2. الموثوقية

- يجب ألا يؤدي الفشل المؤقت في إحدى خدمات التوزيع إلى فقدان الرسائل المخزنة.
- يجب أن يدعم النظام إعادة محاولة عمليات التوزيع الفاشلة وإمكانية استرجاع الرسائل بعد إعادة الاتصال.

3. الأمان

- يجب حماية حسابات المستخدمين وبيانات المصادقة باستخدام آليات آمنة.
- يجب حماية الوصول إلى القنوات والرسائل والملفات وفق صلاحيات المستخدم.
- يجب حماية البيانات الحساسة أثناء التخزين، مع توفير آليات للكشف عن التلاعب بسجل الأحداث.

4. قابلية التوسع

- يجب أن يسمح تصميم النظام بفصل مكونات التخزين والتوزيع والاتصال اللحظي، بما يسهل تطويرها أو توسيعها بصورة مستقلة عند الحاجة.

5. قابلية الصيانة والتطوير

- يجب تنظيم الشيفرة البرمجية إلى مكونات واضحة ذات مسؤوليات محددة.
- يجب أن يكون من الممكن إضافة وظائف جديدة أو تعديل المكونات الحالية دون الحاجة إلى إعادة بناء النظام بالكامل.

6. سهولة الاستخدام

- يجب أن توفر الواجهة طريقة واضحة وبسيطة لإدارة القنوات والعضويات والرسائل.
- يجب أن تعرض حالات الخطأ والعمليات المهمة للمستخدم بصورة مفهومة.

7. قابلية الاختبار

- يجب أن يسمح تصميم النظام باختبار وظائفه ومكوناته بصورة مستقلة ومتكاملة، بما يشمل الاختبارات الوظيفية والأمنية واختبارات التكامل والموثوقية.

6.1- خلاصة الفصل

تم في هذا الفصل تقديم فكرة المشروع وتحديد المشكلة التي يسعى إلى معالجتها، والمتمثلة في بناء نظام مراسلة قادر على توزيع الرسائل بين المستخدمين بصورة موثوقة مع الفصل بين الناشرين والمشاركين. كما تم تحديد أهداف المشروع ونطاقه، واستعراض أهم المتطلبات الوظيفية وغير الوظيفية التي يجب أن يحققها النظام. وتشكل هذه المتطلبات الأساس الذي سيتم الاعتماد عليه في تحليل النظام وتصميم مكوناته وآليات عمله في الفصول اللاحقة.

الفصل الثاني

الدراسة المرجعية

غير ضرورية الآن

الفصل الثالث

الدراسة التحليلية

يوضح هذا الفصل تحليل النظام من خلال تحديد الجهات المتفاعلة معه وحالات الاستخدام الرئيسية، ثم دراسة تسلسل بعض العمليات الأساسية والعلاقات بين الكيانات التي يتعامل معها النظام.

1.3- المقدمة

تهدف الدراسة التحليلية إلى تحديد طريقة تفاعل المستخدمين مع النظام وتحليل الوظائف الأساسية التي يجب أن يوفرها قبل الانتقال إلى مرحلة التصميم والتنفيذ. ويتناول هذا الفصل تحديد الممثلين الرئيسيين في النظام، وحالات الاستخدام المرتبطة بكل منهم، إضافة إلى دراسة تسلسل بعض العمليات الأساسية والقواعد التي تحكم عمل النظام، بما يوضح سلوكه من الناحية الوظيفية.

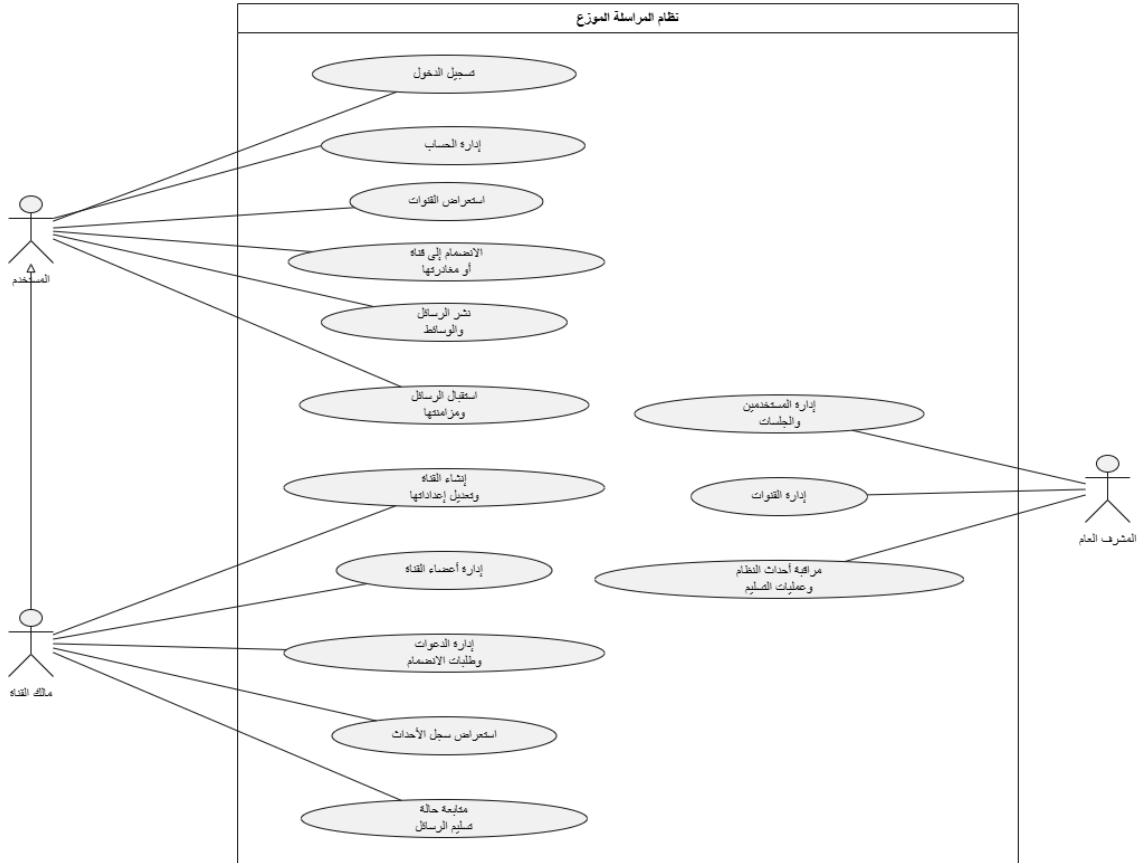
2.3- الممثلون في النظام

يتفاعل مع النظام عدد من الممثلين الذين تختلف صلاحياتهم والعمليات المتاحة لكل منهم. ويمكن تحديد الممثلين الرئيسيين كما يأتي:

1. **المستخدم:** يمثل أي مستخدم مسجل في النظام، ويمكنه تسجيل الدخول، واستعراض القنوات المتاحة، والانضمام إليها وفق شروطها، ونشر الرسائل واستقبالها وإدارة حسابه.
2. **مالك أو مدير القناة:** هو مستخدم يمتلك صلاحيات إضافية ضمن قناة محددة، مثل إدارة إعدادات القناة، وقبول طلبات الانضمام، ودعوة المستخدمين، وإدارة الأعضاء والأدوار والصلاحيات.
3. **المشرف العام (Superadmin):** يمتلك صلاحيات إدارية على مستوى النظام لإدارة الجوانب العامة ومتابعة حالته، مع بقاء الوصول إلى محتوى القنوات الخاصة خاضعاً لقواعد العضوية والصلاحيات المطبقة عليها.

3.3- مخطط حالات الاستخدام

يوضح مخطط حالات الاستخدام (Use Case Diagram) التفاعلات الأساسية بين الممثلين والنظام، ويبين الوظائف المتاحة لكل منهم وفق الصلاحيات الممنوحة له. وتشمل حالات الاستخدام الرئيسية إنشاء الحساب وتسجيل الدخول، وإدارة القنوات والعضويات، ونشر الرسائل واستقبالها، وإدارة الصلاحيات، إضافة إلى الوظائف الإدارية الخاصة بمتابعة النظام. ويوضح الشكل رقم (1) حالات الاستخدام الرئيسية والعلاقات بين الممثلين والوظائف التي يقدمها النظام.



الشكل رقم 1 مخطط حالات الاستخدام الرئيسية في النظام

4.3- السرد النصي لحالات الاستخدام

1.4.3- تسجيل الدخول

العنصر	الوصف
اسم الحالة	تسجيل الدخول
الوصف Description	يقوم المستخدم بتسجيل الدخول إلى حساب موجود مسبقاً للوصول إلى وظائف النظام.
الفاعلون Actors	المستخدم
الشروط السابقة Precondition	أن يمتلك المستخدم حساباً فعالاً مسجلاً مسبقاً في النظام.
الشروط اللاحقة Postcondition	يتم تسجيل دخول المستخدم وإنشاء جلسة مصادقة له، ثم توجيهه إلى الواجهة الرئيسية.

الجدول رقم 1 بيانات حالة استخدام تسجيل الدخول

سير الأحداث

السيناريو الأساسي الناجح

المستخدم	النظام
1. يدخل المستخدم إلى صفحة تسجيل الدخول.	
2. يعرض النظام نموذج تسجيل الدخول ويطلب بيانات الحساب وكلمة المرور.	
3. يدخل المستخدم البيانات المطلوبة ويضغط على زر تسجيل الدخول.	
4. يتحقق النظام من صحة بيانات الحساب وكلمة المرور.	
5. ينشئ النظام جلسة للمستخدم ويصدر رموز المصادقة اللازمة.	
6. يوجه النظام المستخدم إلى الواجهة الرئيسية.	

الجدول رقم 2 سير الأحداث لحالة استخدام تسجيل الدخول

المسارات البديلة

لا يوجد.

مسارات الأخطاء

E1: في الخطوة رقم (4)، إذا كانت بيانات الحساب أو كلمة المرور غير صحيحة، يعرض النظام رسالة توضح تعذر تسجيل الدخول ويطلب من المستخدم إعادة إدخال البيانات.

E2: إذا كان الحساب غير فعال، يرفض النظام عملية تسجيل الدخول ويعرض رسالة مناسبة للمستخدم.

E3: إذا تجاوز المستخدم عدد محاولات تسجيل الدخول المسموح بها، يرفض النظام المحاولة مؤقتاً وفق القيود الأمنية المطبقة.

2.4.3 - إنشاء قناة

العنصر	الوصف
اسم الحالة	إنشاء قناة
الوصف Description	يقوم المستخدم بإنشاء قناة جديدة لاستخدامها في نشر الرسائل والتواصل مع المشتركين فيها.
الفاعلون Actors	المستخدم
الشروط السابقة Precondition	أن يكون المستخدم مسجلاً للدخول في النظام.
الشروط اللاحقة Postcondition	يتم إنشاء القناة وإضافتها إلى النظام، ويصبح المستخدم الذي أنشأها مالكاً لها.

الجدول رقم 3 بيانات حالة استخدام إنشاء قناة

سير الأحداث

السيناريو الأساسي الناجح

المستخدم	النظام
1. يختار المستخدم إنشاء قناة جديدة.	
	2. يعرض النظام نموذج إنشاء القناة.
3. يدخل المستخدم اسم القناة والمعلومات المطلوبة ويحدد نوعها.	
	4. يتحقق النظام من صحة البيانات المدخلة.
5. يضغط المستخدم على زر إنشاء القناة.	
	6. ينشئ النظام القناة ويسجل المستخدم كمالك لها.
	7. يعرض النظام القناة الجديدة للمستخدم.

الجدول رقم 4 سير الأحداث لحالة استخدام إنشاء قناة

المسارات البديلة

لا يوجد.

مسارات الأخطاء

E1: في الخطوة رقم (4)، إذا كانت البيانات المدخلة غير صحيحة أو ناقصة، يرفض النظام إنشاء القناة ويعرض رسالة توضح البيانات المطلوب تصحيحها.

E2: إذا تعذر إنشاء القناة نتيجة خطأ في النظام، يتم إبلاغ المستخدم بفشل العملية دون إنشاء قناة غير مكتملة.

3.4.3- الانضمام إلى القناة

جدول: حالة استخدام الانضمام إلى قناة

العنصر	الوصف
اسم الحالة	الانضمام إلى قناة
الوصف Description	يقوم المستخدم بالانضمام إلى إحدى القنوات المتاحة ليتمكن من استعراض محتواها والمشاركة فيها وفق الصلاحيات الممنوحة له.
الفاعلون Actors	المستخدم
الشروط السابقة Precondition	أن يكون المستخدم مسجلاً للدخول وألا يكون عضواً في القناة مسبقاً.
الشروط اللاحقة Postcondition	يصبح المستخدم عضواً في القناة مباشرة أو يتم تسجيل طلب انضمامه بانتظار الموافقة، بحسب سياسة القناة.

الجدول رقم 5 بيانات حالة استخدام الانضمام إلى قناة

سير الأحداث

السيناريو الأساسي الناجح

المستخدم	النظام
1. يستعرض المستخدم القنوات المتاحة.	
2. يعرض النظام القنوات التي يمكن للمستخدم الوصول إليها.	
3. يختار المستخدم القناة التي يريد الانضمام إليها.	
4. يعرض النظام معلومات القناة وإمكانية الانضمام إليها.	
5. يضغط المستخدم على زر الانضمام.	
6. يتحقق النظام من إمكانية انضمام المستخدم إلى القناة.	
7. يضيف النظام المستخدم إلى أعضاء القناة ويعرض محتواها له.	

الجدول رقم 6 سير الأحداث لحالة استخدام الانضمام إلى قناة

المسارات البديلة

A1: إذا كانت القناة تتطلب موافقة على الانضمام، يقوم النظام في الخطوة رقم (7) بإنشاء طلب انضمام بدلاً من إضافة المستخدم مباشرة، ويصبح الطلب بانتظار موافقة مدير القناة.

مسارات الأخطاء

- E1:** إذا كان المستخدم عضواً في القناة مسبقاً، لا ينشئ النظام عضوية جديدة ويعرض القناة للمستخدم.
- E2:** إذا لم يكن الانضمام إلى القناة مسموحاً للمستخدم، يرفض النظام العملية ويعرض رسالة مناسبة.
- E3:** إذا تعذر تنفيذ العملية نتيجة خطأ في النظام، لا يتم إنشاء عضوية غير مكتملة ويتم إبلاغ المستخدم بفشل العملية.

4.4.3- نشر رسالة في قناة

العنصر	الوصف
اسم الحالة	نشر رسالة في قناة
الوصف Description	يقوم المستخدم بنشر رسالة ضمن إحدى القنوات التي يشترك فيها، ليتم حفظها وتوزيعها على المشتركين في القناة.
الفاعلون Actors	المستخدم
الشروط السابقة Precondition	أن يكون المستخدم مسجلاً للدخول وعضواً في القناة ويمتلك صلاحية النشر فيها.
الشروط اللاحقة Postcondition	يتم حفظ الرسالة في النظام وتسجيلها ضمن القناة، ثم تبدأ عملية توزيعها على المشتركين.

الجدول رقم 7 بيانات حالة استخدام نشر رسالة في قناة

سير الأحداث

السيناريو الأساسي الناجح

المستخدم	النظام
1. يفتح المستخدم القناة التي يريد النشر فيها.	
	2. يعرض النظام رسائل القناة وواجهة كتابة الرسالة.
3. يكتب المستخدم محتوى الرسالة.	
4. يضغط المستخدم على زر الإرسال.	
	5. يتحقق النظام من عضوية المستخدم وصلاحيته للنشر ومن صحة محتوى الرسالة.
	6. يحفظ النظام الرسالة ويربطها بالقناة والمستخدم الناشر.
	7. يبدأ النظام عملية توزيع الرسالة على المشتركين في القناة.
	8. يعرض النظام الرسالة الجديدة ضمن القناة.

الجدول رقم 8 سير الأحداث لحالة استخدام نشر رسالة في قناة

المسارات البديلة

A1: يمكن للمستخدم إرفاق ملف مع الرسالة، وفي هذه الحالة يتحقق النظام من المرفق ويحفظ بياناته ويربطه بالرسالة قبل إتمام عملية النشر.

مسارات الأخطاء

- E1:** إذا لم يكن المستخدم عضواً في القناة أو لم يمتلك صلاحية النشر، يرفض النظام العملية ولا يتم حفظ الرسالة.
- E2:** إذا كان محتوى الرسالة غير صالح، يرفض النظام عملية النشر ويعرض رسالة مناسبة للمستخدم.
- E3:** إذا تعذر حفظ الرسالة نتيجة خطأ في النظام، لا يتم اعتبار عملية النشر ناجحة ويتم إبلاغ المستخدم بفشل العملية.

5.4.3- استعراض رسائل القناة

العنصر	الوصف
اسم الحالة	استعراض رسائل القناة
الوصف Description	يقوم المستخدم باستعراض الرسائل المنشورة ضمن قناة يشترك فيها، بما في ذلك الرسائل السابقة المحفوظة في النظام.
الفاعلون Actors	المستخدم
الشروط السابقة Precondition	أن يكون المستخدم مسجلاً للدخول ويملك صلاحية الوصول إلى القناة.
الشروط اللاحقة Postcondition	يتم عرض رسائل القناة للمستخدم وفقاً لصلاحياته وحالة القناة.

الجدول رقم 9 بيانات حالة استخدام استعراض رسائل القناة

سير الأحداث

السيناريو الأساسي الناجح

المستخدم	النظام
1. يختار المستخدم إحدى القنوات التي يشترك فيها.	
	2. يتحقق النظام من عضوية المستخدم وصلاحيته للوصول إلى القناة.
	3. يجلب النظام الرسائل المحفوظة الخاصة بالقناة.
	4. يعرض النظام الرسائل للمستخدم بترتيبها داخل واجهة القناة.
5. يستعرض المستخدم الرسائل الموجودة في القناة.	
	6. يعرض النظام الرسائل الجديدة للمستخدم عند وصولها أثناء وجوده في القناة.

الجدول رقم 10 سير الأحداث لحالة استخدام استعراض رسائل القناة

المسارات البديلة

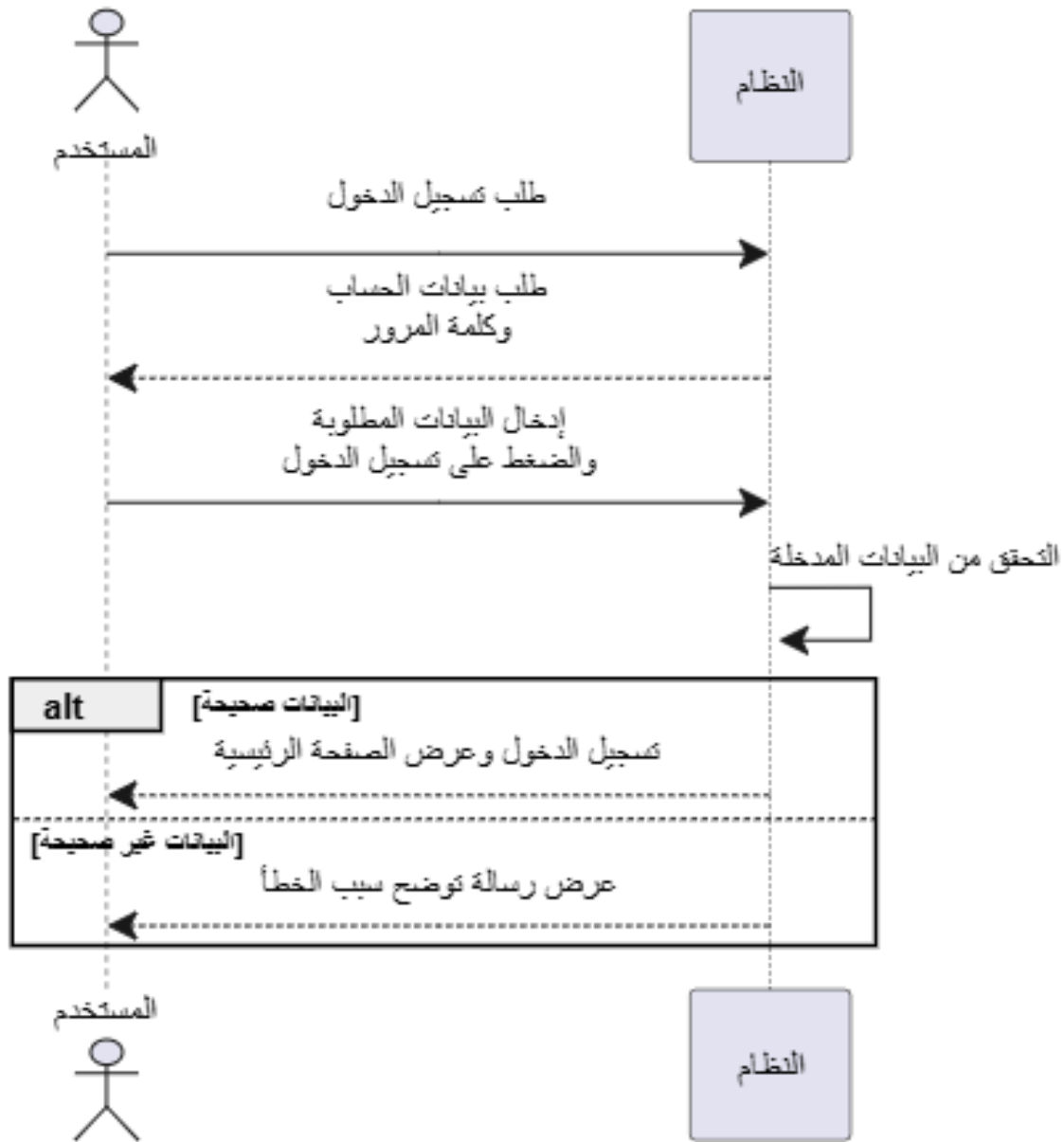
A1: إذا كان المستخدم قد انقطع عن النظام لفترة، يقوم النظام عند فتح القناة بعرض الرسائل التي تم نشرها أثناء فترة انقطاعه ضمن الرسائل المحفوظة.

مسارات الأخطاء

- E1:** إذا لم يكن المستخدم عضواً في القناة أو لم يمتلك صلاحية الوصول إليها، يرفض النظام عرض محتوى القناة.
- E2:** إذا تعذر جلب الرسائل نتيجة خطأ في النظام، يعرض النظام رسالة مناسبة للمستخدم دون عرض بيانات غير مكتملة أو غير صحيحة.

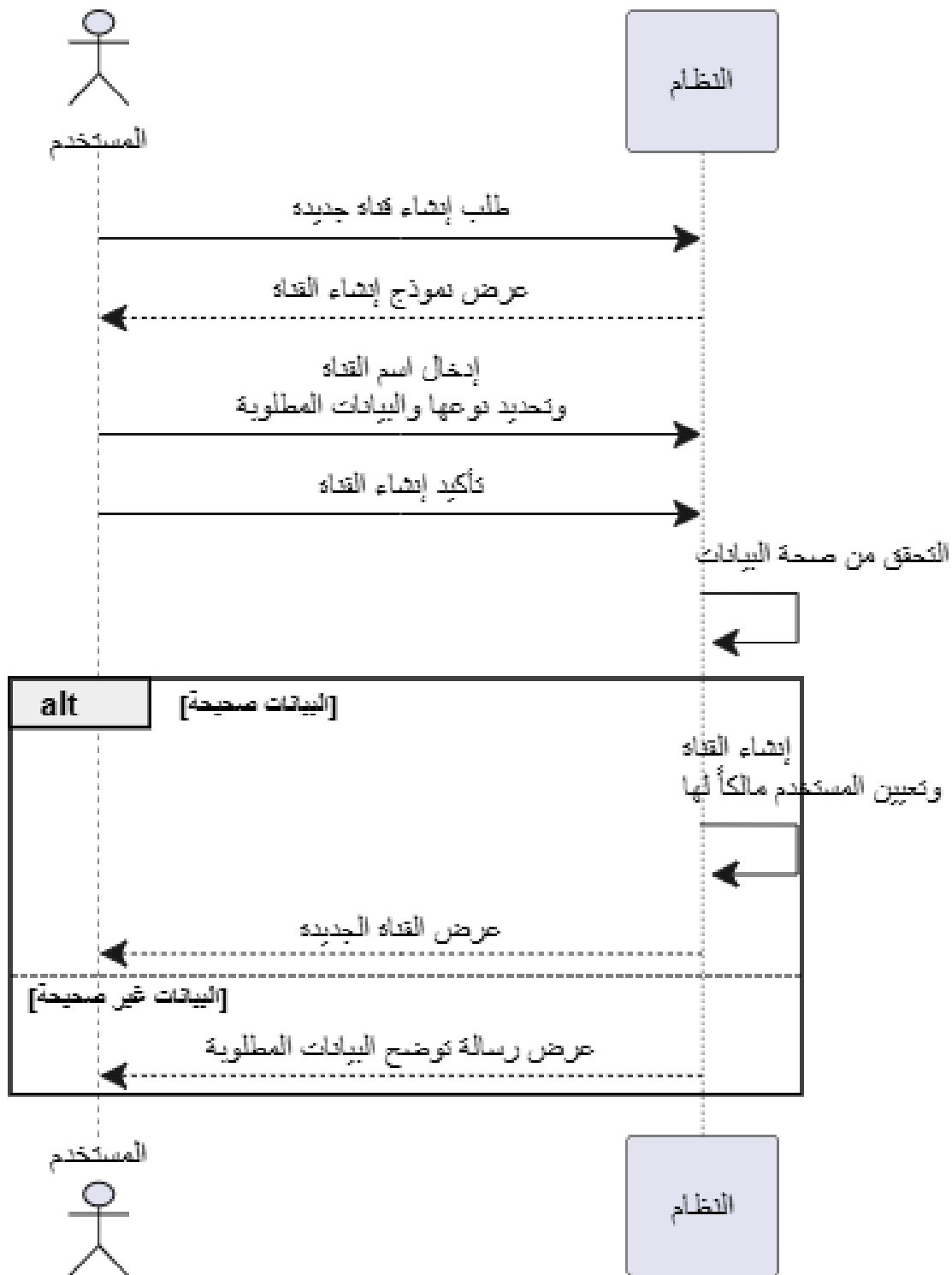
5.3 - مخططات تسلسل النظام

1.5.3 - تسجيل الدخول



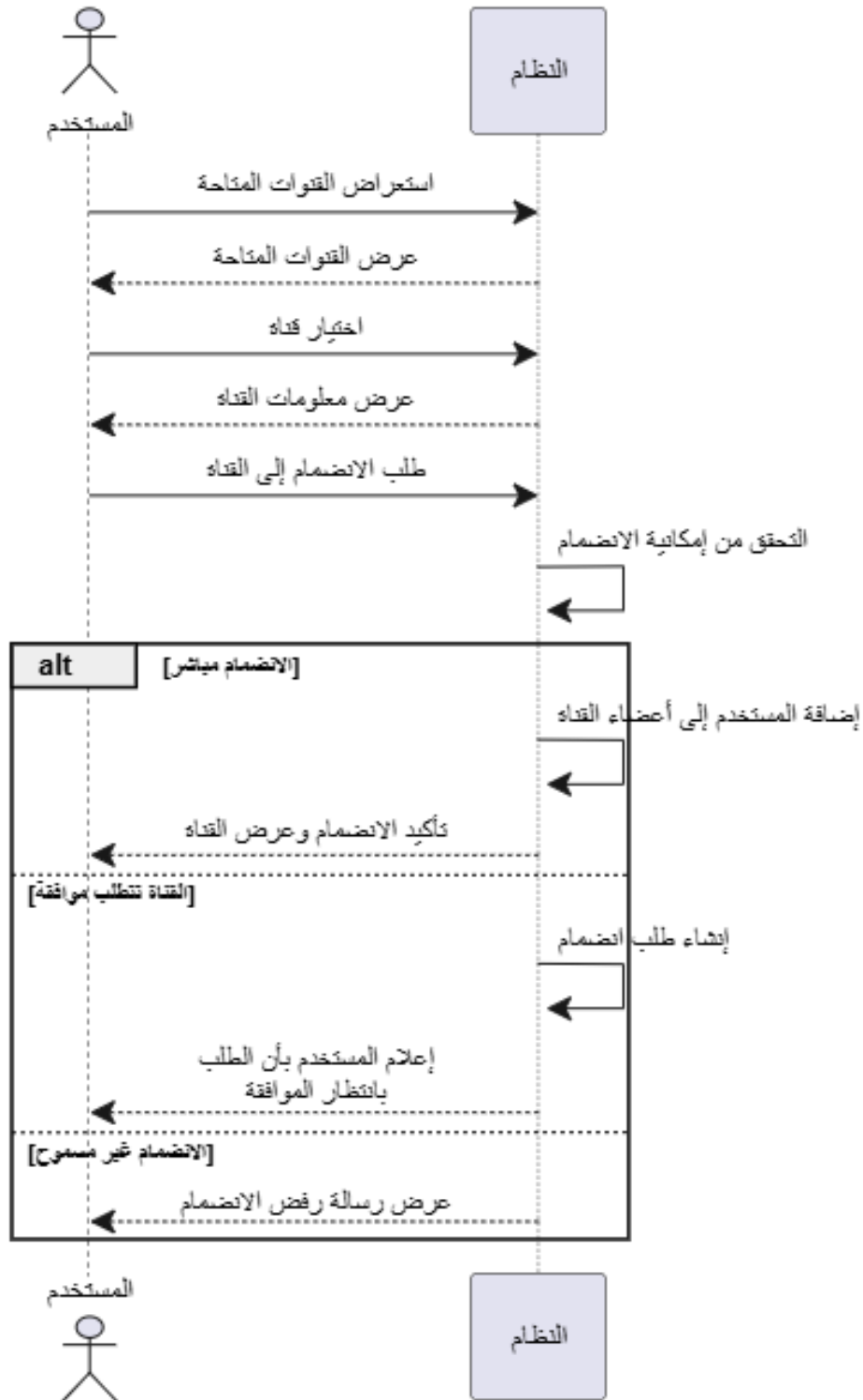
الشكل رقم 2 مخطط تسلسل عملية تسجيل الدخول

2.5.3 - إنشاء القناة



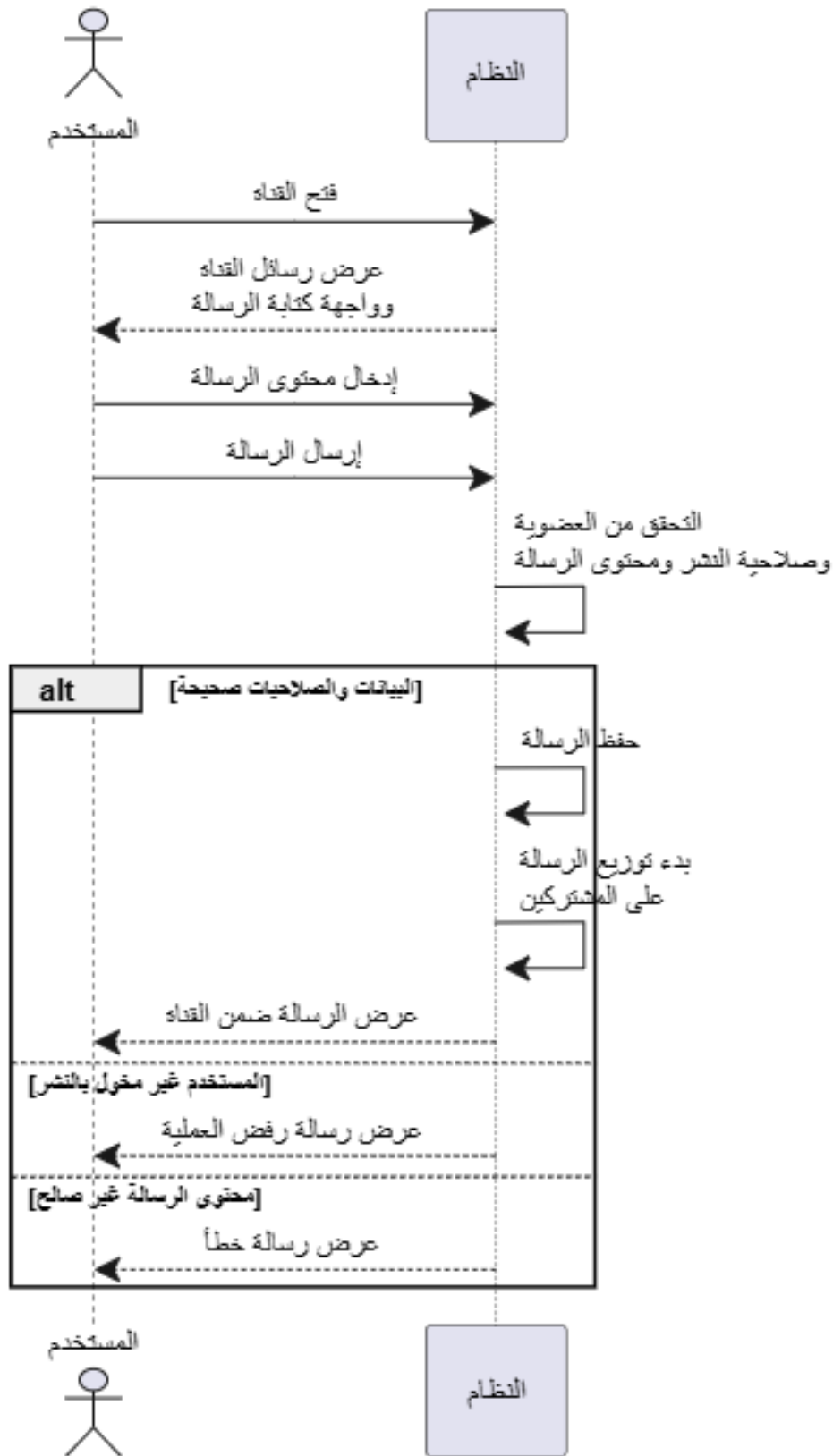
الشكل رقم 3 مخطط تسلسل عملية إنشاء قناة جديدة

3.5.3 - الانضمام إلى قناة



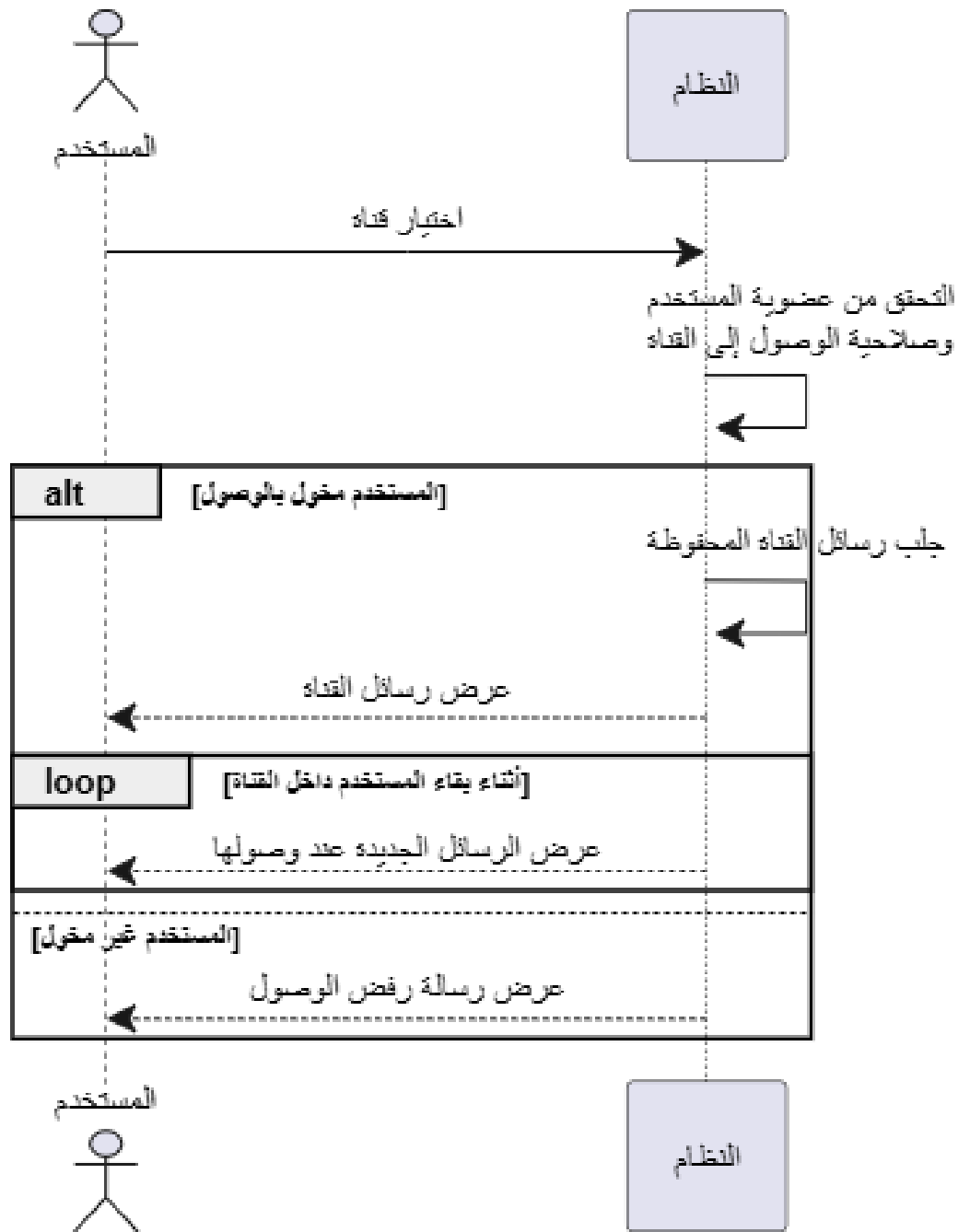
الشكل رقم 4 مخطط تسلسل عملية الانضمام إلى قناة

4.5.3- نشر رسالة في قناة



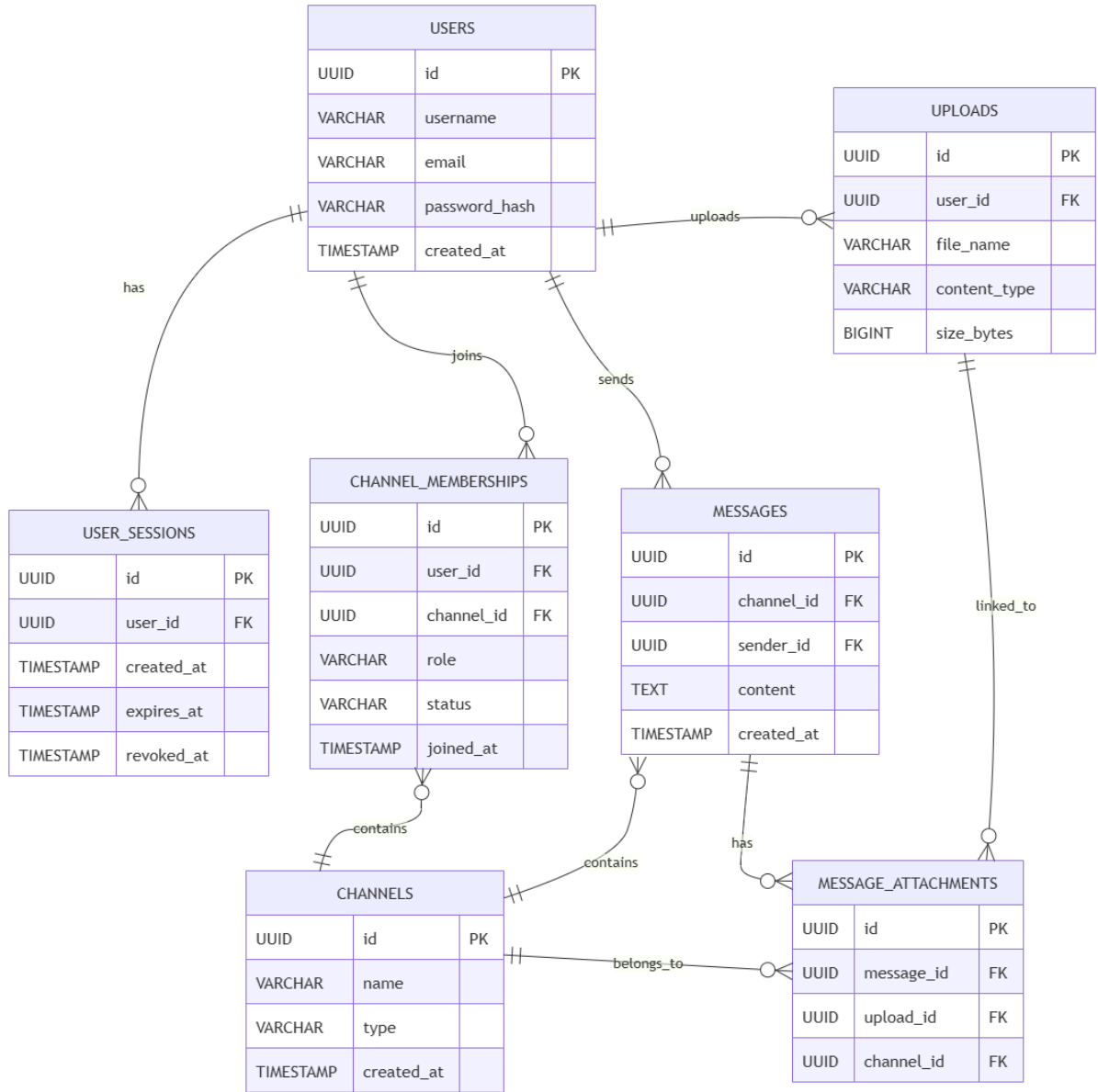
الشكل رقم 5 مخطط تسلسل عملية نشر رسالة في قناة

5.5.3- استعراض رسائل القناة



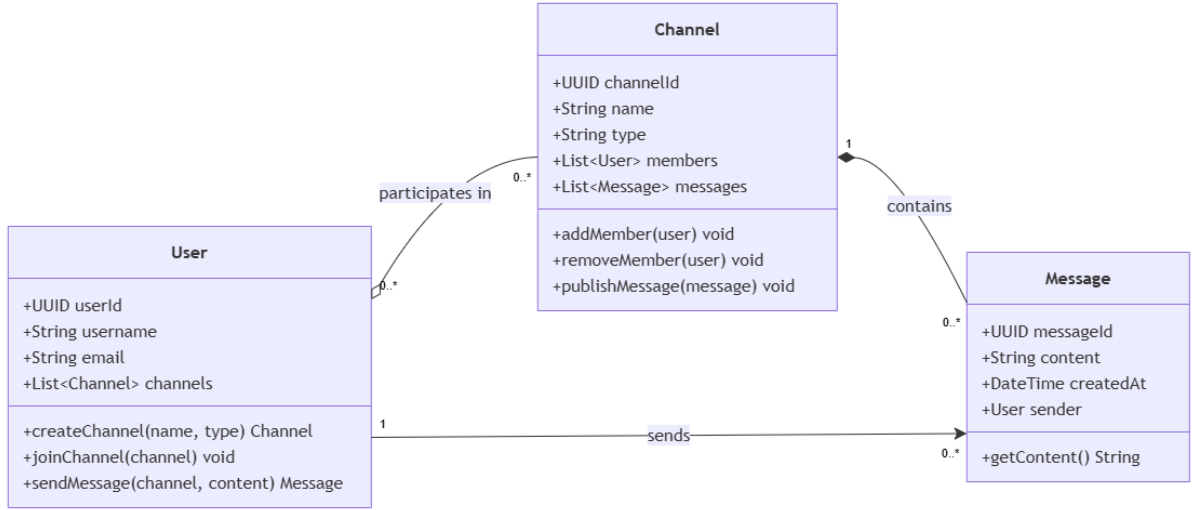
الشكل رقم 6 مخطط تسلسل عملية استعراض رسائل القناة

6.3 - مخطط علاقات الكيانات ERD



الشكل رقم 7 مخطط علاقات الكيانات الرئيسة في النظام (ERD)

7.3 - مخطط الصفوف



الشكل رقم 8 مخطط الصفوف الرئيسة في النظام

8.3- خلاصة

تم في هذا الفصل تحليل النظام من الناحية الوظيفية من خلال تحديد الممثلين الرئيسيين وحالات الاستخدام التي توضح أهم العمليات التي ينفذها المستخدم، مثل تسجيل الدخول وإنشاء القنوات والانضمام إليها ونشر الرسائل واستعراضها. كما تم توضيح سير هذه العمليات باستخدام مخططات تسلسل النظام، إضافة إلى عرض مخطط علاقات الكيانات ومخطط الصفوف لتوضيح البنية العامة للبيانات والكائنات الرئيسة في النظام والعلاقات بينها.

وتوفر هذه الدراسة تصوراً واضحاً عن سلوك النظام ومتطلباته الأساسية، وتشكل أساساً للانتقال إلى دراسة المفاهيم والتقنيات النظرية التي يعتمد عليها النظام في عمليات النشر والاشتراك وتوزيع الرسائل والموثوقية والأمان.

الفصل الرابع

الدراسة النظرية

غير ضرورية الآن.

الفصل الخامس

الدراسة التصميمية

يتناول هذا الفصل التصميم المعماري للنظام، وتقسيم مسؤوليات مكوناته، بالإضافة إلى تصميم مسار نشر الرسائل واستقبالها، وآليات الموثوقية والأمان المعتمدة في التصميم.

1.5 - المقدمة

تهدف الدراسة التصميمية إلى تحويل المتطلبات والمفاهيم التي تم تحديدها في الفصول السابقة إلى بنية واضحة تحدد مكونات النظام ومسؤولية كل مكون والعلاقات التي تربط بينها. ويكتسب هذا الجانب أهمية خاصة في الأنظمة الموزعة، حيث تتوزع عمليات التخزين والتوزيع والاتصال اللحظي والمصادقة على عدة مكونات مترابطة.

يتناول هذا الفصل التصميم العام لنظام المراسلة، بدءاً من المعمارية العامة ومكونات النظام الرئيسية، ثم تصميم الواجهة الخلفية وقاعدة البيانات ومسار نشر الرسائل وتوزيعها. كما يتم توضيح آليات الاتصال اللحظي والمزامنة وإدارة الملفات والمرفقات، إضافة إلى تصميم المصادقة والصلاحيات وحماية البيانات وسجل الأحداث، وصولاً إلى تصميم الواجهة الأمامية وبيئة تشغيل النظام.

ويهدف هذا التصميم إلى تحقيق فصل واضح للمسؤوليات بين المكونات، مع المحافظة على موثوقية الرسائل وإمكانية استرجاعها والتوسع في النظام مستقبلاً.

2.5 - منهجية التصميم

تم تصميم النظام وفق مبدأ فصل المسؤوليات (Separation of Concerns)، بحيث لا تتولى جهة واحدة تنفيذ جميع عمليات المراسلة والتخزين والتوزيع والاتصال بالمستخدمين، وإنما تم تقسيم النظام إلى مجموعة من المكونات المتخصصة التي تتعاون فيما بينها.

تعتمد الواجهة الأمامية على الواجهة الخلفية لتنفيذ العمليات الأساسية والتحقق من هوية المستخدم وصلاحياته. وتتولى الواجهة الخلفية معالجة طلبات المستخدمين وتطبيق قواعد العمل، بينما تستخدم قاعدة البيانات PostgreSQL كمصدر دائم لحفظ المستخدمين والقنوات والعضويات والرسائل والأحداث وحالات عمليات التوزيع.

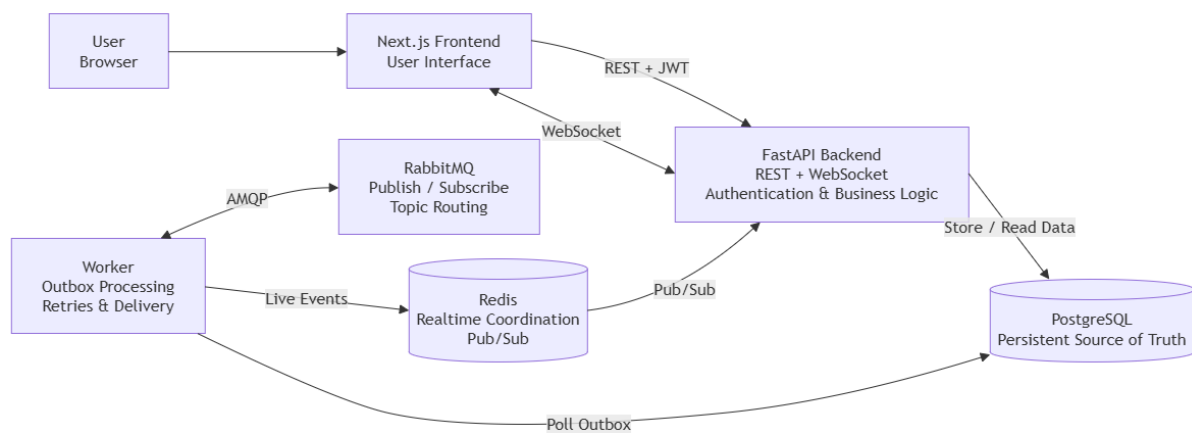
أما توزيع الرسائل بصورة غير متزامنة فيتم من خلال **RabbitMQ**، حيث يتم فصل عملية قبول الرسالة وحفظها عن عملية توزيعها. ويتولى مكوّن مستقل (**Worker**) معالجة مهام التوزيع والتعامل مع وسيط الرسائل، مما يقلل من ارتباط الطلب المباشر للمستخدم بعملية التسليم.

كما تم فصل مسار الاتصال اللحظي عن التخزين الدائم، حيث يستخدم **Redis** و **WebSocket** لدعم إيصال الأحداث الجديدة إلى المستخدمين المتصلين، بينما تبقى الرسائل المخزنة في قاعدة البيانات قابلة للاسترجاع والمزامنة عند إعادة الاتصال.

يسمح هذا التصميم لكل مكوّن بأداء وظيفة محددة، ويساعد على تحسين قابلية الصيانة والتوسع ومعالجة حالات الفشل، مع المحافظة على قاعدة البيانات كمصدر دائم للحقيقة وعدم الاعتماد على المكونات اللحظية وحدها لحفظ بيانات النظام.

3.5- المعمارية العامة للنظام

تم تصميم النظام وفق معمارية موزعة تتكون من عدة مكونات مستقلة، بحيث يتولى كل مكوّن مجموعة محددة من المسؤوليات. وتتفاعل هذه المكونات من خلال واجهات وبروتوكولات اتصال مختلفة لتحقيق التخزين الدائم، والتوزيع غير المتزامن للرسائل، والاتصال اللحظي بالمستخدمين. ويوضح الشكل رقم (9) المعمارية العامة للنظام والعلاقات بين مكوناته الرئيسية.



الشكل رقم 9 المعمارية العامة لنظام المراسلة الموزع

تمثل الواجهة الأمامية المبنية باستخدام **Next.js** نقطة تفاعل المستخدم مع النظام، وتتواصل مع الواجهة الخلفية المبنية باستخدام **FastAPI** من خلال طلبات REST لتنفيذ العمليات الأساسية، إضافة إلى اتصال **WebSocket** المستخدم لاستقبال الأحداث والرسائل الجديدة بصورة لحظية.

تتولى الواجهة الخلفية عمليات المصادقة والتحقق من الصلاحيات وتطبيق قواعد العمل، كما تتعامل مع قاعدة بيانات **PostgreSQL** لحفظ المستخدمين والقنوات والعضويات والرسائل والأحداث وبيانات عمليات التوزيع. وتعد

PostgreSQL المصدر الدائم للحقيقة في النظام، بحيث يمكن الرجوع إليها لاسترجاع البيانات حتى عند حدوث فشل في المكونات الأخرى.

تتم معالجة توزيع الرسائل بصورة مستقلة عن طلب المستخدم من خلال مكوّن **Worker**، حيث يقرأ سجلات التوزيع المحفوظة في قاعدة البيانات، ثم يقوم بنشر الرسائل إلى **RabbitMQ** الذي يتولى توجيهها إلى طوابير المشتركين المناسبة. ويتعامل العامل أيضاً مع عمليات إعادة المحاولة وحالات فشل التوزيع.

أما **Redis** فيستخدم كطبقة سريعة لدعم التوزيع اللحظي والتنسيق بين عمليات **WebSocket**، حيث يتم تمرير الأحداث الخاصة بالمستخدمين المتصلين إلى خوادم الواجهة الخلفية، والتي تقوم بدورها بإيصالها إلى الواجهة الأمامية من خلال اتصال **WebSocket**.

يسمح هذا التصميم بالفصل بين التخزين الدائم، والتوزيع الموثوق، والاتصال اللحظي، بحيث لا يؤدي تعطل أحد المسارات المؤقتة إلى فقدان البيانات المحفوظة، كما يسهل تطوير مكونات النظام وتوسيعها بصورة مستقلة.

4.5 - مكونات النظام ومسؤولياتها

يتكون النظام من مجموعة من المكونات التي تتكامل فيما بينها لتنفيذ عمليات المراسلة والتخزين والتوزيع والاتصال اللحظي. وقد تم توزيع المسؤوليات بينها بحيث يؤدي كل مكوّن وظيفة محددة ضمن المعمارية العامة للنظام.

الواجهة الأمامية (Frontend):

تمثل نقطة تفاعل المستخدم مع النظام، وتوفر الواجهات اللازمة لتسجيل الدخول وإدارة القنوات والعضويات وعرض الرسائل ونشرها. وتتواصل مع الواجهة الخلفية باستخدام طلبات **REST**، كما تستخدم اتصال **WebSocket** لاستقبال الرسائل والأحداث الجديدة بصورة لحظية.

الواجهة الخلفية (backend):

تمثل الجزء المسؤول عن معالجة طلبات المستخدم وتطبيق قواعد العمل في النظام. وتشمل مسؤولياتها المصادقة وإدارة الجلسات، والتحقق من العضويات والصلاحيات، وإدارة القنوات والرسائل والملفات، إضافة إلى التعامل مع قاعدة البيانات وتوفير واجهات **REST** و **WebSocket** للواجهة الأمامية.

قاعدة البيانات PostgreSQL:

تمثل مصدر التخزين الدائم والمصدر الأساسي للحقيقة في النظام. وتحتفظ ببيانات المستخدمين والجلسات والقنوات والعضويات والرسائل والمرفقات والأحداث، إضافة إلى السجلات اللازمة لمتابعة عمليات توزيع الرسائل. ويتيح هذا التخزين استرجاع البيانات حتى عند توقف مكونات التوزيع أو الاتصال اللحظي بصورة مؤقتة.

مكوّن Worker:

يعمل بصورة مستقلة عن معالجة طلبات المستخدمين، ويتولى تنفيذ العمليات غير المتزامنة المرتبطة بتوزيع الرسائل. ويقوم بمعالجة سجلات Outbox ونشر مهام التوزيع إلى وسيط الرسائل، بالإضافة إلى التعامل مع الرسائل المستلمة وحالات إعادة المحاولة والفشل.

:RabbitMQ

يعمل كوسيط رسائل مسؤول عن التوزيع غير المتزامن وتوجيه الرسائل إلى الوجهات المناسبة. ويساعد استخدامه على فصل عملية نشر الرسالة عن عملية إيصالها إلى المشتركين، كما يوفر آليات تساعد على إدارة الطوابير والتأكيد وإعادة المحاولة عند حدوث فشل مؤقت.

:Redis

يستخدم كطبقة سريعة لدعم العمليات اللحظية والتنسيق بين مكونات النظام. ويتم الاستفادة منه في تمرير الأحداث المتعلقة بالمستخدمين المتصلين إلى خوادم WebSocket، بالإضافة إلى بعض البيانات المؤقتة اللازمة لإدارة الاتصالات والعمليات ذات العمر القصير.

:WebSocket

يوفر قناة اتصال مستمرة وثنائية الاتجاه بين الواجهة الأمامية والواجهة الخلفية، ويستخدم لإيصال الرسائل والأحداث الجديدة إلى المستخدمين أثناء اتصالهم بالنظام دون الحاجة إلى تنفيذ طلبات متكررة للحصول على التحديثات. ويحقق هذا التقسيم فصلاً واضحاً بين مسؤوليات التخزين والمعالجة والتوزيع والاتصال اللحظي، كما يسمح لكل مكون بأن يتطور أو يتوسع بصورة مستقلة مع المحافظة على التكامل بين أجزاء النظام.

5.5 - تصميم الواجهة الخلفية

تم تصميم الواجهة الخلفية باستخدام FastAPI وفق بنية تفصل بين استقبال طلبات المستخدم وتنفيذ قواعد العمل والتعامل مع البيانات. ويهدف هذا الفصل بين الطبقات إلى تقليل الترابط بين أجزاء التطبيق وتسهيل تطوير كل جزء واختباره بصورة مستقلة.

تمثل طبقة **API Routes** نقطة الدخول للطلبات القادمة من الواجهة الأمامية، حيث تستقبل البيانات وتتحقق من صحة الطلب والمستخدم المصادق عليه، ثم تحول تنفيذ العملية إلى الخدمة المناسبة بدلاً من وضع جميع قواعد العمل داخل المسار نفسه.

تحتوي طبقة **Services** على المنطق الأساسي للنظام، مثل عمليات المصادقة وإدارة القنوات والعضويات والرسائل والملفات وسجل الأحداث. وتتولى هذه الخدمات التحقق من القواعد والصلاحيات المطلوبة، ثم تنفيذ العمليات اللازمة باستخدام طبقة البيانات أو الخدمات المساندة.

أما طبقة البيانات فتتكون من نماذج قاعدة البيانات والجلسات المستخدمة للتعامل مع PostgreSQL، حيث تمثل النماذج الجداول والعلاقات الموجودة بينها. ويستخدم النظام عمليات قاعدة بيانات غير متزامنة بما يتناسب مع طبيعة FastAPI ويسمح للخادم بمعالجة عدد أكبر من الطلبات دون حجز مسار التنفيذ أثناء انتظار عمليات الإدخال والإخراج. كما تتكامل الواجهة الخلفية مع المكونات الخارجية من خلال وحدات مخصصة، مثل Redis لدعم العمليات اللحظية، وخدمات WebSocket لإدارة الاتصالات المستمرة، بينما يتم فصل المعالجة الخاصة بتوزيع الرسائل عبر RabbitMQ إلى مكوّن Worker مستقل عن خادم الـ API.

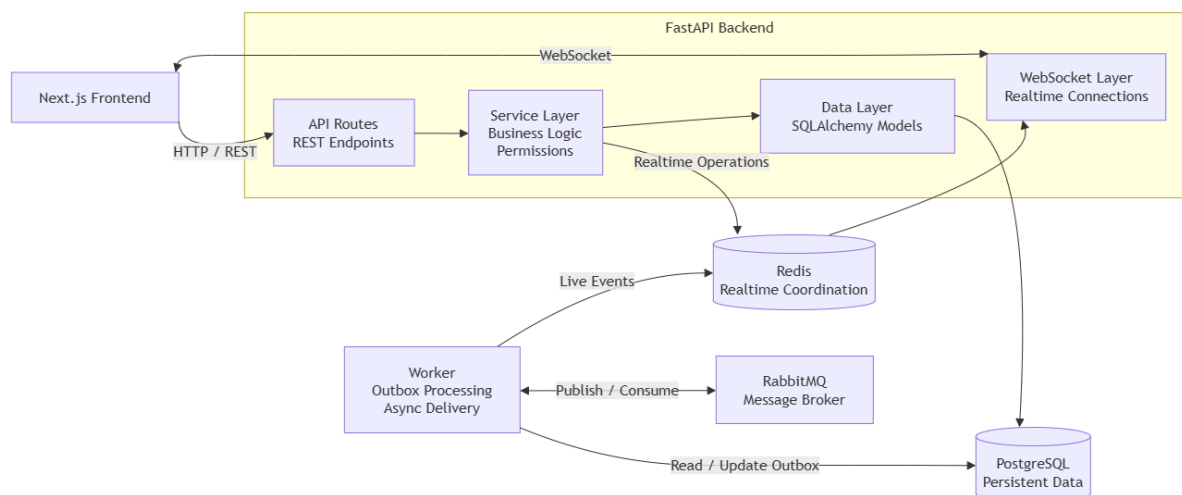
ويمكن تلخيص البنية الداخلية للواجهة الخلفية على النحو الآتي:

API Routes → Services → Database Models / Supporting Components

يساعد هذا التنظيم على إبقاء مسارات الـ API بسيطة، وتجميع قواعد العمل في طبقة الخدمات، وفصل تفاصيل التخزين والاتصال بالمكونات الخارجية عن واجهات المستخدم.

6.5 - مخطط الواجهة الخلفية

يوضح مخطط مكونات الواجهة الخلفية طريقة تقسيم المسؤوليات داخل النظام والعلاقات بين الطبقات المختلفة. تبدأ معالجة الطلبات من طبقة **API Routes** التي تستقبل طلبات الواجهة الأمامية، ثم تنتقل إلى طبقة **Services** المسؤولة عن تنفيذ قواعد العمل والتحقق من الصلاحيات، وبعد ذلك يتم التعامل مع طبقة البيانات والخدمات المساندة وفق طبيعة العملية المطلوبة. ويوضح الشكل رقم (10) هذا التقسيم الداخلي ومكونات الواجهة الخلفية الرئيسية.



الشكل رقم 10 البنية الداخلية للواجهة الخلفية ومكوناتها الرئيسية

تتعامل طبقة الخدمات مع نماذج البيانات وقاعدة PostgreSQL لتنفيذ عمليات التخزين والاسترجاع، كما تتكامل مع Redis ومكوّنات WebSocket لدعم العمليات اللحظية. أما العمليات غير المتزامنة المتعلقة بتوزيع الرسائل، فيتم فصلها

عن مسار طلبات المستخدم من خلال مكوّن **Worker** الذي يتعامل مع سجلات Outbox وقاعدة البيانات وRabbitMQ، ثم يمرر الأحداث اللحظية عند الحاجة عبر Redis.

يساعد هذا التصميم على فصل معالجة طلب المستخدم عن عمليات التوزيع الخلفية، كما يمنع تراكم منطق النظام داخل مسارات API ويجعل كل طبقة مسؤولة عن مجموعة محددة من المهام.

7.5 - تصميم قاعدة البيانات

تم تصميم قاعدة البيانات باستخدام PostgreSQL لتكون المصدر الدائم والأساسي لبيانات النظام، بحيث لا يعتمد حفظ الرسائل أو العضويات أو حالة القنوات على المكونات المؤقتة مثل Redis أو RabbitMQ. ويسمح ذلك باسترجاع الحالة الصحيحة للنظام حتى بعد حدوث انقطاع أو إعادة تشغيل لأحد المكونات.

تم تقسيم البيانات إلى مجموعة من الجداول المترابطة وفق طبيعة الكيانات التي يتعامل معها النظام. يحتوي جدول المستخدمين على بيانات الحسابات، وترتبط به جداول الجلسات المستخدمة في إدارة عمليات تسجيل الدخول. كما يتم تخزين القنوات بصورة مستقلة، بينما يستخدم جدول العضويات لربط المستخدمين بالقنوات وتحديد دور وحالة كل مستخدم ضمن القناة. يتم تخزين الرسائل في جدول مستقل وربط كل رسالة بالقناة والجهة التي قامت بنشرها، بينما تستخدم جداول إضافية لإدارة المرفقات والتفاعلات والرسائل المثبتة وحالة المستخدم داخل القنوات. ويساعد هذا الفصل على تجنب تكرار البيانات وإتاحة إدارة كل جزء بصورة مستقلة.

كما تتضمن قاعدة البيانات جداول مخصصة للعمليات الداخلية للنظام، ومن أهمها جدول Outbox الذي يحتفظ بمهام التوزيع قبل إرسالها إلى RabbitMQ، بالإضافة إلى جداول تسجيل الأحداث والبيانات المستخدمة للتحقق من سلامة سجل الأحداث.

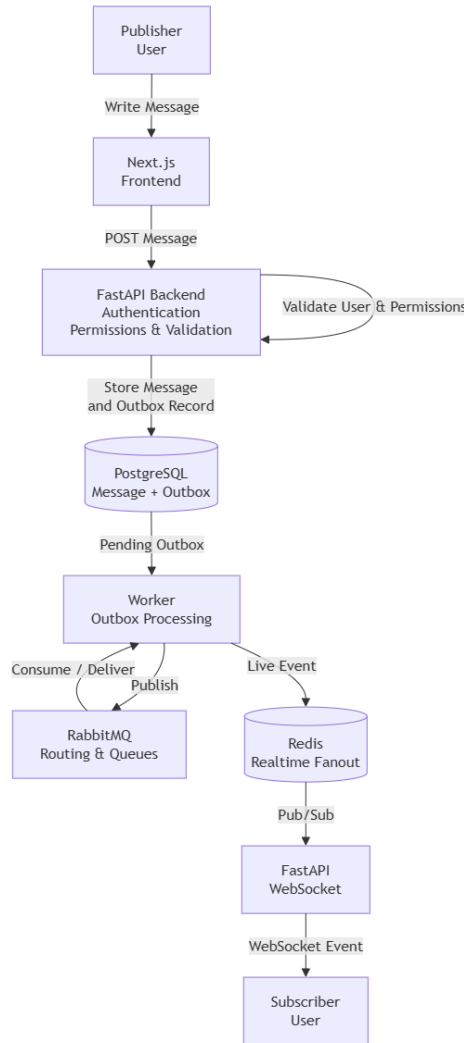
تم استخدام المفاتيح الأساسية (Primary Keys) لتعريف السجلات بصورة فريدة، والمفاتيح الأجنبية (Foreign Keys) لتمثيل العلاقات بين الجداول والمحافظة على التكامل المرجعي للبيانات. كما تم الاعتماد على القيود المناسبة لمنع إنشاء علاقات أو سجلات غير متناسقة.

ويحقق هذا التصميم فصلاً بين بيانات العمل الأساسية، مثل المستخدمين والقنوات والرسائل، وبين البيانات المساندة لعمليات التوزيع والأمان والتدقيق، مع المحافظة على قاعدة البيانات كمصدر موثوق يمكن الاعتماد عليه في عمليات المزامنة والاسترجاع.

8.5 - تصميم مسار نشر الرسالة وتوزيعها

تم تصميم مسار نشر الرسائل بحيث يتم فصل عملية قبول الرسالة وحفظها عن عملية توزيعها على المشتركين. فعند قيام المستخدم بإرسال رسالة إلى قناة، تستقبل الواجهة الخلفية الطلب وتتحقق أولاً من هوية المستخدم وعضويته في القناة وصلاحيته للنشر.

بعد نجاح التحقق، يتم حفظ الرسالة في قاعدة بيانات PostgreSQL مع إنشاء سجل مرتبط بعملية التوزيع ضمن جدول Outbox. ويتم تنفيذ هاتين العمليتين ضمن المعاملة المناسبة، بحيث لا تعتبر عملية النشر مقبولة دون وجود سجل دائم يمكن من خلاله متابعة عملية التوزيع لاحقاً. ويوضح الشكل رقم (11) المسار الكامل لنشر الرسالة وتوزيعها على المشتركين.



الشكل رقم 11 مسار نشر الرسالة وتوزيعها على المشتركين

بعد حفظ البيانات، يتولى مكوّن **Worker** قراءة سجلات Outbox التي تحتاج إلى المعالجة ونشرها إلى RabbitMQ. يقوم وسيط الرسائل بعد ذلك بتوجيه الرسالة وفق قواعد التوجيه والطواير المخصصة لعملية التسليم، ثم تتم معالجة الرسائل الموجهة إلى المستلمين بصورة غير متزامنة.

عند وجود مستخدم متصل بالنظام، يتم تمرير الحدث اللحظي عبر **Redis** إلى خادم الواجهة الخلفية الذي يحتفظ باتصال **WebSocket** الخاص بالمستخدم، ومن ثم يتم إرسال الرسالة الجديدة إلى الواجهة الأمامية بصورة مباشرة.

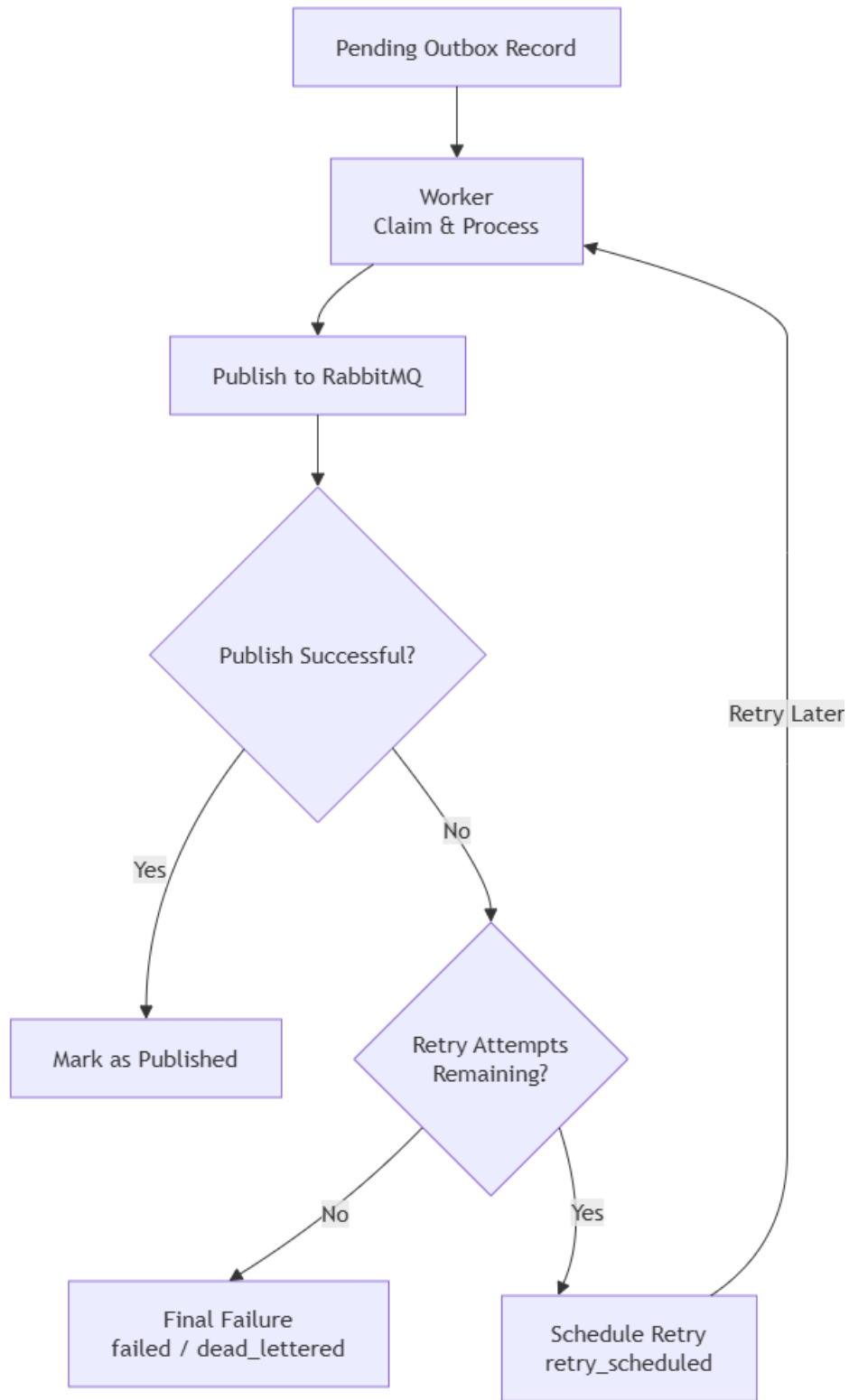
أما في حال كان المستخدم غير متصل، فلا يعتمد النظام على **WebSocket** أو **Redis** للاحتفاظ بالرسالة، إذ تبقى الرسالة محفوظة بصورة دائمة في **PostgreSQL** ويمكن استرجاعها من خلال سجل الرسائل أو آليات المزامنة عند عودة المستخدم إلى الاتصال.

يساعد هذا المسار على الفصل بين حفظ الرسالة وتسليمها اللحظي، ويضمن أن الفشل المؤقت في **RabbitMQ** أو **Redis** أو **WebSocket** لا يؤدي إلى فقدان الرسالة الأصلية بعد نجاح تخزينها في قاعدة البيانات.

9.5- تصميم آليات موثوقية التوزيع ومعالجة الفشل

تم تصميم مسار توزيع الرسائل بحيث يستطيع التعامل مع حالات الفشل المؤقت دون فقدان الرسالة الأصلية. ويعتمد ذلك بصورة أساسية على الفصل بين تخزين الرسالة وبين تنفيذ عملية التوزيع، حيث يتم الاحتفاظ بالرسالة وسجل **Outbox** الخاص بها في قاعدة البيانات قبل الاعتماد على أي مكوّن خارجي.

عند معالجة سجل **Outbox**، يحاول مكوّن **Worker** نشر عملية التوزيع إلى **RabbitMQ**. وفي حال نجاح العملية يتم تحديث حالة سجل **Outbox** بما يعكس تقدم عملية النشر، أما عند حدوث فشل مؤقت فيتم الاحتفاظ بالسجل وإعادة محاولة معالجته وفق سياسة إعادة المحاولة المحددة. ويوضح الشكل رقم (12) الحالات التي يمكن أن يمر بها سجل **Outbox** أثناء المعالجة وإعادة المحاولة.



الشكل رقم 12 حالات معالجة سجل Outbox وإعادة المحاولة عند الفشل

تُستخدم حالات مختلفة داخل سجل Outbox لتمثيل مرحلة معالجة العملية، مثل انتظار المعالجة أو النشر أو جدولة إعادة المحاولة أو الفشل النهائي. ويتيح ذلك معرفة حالة كل عملية توزيع وعدم اعتبار العملية ناجحة قبل إتمام المرحلة المطلوبة منها.

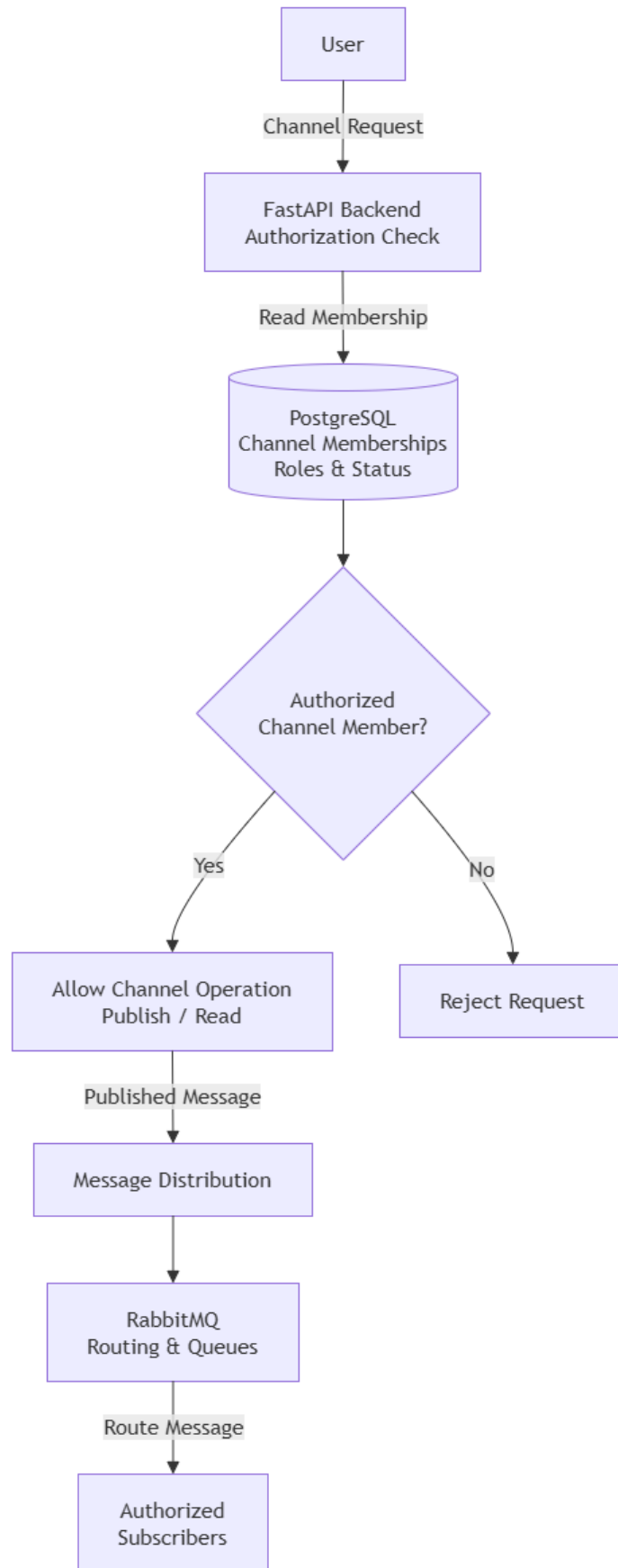
كما يعتمد مسار التوزيع على تأكيد معالجة الرسائل عند التعامل مع وسيط الرسائل، مما يسمح بإعادة تسليم الرسالة في بعض حالات الفشل. ونظراً لإمكانية إعادة المعالجة، يجب أن تكون العمليات الحساسة مصممة بحيث تتحمل التكرار دون إنتاج نتائج غير صحيحة.

وفي حال استنفاد محاولات المعالجة، لا يتم حذف أثر العملية بصورة صامتة، وإنما يتم الاحتفاظ بحالة تدل على فشلها أو انتقالها إلى حالة نهائية يمكن متابعتها. ويساعد هذا التصميم على اكتشاف المشكلات وتشخيصها مع بقاء البيانات الأساسية محفوظة في PostgreSQL.

10.5 - تصميم إدارة العضويات وربطها بتوجيه الرسائل

تعتمد عملية الوصول إلى القنوات وتوزيع الرسائل على معلومات العضوية المحفوظة بصورة دائمة في قاعدة البيانات. فالعلاقة بين المستخدم والقناة لا تقتصر على تحديد ما إذا كان المستخدم عضواً فيها، وإنما تتضمن أيضاً حالته ودوره والصلاحيات الممنوحة له داخل القناة.

عند محاولة تنفيذ عملية مرتبطة بقناة، مثل نشر رسالة أو استعراض محتواها، تقوم الواجهة الخلفية بالتحقق من بيانات العضوية الموجودة في PostgreSQL قبل السماح بتنفيذ العملية. وبذلك تبقى قاعدة البيانات هي المرجع الأساسي لتحديد صلاحية المستخدم، ولا يعتمد القرار الأمني على حالة RabbitMQ أو Redis. ويوضح الشكل رقم (13) ارتباط معلومات العضوية بمسار توجيه الرسائل.



الشكل رقم 13 ربط العضويات بمسار توجيه الرسائل

ترتبط العضويات أيضاً بعملية توزيع الرسائل، حيث يتم استخدام معلومات القناة والمستخدمين المشتركين فيها لتحديد الجهات التي يمكن توجيه الرسائل إليها. ويستخدم **RabbitMQ** آليات التوجيه والطاير لتنفيذ عملية التوزيع غير المتزامن، بينما تتم المحافظة على حالة الربط اللازمة بين عضويات النظام ومسار وسيط الرسائل بصورة منفصلة عن بيانات العضوية الأساسية.

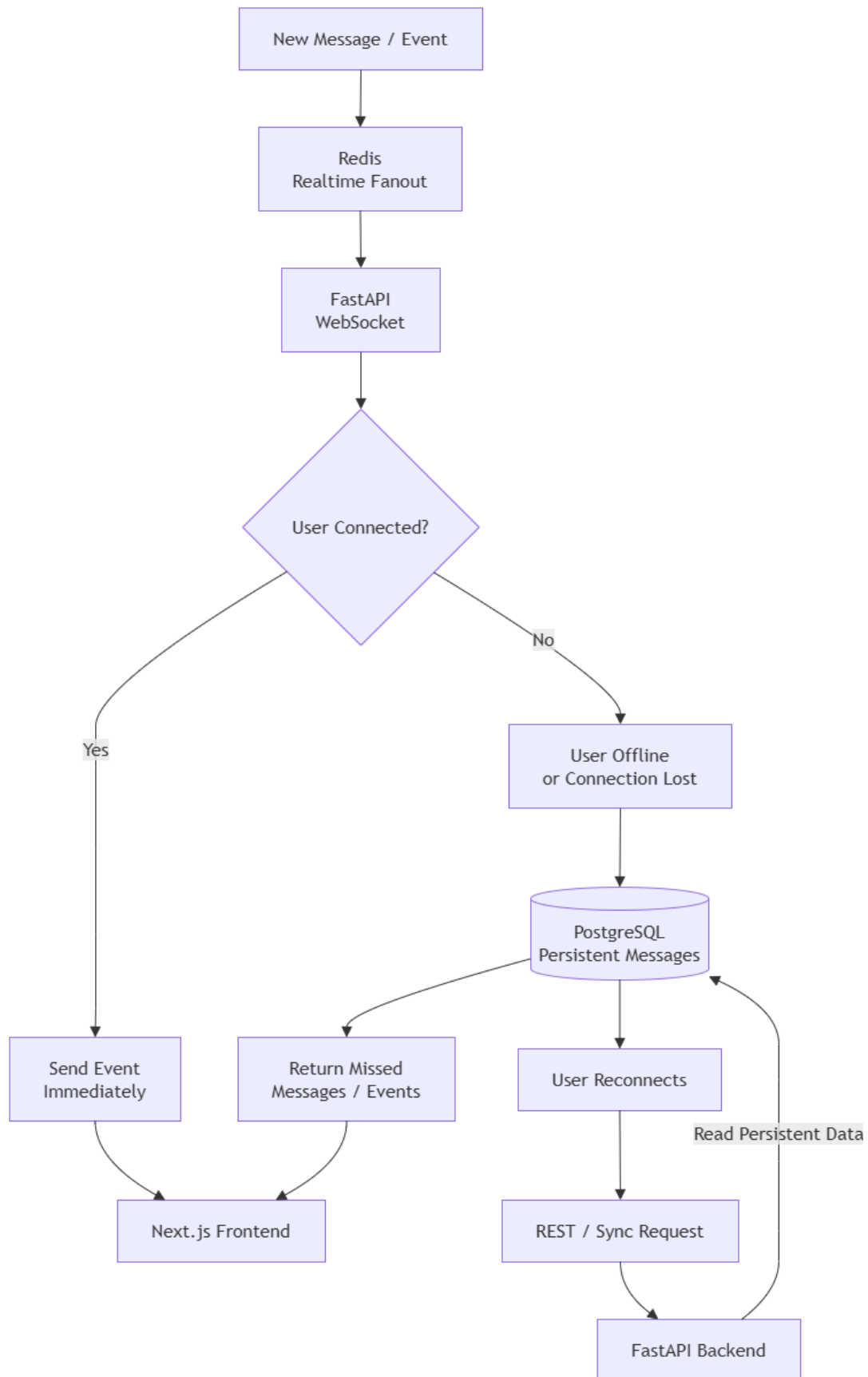
وعند حدوث تغيير في العضوية، مثل انضمام مستخدم إلى قناة أو خروجه منها، يجب أن ينعكس ذلك على عمليات التوزيع اللاحقة، بحيث لا يستقبل المستخدم رسائل قناة لم يعد مخولاً بالوصول إليها. وفي جميع الحالات تبقى صلاحية الوصول النهائية مرتبطة بالحالة الدائمة المحفوظة في قاعدة البيانات.

يساعد هذا التصميم على فصل مفهوم العضوية والصلاحيات عن البنية الداخلية لوسيط الرسائل، بحيث تستخدم **RabbitMQ** كوسيلة للتوزيع، بينما تبقى **PostgreSQL** المرجع المسؤول عن تحديد المستخدمين والقنوات والعلاقات المسموح بها.

11.5 - تصميم الاتصال اللحظي ومزامنة الرسائل

تم تصميم النظام بحيث يجمع بين الإيصال اللحظي للرسائل وإمكانية استرجاع الرسائل التي لم تصل إلى المستخدم أثناء انقطاع الاتصال. ولذلك تم فصل مسار الاتصال اللحظي عن التخزين الدائم للبيانات، بحيث لا يعتمد النظام على اتصال **WebSocket** وحده لضمان وصول الرسائل.

أثناء اتصال المستخدم بالنظام، تحتفظ الواجهة الأمامية باتصال **WebSocket** مع الواجهة الخلفية. وعند وصول حدث جديد خاص بالمستخدم، يتم تمريره عبر **Redis** إلى مكّون **WebSocket** المناسب، الذي يقوم بإرساله مباشرة إلى الواجهة الأمامية لعرضه دون الحاجة إلى تنفيذ طلبات دورية للحصول على التحديثات. ويوضح الشكل رقم (14) مسار الاتصال اللحظي وآلية استعادة الرسائل بعد انقطاع الاتصال.

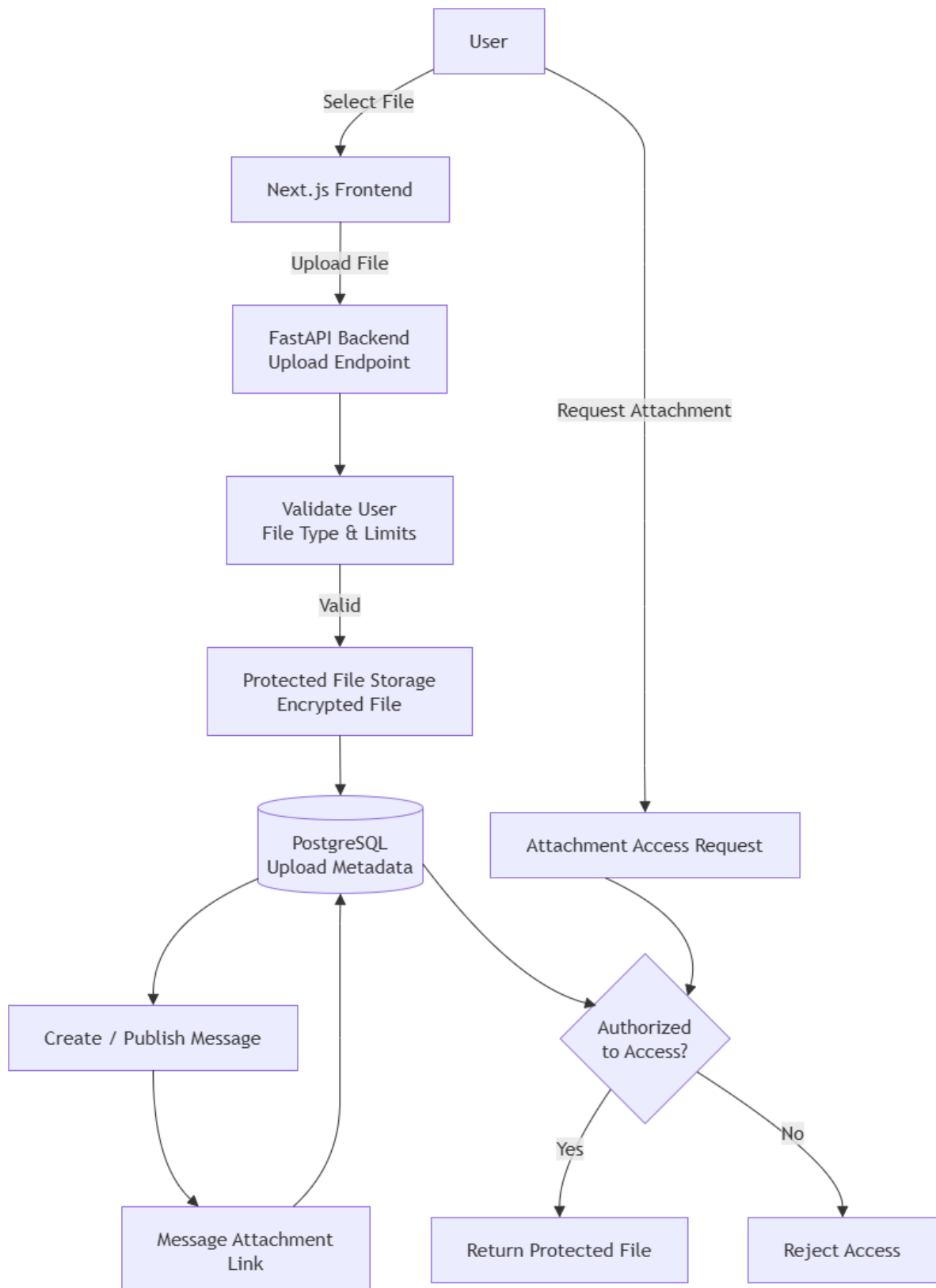


الشكل رقم 14 مسار الاتصال اللحظي ومزامنة الرسائل بعد انقطاع الاتصال

في حال انقطاع اتصال WebSocket، قد لا يستقبل المستخدم بعض الأحداث اللحظية التي تم إرسالها أثناء فترة الانقطاع. إلا أن ذلك لا يؤدي إلى فقدان الرسائل، لأن الرسائل الأساسية تبقى محفوظة بصورة دائمة في PostgreSQL. عند إعادة الاتصال، تستطيع الواجهة الأمامية طلب الرسائل أو التحديثات التي لم تصل إليها من خلال واجهات المزامنة في الواجهة الخلفية، التي تقوم بالاعتماد على البيانات الدائمة لتحديد المعلومات الأحدث وإعادة إرسالها إلى المستخدم. وبعد اكتمال عملية المزامنة يستمر استقبال الأحداث الجديدة من خلال WebSocket. وبذلك يؤدي WebSocket و Redis دور المسار السريع لإيصال التحديثات للمستخدمين المتصلين، بينما تمثل PostgreSQL وآليات المزامنة المسار الموثوق لاسترجاع ما قد يفوت المستخدم أثناء فترات الانقطاع. ويسمح هذا التصميم بتحقيق سرعة الاتصال دون التضحية بموثوقية البيانات.

12.5 - تصميم إدارة الملفات والمرفقات

يدعم النظام إرفاق الملفات بالرسائل مع فصل عملية تخزين الملف عن تخزين محتوى الرسالة نفسها. ويهدف هذا التصميم إلى تسهيل إدارة الملفات وتطبيق قواعد الوصول والحماية عليها بصورة مستقلة عن الرسائل النصية. عند قيام المستخدم برفع ملف، تستقبل الواجهة الخلفية طلب الرفع وتتحقق من هوية المستخدم وصحة الملف والقيود المطبقة عليه. بعد ذلك يتم تخزين الملف وفق آلية التخزين المعتمدة، مع إنشاء سجل خاص به في قاعدة البيانات يحتوي على البيانات اللازمة لتعريفه وربطه بالمستخدم الذي قام برفعه. ويوضح الشكل رقم (15) المراحل الرئيسة لرفع الملف وربطه بالرسالة.



الشكل رقم 15 مسار رفع الملفات وربطها بالرسائل

عند نشر رسالة تحتوي على مرفق، يتم إنشاء علاقة بين الرسالة والملف المخزن، بحيث يمكن استرجاع المرفقات المرتبطة بكل رسالة دون الحاجة إلى تخزين البيانات الثنائية للملف داخل سجل الرسالة نفسه. ويساعد ذلك على المحافظة على تنظيم قاعدة البيانات وفصل بيانات الرسائل عن بيانات الملفات.

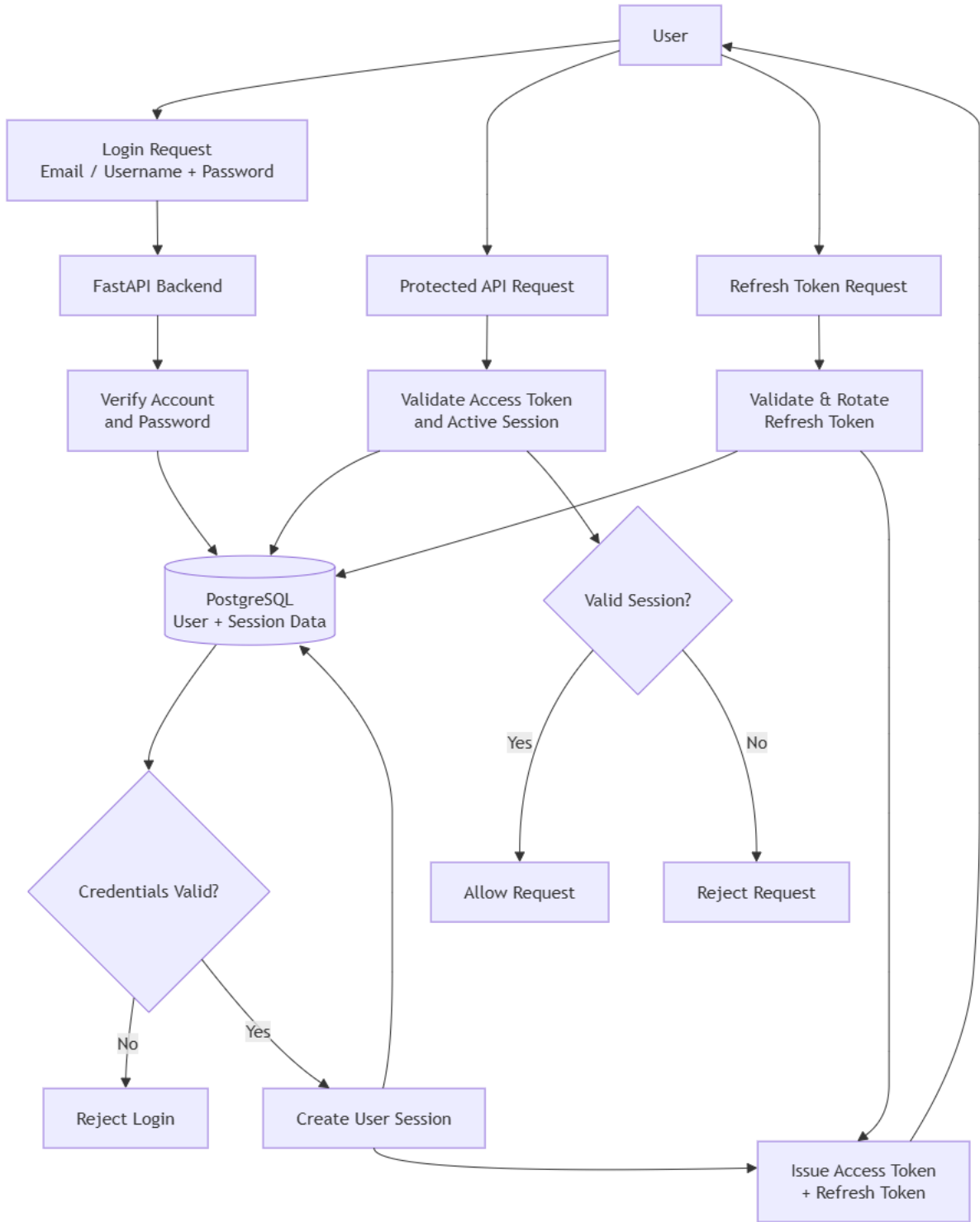
كما يتم التحقق من صلاحية المستخدم عند محاولة الوصول إلى ملف محمي، بحيث لا يكفي امتلاك رابط الملف للوصول إليه، وإنما يتم التحقق من أن المستخدم يمتلك الصلاحية المناسبة للوصول إلى القناة أو الرسالة المرتبطة به.

وتخضع الملفات الحساسة لآليات الحماية المعتمدة في النظام أثناء التخزين، بحيث يتم الجمع بين حماية المحتوى والتحقق من صلاحيات الوصول إليه. وبذلك تصبح إدارة المرفقات جزءاً من نموذج الصلاحيات العام للنظام بدلاً من التعامل معها كملفات عامة مستقلة.

13.5 - تصميم المصادقة وإدارة الجلسات

تم تصميم نظام المصادقة بحيث يتم التحقق من هوية المستخدم قبل السماح له بالوصول إلى الوظائف المحمية، مع الاحتفاظ بحالة مستقلة لكل جلسة مستخدم بما يسمح بمتابعتها وإلغائها عند الحاجة. ويتم تخزين كلمات المرور بصورة آمنة باستخدام خوارزمية مخصصة لتجزئة كلمات المرور بدلاً من حفظها بصورتها الأصلية.

عند تسجيل الدخول، ترسل بيانات الاعتماد إلى الواجهة الخلفية، التي تتحقق من صحة الحساب وكلمة المرور. وفي حال نجاح العملية يتم إنشاء سجل للجلسة في قاعدة البيانات وإصدار **Access Token** قصير الاستخدام مع **Refresh Token** يسمح بتجديد بيانات المصادقة دون مطالبة المستخدم بإدخال كلمة المرور في كل مرة. ويوضح الشكل رقم (16) مسار المصادقة وإنشاء الجلسة وإدارة رموز الوصول والتحديث.



الشكل رقم 16 مسار المصادقة وإدارة جلسات المستخدم

عند تنفيذ طلب محمي، يتم إرسال Access Token مع الطلب، وتتحقق الواجهة الخلفية من صحة الرمز وارتباطه بجلسة فعالة قبل تنفيذ العملية. وبذلك لا يعتمد النظام على صحة الرمز فقط، وإنما يمكن أيضاً رفضه إذا كانت الجلسة المرتبطة به قد ألغيت أو أصبحت غير صالحة.

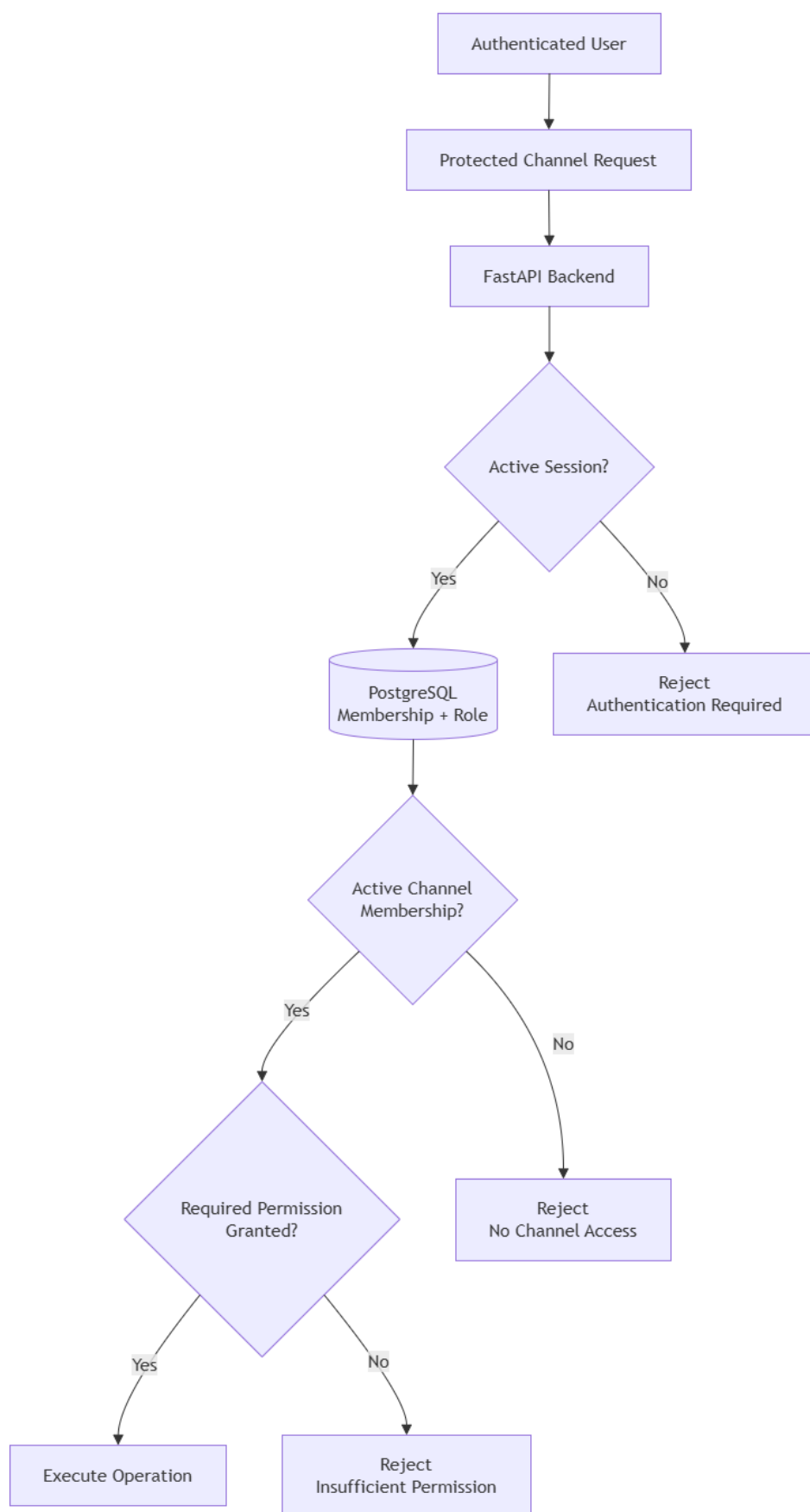
وعند انتهاء صلاحية رمز الوصول، يمكن استخدام Refresh Token للحصول على رمز جديد بعد التحقق من الجلسة وصحة رمز التحديث. ويتم تدوير رموز التحديث عند استخدامها، مما يقلل من إمكانية إعادة استخدام رمز قديم بصورة غير مشروعة ويساعد على اكتشاف بعض حالات إعادة الاستخدام غير المتوقع.

أما عند تسجيل الخروج، فيتم إلغاء الجلسة المرتبطة بالمستخدم، وبذلك تصبح الرموز المرتبطة بها غير قابلة للاستخدام للوصول إلى الوظائف المحمية. ويسمح هذا التصميم بإدارة الجلسات بصورة مركزية مع إمكانية إبطال الوصول دون الحاجة إلى تغيير بيانات حساب المستخدم.

14.5- تصميم التحكم في الصلاحيات والأدوار

بعد التحقق من هوية المستخدم من خلال نظام المصادقة، يتم تطبيق آلية مستقلة للتحقق من الصلاحيات قبل تنفيذ العمليات المرتبطة بموارد النظام. ويهدف ذلك إلى ضمان أن المستخدم المصادق عليه لا يستطيع تنفيذ عملية إلا إذا كان يمتلك العضوية والدور والصلاحيات المناسبة لها.

تعتمد صلاحيات القنوات على معلومات العضوية المخزنة في قاعدة البيانات، حيث تربط العضوية بين المستخدم والقناة وتحدد حالته ودوره ضمنها. وبناءً على هذه البيانات يمكن للنظام التمييز بين المستخدم العادي ومالك القناة أو المستخدم الذي يمتلك صلاحيات إدارية إضافية. ويوضح الشكل رقم (17) خطوات التحقق من العضوية والدور والصلاحيات قبل تنفيذ العملية.



الشكل رقم 17 مسار التحقق من الصلاحيات والأدوار داخل القنوات

عند وصول طلب لتنفيذ عملية، مثل نشر رسالة أو تعديل إعدادات قناة أو إدارة أحد الأعضاء، تتحقق الواجهة الخلفية أولاً من وجود جلسة مصادقة فعالة، ثم تحدد المورد المطلوب وتقرأ معلومات العضوية والدور المرتبطين بالمستخدم.

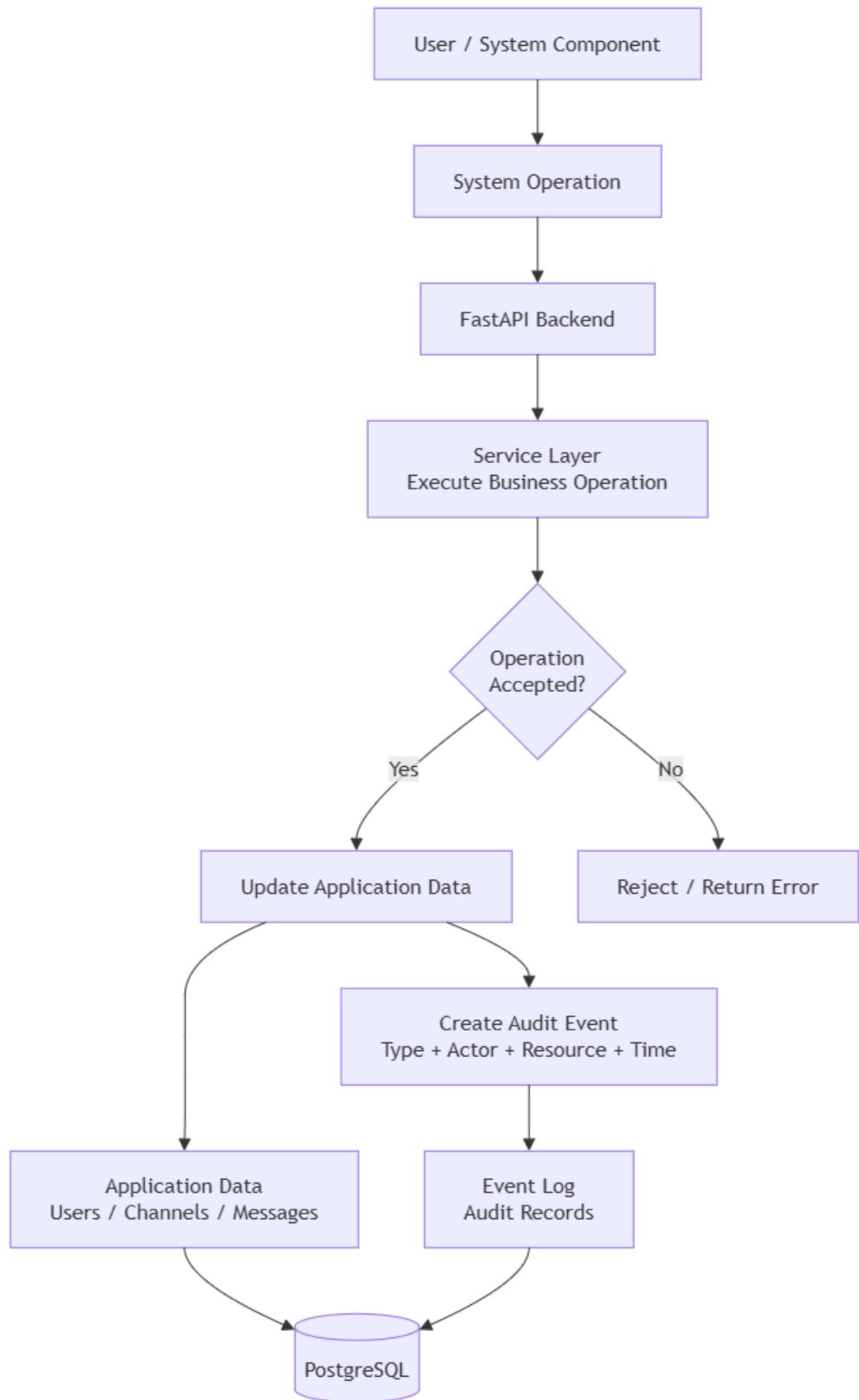
بعد ذلك تتم مقارنة العملية المطلوبة بالصلاحيات المسموح بها. فإذا كانت العضوية فعالة وكان المستخدم يمتلك الصلاحية المناسبة يتم السماح بتنفيذ العملية، أما في حال عدم وجود العضوية أو عدم كفاية الدور فيتم رفض الطلب قبل إجراء أي تغيير على بيانات النظام.

ويساعد هذا التصميم على تطبيق مبدأ أقل صلاحية ممكنة (**Principle of Least Privilege**)، بحيث يحصل كل مستخدم على الصلاحيات اللازمة لدوره فقط. كما يؤدي تنفيذ التحقق في الواجهة الخلفية إلى منع الاعتماد على إخفاء الخيارات في الواجهة الأمامية كوسيلة للحماية، إذ يبقى قرار السماح النهائي مسؤولية الخادم.

15.5 - تصميم سجل الأحداث والتدقيق

تم تصميم النظام ليحتفظ بسجل للأحداث المهمة التي تحدث أثناء استخدامه، بهدف توفير إمكانية تتبع العمليات ومراجعتها لاحقاً. وتشمل هذه الأحداث العمليات المرتبطة بالمستخدمين والقنوات والعضويات والرسائل، إضافة إلى بعض العمليات الإدارية والأمنية التي يكون من المهم الاحتفاظ بأثر لها.

عند تنفيذ عملية تتطلب التسجيل، تقوم طبقة الخدمات في الواجهة الخلفية بإنشاء حدث يتضمن المعلومات اللازمة لوصف العملية، مثل نوع الحدث والجهة التي نفذته والمورد المرتبط به ووقت حدوثه. ويتم بعد ذلك تخزين الحدث بصورة دائمة في قاعدة بيانات PostgreSQL. ويوضح الشكل رقم (18) مسار إنشاء الحدث وتخزينه ضمن سجل الأحداث.



الشكل رقم 18 مسار إنشاء سجل الأحداث وتخزينه

يتم فصل سجل الأحداث عن البيانات التشغيلية الأساسية، بحيث لا يمثل بديلاً عن جداول المستخدمين أو القنوات أو الرسائل، وإنما يحتفظ بأثر للعمليات التي تمت على هذه الموارد. ويسمح ذلك باستخدام السجل عند الحاجة إلى مراجعة تسلسل العمليات أو معرفة الجهة التي قامت بإجراء معين.

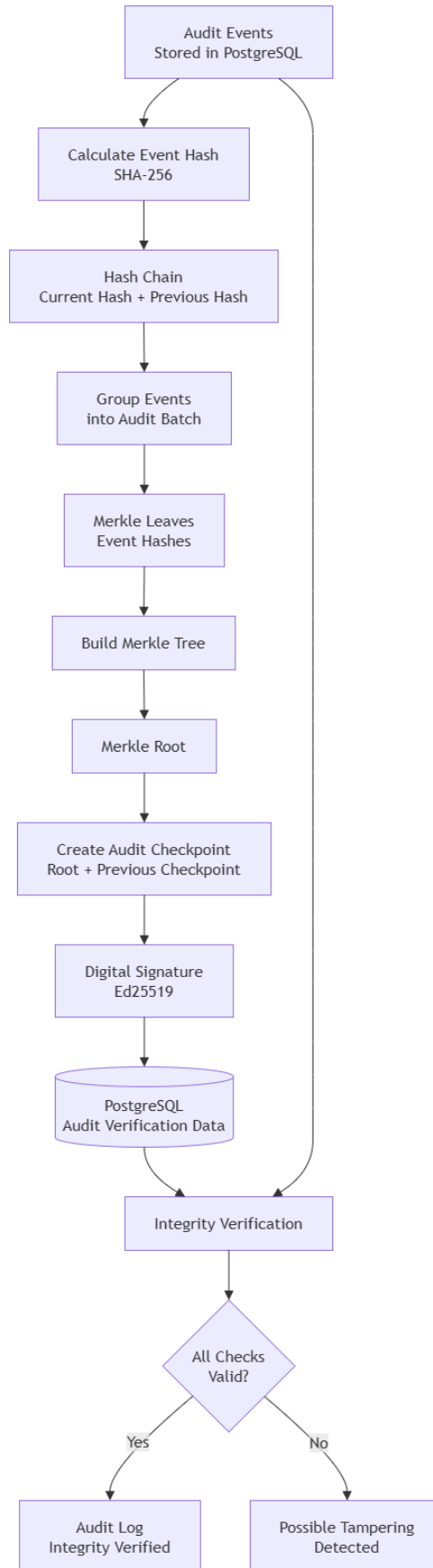
كما يتم تسجيل الأحداث من داخل الواجهة الخلفية بعد التحقق من هوية المستخدم وصلاحياته، مما يسمح بربط العملية المنفذة بسياقها الصحيح. ويمكن كذلك تسجيل بعض الأحداث التي تنتج عن مكونات النظام الداخلية وليس عن طلب مباشر من المستخدم، وفق نوع العملية المطلوب تتبعها.

ولا يقتصر تصميم سجل الأحداث على تخزين السجلات فقط، بل يرتبط بآليات إضافية للتحقق من سلامة السجل واكتشاف التعديل غير المصرح به. ولهذا يتم بناء طبقة تحقق تعتمد على قيم التجزئة ونقاط تحقق مشفرة، وسيتم توضيح تصميمها في القسم التالي.

16.5 - تصميم التحقق من سلامة سجل البحث

تم تصميم آلية للتحقق من سلامة سجل الأحداث بهدف اكتشاف أي تعديل غير مصرح به قد يحدث على الأحداث المحفوظة. ولا تمنع هذه الآلية إمكانية تعديل البيانات بشكل مطلق، وإنما توفر دليلاً يمكن استخدامه للكشف عن حدوث تغيير في السجل بعد إنشائه.

يتم أولاً ربط الأحداث المتتابعة باستخدام سلسلة تجزئة (Hash Chain)، حيث تحسب لكل حدث قيمة تجزئة تعتمد على محتواه وعلى قيمة التجزئة الخاصة بالحدث السابق. وبهذه الطريقة يؤدي تغيير محتوى حدث قديم إلى عدم تطابق قيم التجزئة في الأحداث التي تأتي بعده، مما يسمح باكتشاف العبث عند التحقق من السلسلة. ويوضح الشكل رقم (19) المراحل المستخدمة للتحقق من سلامة سجل الأحداث.



الشكل رقم 19 مراحل التحقق من سلامة سجل الأحداث باستخدام التجزئة وشجرة Merkle والتوقيع الرقمي

إضافة إلى سلسلة التجزئة، يتم تجميع مجموعة من الأحداث ضمن دفعات تستخدم لبناء **شجرة ميركل (Merkle Tree)**. تمثل قيم تجزئة الأحداث أوراق الشجرة، ثم يتم دمجها على عدة مستويات حتى الوصول إلى قيمة واحدة تعرف باسم **Merkle Root** وتمثل حالة مجموعة الأحداث الموجودة في تلك الدفعة.

تسمح شجرة ميركل أيضاً بإنتاج **Merkle Proof** للتحقق من أن حدثاً محدداً ينتمي إلى دفعة معينة دون الحاجة إلى إعادة فحص جميع أحداث الدفعة. ويصبح هذا الأسلوب أكثر فائدة مع ازدياد عدد السجلات التي يجب التحقق منها. بعد إنشاء نقطة التحقق، يتم توقيع البيانات المرتبطة بها باستخدام **توقيع رقمي**، مما يوفر إمكانية التحقق من أن نقطة التحقق صادرة عن النظام ولم يتم تغييرها بعد توقيعها. كما يتم ربط نقاط التحقق المتتالية ببعضها للمساعدة على اكتشاف حذف أو استبدال جزء من تسلسل نقاط التحقق السابقة.

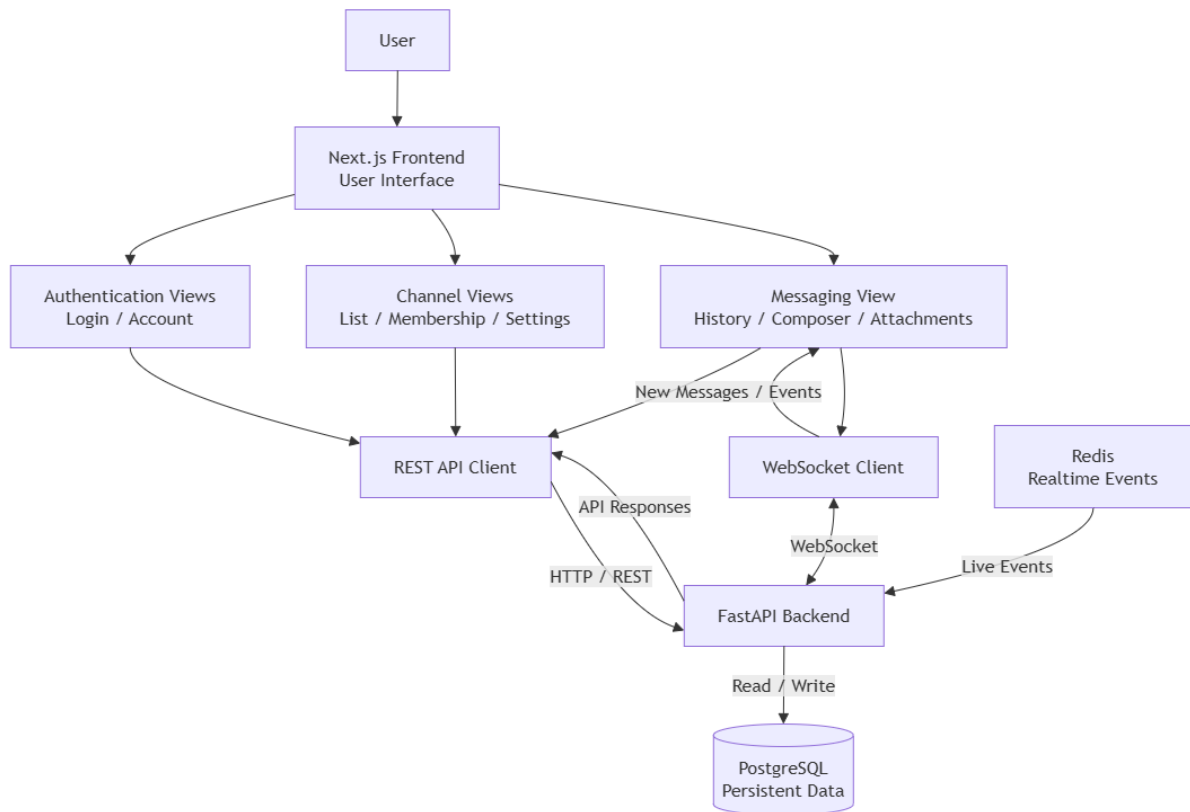
وعند تنفيذ عملية تحقق، يعيد النظام حساب قيم التجزئة المطلوبة ويقارنها بالقيم المحفوظة، ثم يتحقق من **Merkle Root** والتوقيع الرقمي والترابط بين نقاط التحقق. وفي حال عدم تطابق إحدى هذه القيم يمكن اعتبار سلامة السجل غير مؤكدة والإشارة إلى وجود تعديل محتمل.

يوفر هذا التصميم عدة مستويات للتحقق من سلامة سجل الأحداث؛ حيث تؤمن سلسلة التجزئة ترابط الأحداث، وتتيح شجرة ميركل التحقق الفعال من مجموعات السجلات، بينما يوفر التوقيع الرقمي وسيلة للتحقق من صحة نقاط التحقق المنشأة. وبذلك يصبح سجل الأحداث قابلاً لاكتشاف العبث (**Tamper-Evident**) دون اعتباره سجلاً غير قابل للتعديل بصورة مطلقة.

17.5 - تصميم الواجهة الأمامية

تم تصميم الواجهة الأمامية باستخدام **Next.js** لتكون نقطة التفاعل الرئيسية بين المستخدم ووظائف نظام المراسلة. وتركز الواجهة على توفير طريقة واضحة للوصول إلى القنوات وإدارة العضويات واستعراض الرسائل ونشرها، مع التعامل مع عمليات المصادقة والاتصال اللحظي بصورة متكاملة مع الواجهة الخلفية.

تتواصل الواجهة الأمامية مع **FastAPI Backend** بطريقتين رئيسيتين. تستخدم طلبات **REST API** لتنفيذ العمليات التي تتطلب طلباً مباشراً من المستخدم، مثل تسجيل الدخول وإنشاء القنوات والانضمام إليها وجلب سجل الرسائل ونشر المحتوى. أما **WebSocket** فيستخدم لاستقبال الرسائل والأحداث الجديدة بصورة لحظية أثناء اتصال المستخدم بالنظام. ويوضح الشكل رقم (20) البنية العامة للواجهة الأمامية وطريقة تكاملها مع خدمات النظام.



الشكل رقم 20 البنية العامة للواجهة الأمامية وتكاملها مع خدمات النظام

تم تنظيم واجهة المستخدم حول مجموعة من الشاشات والمكونات الأساسية، مثل شاشات تسجيل الدخول وإدارة الحساب، وقائمة القنوات، وواجهة المحادثة، وإدارة أعضاء القناة وإعداداتها. ويتيح هذا التقسيم إعادة استخدام المكونات المشتركة وفصل الوظائف المختلفة داخل الواجهة.

عند فتح قناة، يتم جلب الرسائل المحفوظة من الواجهة الخلفية وعرضها للمستخدم، بينما تصل الرسائل الجديدة من خلال اتصال WebSocket. وعند فقد الاتصال اللحظي، تبقى الواجهة قادرة على استعادة الحالة من الخادم من خلال طلبات المزامنة بدلاً من الاعتماد على البيانات الموجودة محلياً فقط.

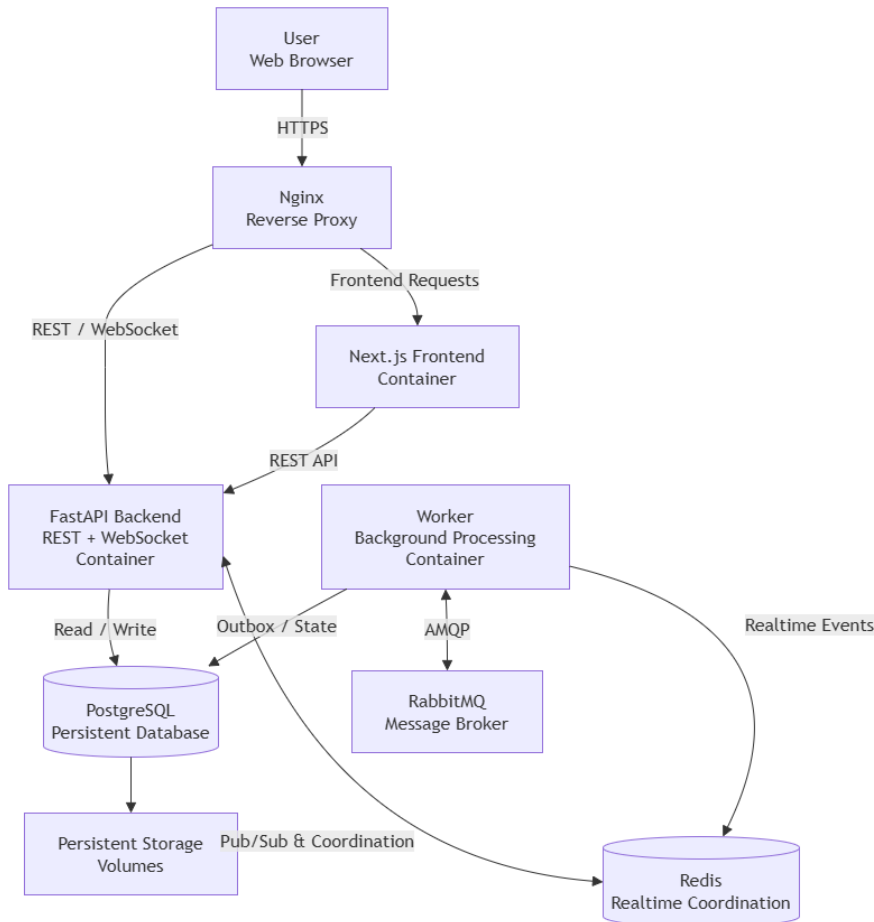
كما تتعامل الواجهة مع حالات نجاح وفشل الطلبات وتوضح للمستخدم حالة العمليات التي يقوم بها، مثل فشل تسجيل الدخول أو عدم امتلاك صلاحية لتنفيذ عملية معينة. ومع ذلك تبقى جميع عمليات التحقق الأمنية النهائية مسؤولية الواجهة الخلفية، ولا يتم اعتبار إخفاء عنصر من عناصر الواجهة بديلاً عن التحقق من الصلاحية في الخادم.

ويساعد هذا التصميم على الفصل بين عرض البيانات وتنفيذ قواعد العمل، بحيث تركز الواجهة الأمامية على تجربة المستخدم وإدارة الحالة المحلية، بينما تبقى العمليات الحساسة والبيانات الدائمة وقواعد الصلاحيات ضمن مكونات الخادم.

18.5 - تصميم بيئة التشغيل والنشر

تم تصميم بيئة تشغيل النظام بحيث تعمل المكونات المختلفة بصورة منفصلة مع توفير شبكة اتصال داخلية تسمح لها بالتكامل فيما بينها. ويساعد هذا الأسلوب على عزل مسؤوليات كل خدمة وتسهيل تشغيل النظام وإعادة تشغيل أحد مكوناته دون الحاجة إلى دمج جميع الوظائف ضمن عملية واحدة.

يتم استخدام **Docker** لتشغيل مكونات النظام ضمن حاويات مستقلة، بينما تستخدم إعدادات **Docker Compose** لتعريف الخدمات والعلاقات بينها وإعداد الشبكات ووحدات التخزين الدائمة اللازمة. وتشمل بيئة التشغيل الواجهة الأمامية والواجهة الخلفية ومكوّن **Worker**، إضافة إلى **PostgreSQL** و **RabbitMQ** و **Redis**. ويوضح الشكل رقم (20) البنية العامة للواجهة الأمامية وطريقة تكاملها مع خدمات النظام. ويوضح الشكل رقم (21) توزيع خدمات النظام ضمن بيئة **Docker** وطريقة توجيه الاتصالات بينها.



الشكل رقم 21 بنية تشغيل ونشر مكونات النظام باستخدام Docker و Nginx

توجد **Nginx** في نقطة الدخول إلى النظام، حيث تستقبل اتصالات المستخدم وتقوم بتوجيه الطلبات إلى الخدمة المناسبة. ويتم توجيه طلبات الواجهة والتطبيق إلى المكونات المخصصة لها، مع دعم تمرير اتصالات **WebSocket** اللازمة للاتصال اللحظي.

تتصل الواجهة الخلفية بقاعدة بيانات PostgreSQL لتنفيذ عمليات التخزين والاسترجاع، كما تتصل بRedis لتنفيذ الوظائف اللحظية والتنسيق المرتبط باتصالات WebSocket. ويعمل مكوّن Worker بصورة مستقلة ويتصل بقاعدة البيانات لمعالجة سجلات Outbox ، و RabbitMQ لتنفيذ عمليات النشر والاستهلاك، و Redis عند الحاجة إلى تمرير الأحداث اللحظية.

أما البيانات التي يجب أن تستمر بعد إعادة تشغيل الحاويات، فلا يتم الاعتماد في حفظها على دورة حياة الحاوية نفسها، وإنما تستخدم وحدات تخزين دائمة للخدمات التي تحتاج إليها، وبشكل خاص قاعدة البيانات والبيانات المخزنة الخاصة بالنظام.

يسمح هذا التصميم بفصل الخدمات التشغيلية عن بعضها، وتحديد الاتصالات المطلوبة بينها، كما يسهل نقل النظام وتشغيله ضمن بيئات مختلفة مع المحافظة على بنية تشغيل موحدة وقابلة للإدارة.

19.5 - الخلاصة

تم في هذا الفصل توضيح التصميم العام لنظام المراسلة الموزع، بدءاً من المعمارية العامة وتقسيم النظام إلى مكونات مستقلة، مروراً بتصميم الواجهة الخلفية وقاعدة البيانات ومسار نشر الرسائل وتوزيعها، إضافة إلى آليات معالجة حالات الفشل وإعادة المحاولة.

كما تم توضيح تصميم العضويات والصلاحيات والاتصال اللحظي ومزامنة الرسائل، إلى جانب إدارة الملفات والمرفقات ونظام المصادقة وإدارة الجلسات. وتم كذلك عرض تصميم سجل الأحداث وآليات التحقق من سلامته بالاعتماد على سلسلة التجزئة وشجرة ميركل والتوقيع الرقمي.

وتناول الفصل أيضاً تصميم الواجهة الأمامية وبيئة تشغيل النظام، موضحاً كيفية تكامل المكونات المختلفة مع PostgreSQL و RabbitMQ و Redis و WebSocket ضمن بيئة تشغيل تعتمد على Docker.

وقد ساعد هذا التصميم على تحديد مسؤولية كل مكوّن ومسارات الاتصال بين الأجزاء المختلفة قبل الانتقال إلى مرحلة التنفيذ العملي. وفي الفصل التالي سيتم استعراض الأدوات والتقنيات البرمجية التي تم استخدامها لبناء هذه المكونات وتنفيذ النظام.

الفصل السادس

الأدوات المستخدمة

يتناول هذا الفصل أهم التقنيات والأطر البرمجية المستخدمة في بناء النظام، مع توضيح وظيفة كل منها ضمن المعمارية العامة للمشروع.

1.6 - المقدمة

اعتمد تنفيذ نظام المراسلة على مجموعة من الأدوات والتقنيات البرمجية التي تؤدي أدواراً مختلفة ضمن بنية النظام، بدءاً من تطوير الواجهة الخلفية والواجهة الأمامية، وصولاً إلى تخزين البيانات وتوزيع الرسائل والاتصال اللحظي وتشغيل الخدمات واوزيع الرسائل والاتصال اللحظي وتشغيل الخدمات واختبارها.

تم اختيار هذه الأدوات بما يتناسب مع طبيعة النظام الموزع ومتطلباته، حيث استخدمت **Python** و **FastAPI** لتطوير الواجهة الخلفية، و **PostgreSQL** للتخزين الدائم، و **RabbitMQ** لتنفيذ عمليات توزيع الرسائل غير المتزامنة، و **Redis** لدعم الاتصال اللحظي والتنسيق بين اتصالات **WebSocket**. كما تم بناء الواجهة الأمامية باستخدام **Next.js** و **React** و **TypeScript**.

ولإدارة تشغيل المكونات المختلفة تم الاعتماد على **Docker** و **Docker Compose**، مع استخدام **Nginx** كنقطة دخول وتوجيه للطلبات. كما استخدمت مجموعة من المكتبات والأدوات المساندة لإدارة قاعدة البيانات والمصادقة والأمان والاختبارات وإدارة الشيفرة المصدرية.

يتناول هذا الفصل الأدوات والتقنيات الرئيسة المستخدمة في تنفيذ المشروع، مع توضيح الوظيفة التي تؤديها كل أداة وأسباب استخدامها ضمن النظام.

2.6 - لغة Python

تم استخدام لغة **Python** كلغة أساسية لتطوير الواجهة الخلفية ومكونات المعالجة الخلفية في النظام. وتتميز **Python** ببنية واضحة وتوفر مجموعة كبيرة من المكتبات والأطر التي تدعم تطوير تطبيقات الويب والأنظمة الشبكية والتعامل مع قواعد البيانات ووسطاء الرسائل.

استخدمت **Python** في المشروع لتنفيذ منطق العمل الخاص بالمستخدمين والقنوات والعضويات والرسائل والمصادقة والصلاحيات، بالإضافة إلى الخدمات المرتبطة بتسجيل الأحداث وإدارة عمليات التوزيع والتحقق من سلامة البيانات.

كما تم استخدامها في بناء مكّون **Worker** المسؤول عن تنفيذ المهام غير المتزامنة، مثل معالجة سجلات **Outbox** والتعامل مع **RabbitMQ** و **Redis**. وقد ساعد دعم **Python** للبرمجة غير المتزامنة (**Asynchronous Programming**) على التعامل مع العمليات التي تعتمد بصورة كبيرة على الإدخال والإخراج، مثل طلبات الشبكة وقاعدة البيانات ووسطاء الرسائل، دون الحاجة إلى حجز التنفيذ أثناء انتظار اكتمال هذه العمليات.

وقد تم اختيار **Python** لملاءمتها مع إطار **FastAPI** والأدوات الأخرى المستخدمة في الواجهة الخلفية، إضافة إلى سهولة تنظيم الشيفرة واختبار المكونات المختلفة وصيانتها.

3.6 – FastAPI وUvicorn

تم استخدام إطار **FastAPI** لبناء الواجهة الخلفية وتوفير واجهات البرمجة التي تتعامل معها الواجهة الأمامية. وهو إطار مخصص لتطوير تطبيقات الويب باستخدام **Python**، ويدعم بصورة مباشرة البرمجة غير المتزامنة وتعريف نماذج البيانات والتحقق من صحة المدخلات.

استخدم **FastAPI** في المشروع لبناء واجهات **REST API** الخاصة بالمصادقة وإدارة المستخدمين والقنوات والعضويات والرسائل والملفات، إضافة إلى الوظائف الإدارية والخدمات المرتبطة بالمرامنة. كما تم استخدام دعمه لاتصالات **WebSocket** لإنشاء قناة اتصال مستمرة مع المستخدمين المتصلين واستقبال وإرسال الأحداث اللحظية.

يساعد أسلوب تعريف المسارات والنماذج في **FastAPI** على فصل واجهات الطلبات عن طبقة الخدمات التي تحتوي على قواعد العمل، كما يوفر آليات مناسبة لمعالجة الأخطاء والتحقق من البيانات وإدارة الاعتماديات مثل التحقق من المستخدم المصادق عليه قبل تنفيذ الطلب.

أما **Uvicorn** فهو خادم تطبيقات يدعم معيار **ASGI**، وتم استخدامه لتشغيل تطبيق **FastAPI** واستقبال طلبات **HTTP** واتصالات **WebSocket**. ويتيح **ASGI** التعامل مع عدد من الاتصالات والعمليات غير المتزامنة بصورة مناسبة لطبيعة تطبيقات المراسلة التي تحتوي على طلبات تقليدية واتصالات طويلة الأمد في الوقت نفسه.

وقد وفر الجمع بين **FastAPI** و **Uvicorn** بيئة مناسبة لبناء الواجهة الخلفية وتشغيلها، مع دعم العمليات غير المتزامنة والاتصال اللحظي والتكامل مع بقية مكونات النظام.

4.6 – PostgreSQL

تم استخدام نظام إدارة قواعد البيانات **PostgreSQL** لتخزين البيانات الدائمة الخاصة بالنظام. وهو نظام قواعد بيانات علائقية يوفر دعماً للمعاملات والعلاقات والقيود، مما يجعله مناسباً للبيانات المترابطة الموجودة في نظام المراسلة.

تستخدم **PostgreSQL** في المشروع لتخزين بيانات المستخدمين والجلسات والقنوات والعضويات والرسائل والمرفقات، بالإضافة إلى سجل الأحداث والبيانات المتعلقة بعمليات التوزيع مثل سجلات **Outbox**.

تم اعتماد قاعدة البيانات كمصدر أساسي للحقيقة (**Source of Truth**) في النظام، بحيث تبقى الرسائل وحالة القنوات والعضويات محفوظة بصورة مستقلة عن **RabbitMQ** و **Redis**. لذلك فإن توقف أحد مكونات التوزيع أو الاتصال اللحظي بصورة مؤقتة لا يؤدي إلى فقدان البيانات الأساسية المحفوظة.

كما توفر PostgreSQL دعماً للمعاملات (**Transactions**)، وقد تمت الاستفادة من ذلك في العمليات التي تتطلب حفظ عدة تغييرات مترابطة بصورة ذرية، ومن أبرزها حفظ الرسالة وإنشاء سجل Outbox المرتبط بعملية توزيعها. فإذا فشلت المعاملة لا يتم الاحتفاظ بجزء منها دون الآخر.

وقد ساعد استخدام العلاقات والمفاتيح الأساسية والأجنبية والقيود على المحافظة على تكامل البيانات بين المستخدمين والقنوات والعضويات والرسائل، إضافة إلى توفير أساس موثوق لعمليات الاسترجاع والمزامنة والتدقيق.

5.6 – SQLAlchemy و AsyncPG

تم استخدام SQLAlchemy لإدارة التعامل بين الواجهة الخلفية وقاعدة بيانات PostgreSQL، حيث يوفر طبقة برمجية تسمح بتمثيل جداول قاعدة البيانات على شكل نماذج داخل تطبيق Python وتنفيذ عمليات القراءة والإضافة والتعديل والحذف بطريقة منظمة.

استخدمت نماذج SQLAlchemy لتمثيل الكيانات المختلفة في النظام، مثل المستخدمين والجلسات والقنوات والعضويات والرسائل والمرفقات وسجلات الأحداث وعمليات Outbox. كما تم استخدام العلاقات بين النماذج للتعبير عن الارتباطات الموجودة بين هذه الكيانات والمحافظة على تنظيم طبقة البيانات.

ويعتمد النظام على واجهات SQLAlchemy غير المتزامنة، بما في ذلك **AsyncSession**، لتنفيذ عمليات قاعدة البيانات دون حجز مسار التنفيذ أثناء انتظار استجابة PostgreSQL. ويتناسب هذا الأسلوب مع طبيعة FastAPI والعمليات الشبكية غير المتزامنة المستخدمة في النظام.

أما **AsyncPG** فهو برنامج تشغيل (**Database Driver**) غير متزامن مخصص للتعامل مع PostgreSQL من تطبيقات Python، ويستخدم لإنشاء الاتصال الفعلي بقاعدة البيانات وتنفيذ الاستعلامات التي تديرها طبقة SQLAlchemy. وقد أتاح الجمع بين SQLAlchemy و AsyncPG فصل تفاصيل قاعدة البيانات عن منطق العمل، مع توفير وصول غير متزامن ومنظم إلى البيانات يتناسب مع بنية الواجهة الخلفية للنظام.

6.6 – Alembic

تم استخدام أداة **Alembic** لإدارة التغييرات التي تطرأ على بنية قاعدة بيانات PostgreSQL أثناء تطوير النظام. وتعمل Alembic بالتكامل مع SQLAlchemy، وتسمح بإنشاء سلسلة من ملفات الترحيل (**Migrations**) التي توثق التعديلات التي أجريت على الجداول والعلاقات والحقول والقيود.

بدلاً من تعديل قاعدة البيانات يدوياً في كل مرة تتغير فيها نماذج النظام، يتم إنشاء Migration يصف الانتقال من البنية السابقة إلى البنية الجديدة. ويمكن بعد ذلك تطبيق هذه التغييرات بصورة مرتبة على بيئات التطوير أو الاختبار أو التشغيل.

تمت الاستفادة من Alembic في المشروع عند إضافة الجداول والحقول المرتبطة بتطور وظائف النظام، مثل بيانات الجلسات والعضويات والرسائل والمرفقات وسجلات Outbox والأحداث والبيانات المستخدمة في آليات التدقيق والتحقق من سلامة السجل.

كما تساعد ملفات الترحيل على المحافظة على توافق بنية قاعدة البيانات مع الشيفرة البرمجية، وتوفر سجلاً زمنياً للتغييرات التي طرأت عليها، مع إمكانية معرفة الإصدار الحالي وتنفيذ التحديثات المطلوبة بترتيبها الصحيح.

وقد ساعد استخدام Alembic على جعل تطوير قاعدة البيانات أكثر تنظيماً وقابلية للتكرار، خصوصاً عند تشغيل النظام في أكثر من بيئة أو إعادة إنشائه من البداية.

7.6 – RabbitMQ و aio-pika

تم استخدام RabbitMQ كوسيط رسائل (Message Broker) لتنفيذ عمليات توزيع الرسائل غير المتزامنة بين مكونات النظام. ويسمح RabbitMQ بفصل الجهة التي تنتج عملية التوزيع عن الجهة التي تقوم بمعالجتها، مما يقلل من الترابط بين المكونات ويمنع ارتباط زمن استجابة طلب المستخدم بإتمام جميع عمليات التسليم.

يعتمد النظام على الطوابير وآليات التوجيه التي يوفرها RabbitMQ لتنظيم عملية إيصال الرسائل إلى الوجهات المناسبة، مع الاستفادة من خصائص مثل استمرارية الطوابير وتأكيد المعالجة وإعادة المحاولة عند حدوث بعض حالات الفشل.

يستخدم مكوّن Worker RabbitMQ لمعالجة مسار التوزيع بصورة مستقلة عن خادم FastAPI. فبعد قراءة سجل Outbox المطلوب معالجته، يمكن نشر بيانات التوزيع إلى وسيط الرسائل ثم استهلاك الرسائل الموجهة ومعالجتها وفق قواعد النظام.

أما مكتبة aio-pika فهي مكتبة Python توفر واجهة غير متزامنة للتعامل مع RabbitMQ باستخدام بروتوكول AMQP. وتم استخدامها لإنشاء الاتصالات والقنوات والتعامل مع عمليات النشر والاستهلاك والتأكيد بصورة تتناسب مع البنية غير المتزامنة لمكوّن Worker.

وقد ساعد الجمع بين RabbitMQ و aio-pika على تنفيذ مسار توزيع مستقل وموثوق نسبياً، مع إمكانية التعامل مع حالات الفشل وإعادة المعالجة دون التأثير على التخزين الدائم للرسائل في PostgreSQL.

8.6 – Redis و WebSocket

تم استخدام Redis كخدمة سريعة للبيانات المؤقتة والتنسيق بين مكونات النظام التي تحتاج إلى تبادل المعلومات بصورة لحظية. ولا يتم الاعتماد على Redis كمصدر دائم للرسائل، إذ تبقى البيانات الأساسية محفوظة في PostgreSQL.

يستخدم Redis في النظام لدعم عملية تمرير الأحداث اللحظية بين مكوّن Worker وخوادم الواجهة الخلفية، بحيث يمكن إيصال الرسائل والتحديثات الجديدة إلى المستخدم الذي يمتلك اتصالاً فعالاً مع النظام. كما تستخدم بعض قدراته في الوظائف المؤقتة المرتبطة بإدارة الاتصال والتنسيق وفرض بعض القيود التشغيلية.

أما **WebSocket** فهو بروتوكول اتصال يسمح بإنشاء قناة اتصال مستمرة وثنائية الاتجاه بين المتصفح والخادم. وتم استخدامه لإيصال الرسائل والأحداث الجديدة إلى الواجهة الأمامية مباشرة، دون الحاجة إلى قيام المتصفح بإرسال طلبات دورية للتحقق من وجود تحديثات جديدة.

عند توفر حدث جديد، يتم تمريره عبر المسار اللحظي إلى خادم FastAPI الذي يدير اتصال WebSocket الخاص بالمستخدم، ثم يرسل الحدث إلى الواجهة الأمامية. وفي حال لم يكن المستخدم متصلاً في تلك اللحظة، لا تعتبر الرسالة مفقودة، لأن النسخة الدائمة منها تبقى محفوظة في PostgreSQL ويمكن استرجاعها لاحقاً من خلال واجهات المزامنة. وقد ساعد استخدام Redis و WebSocket على توفير تجربة مراسلة لحظية للمستخدمين المتصلين، مع المحافظة على الفصل بين مسار الإيصال السريع ومسار التخزين الدائم والموثوق للبيانات.

9.6 – TypeScript و React و Next.js

تم تطوير الواجهة الأمامية للنظام باستخدام **Next.js**، **React** و **TypeScript**، حيث تمثل هذه التقنيات الأساس المستخدم لبناء صفحات التطبيق ومكوناته والتعامل مع تفاعل المستخدم مع نظام المراسلة.

استخدم **React** لبناء واجهة المستخدم على شكل مكونات (**Components**) مستقلة وقابلة لإعادة الاستخدام، مثل مكونات القنوات وقائمة الرسائل ونموذج إرسال الرسالة وإدارة الأعضاء والمرفقات. ويساعد هذا الأسلوب على تقسيم الواجهة إلى أجزاء واضحة بدلاً من بناء الصفحة كوحدة واحدة كبيرة.

أما **Next.js** فقد استخدم كإطار لبناء تطبيق الواجهة الأمامية وتنظيم الصفحات والمسارات والمكونات، إضافة إلى توفير بيئة موحدة لتطوير وتشغيل التطبيق. وتتولى الواجهة المبنية باستخدام **Next.js** التواصل مع الواجهة الخلفية لتنفيذ العمليات المختلفة وعرض البيانات التي يعيدها الخادم.

تم استخدام **TypeScript** بدلاً من الاعتماد على **JavaScript** وحدها، إذ تضيف **TypeScript** نظاماً للأنواع (**Type System**) يساعد على تحديد أشكال البيانات المستخدمة بين المكونات واكتشاف عدد من الأخطاء أثناء مرحلة التطوير. ويكتسب ذلك أهمية عند التعامل مع كائنات مثل المستخدمين والقنوات والرسائل ونتائج واجهات **REST**.

كما تتعامل الواجهة مع نوعين رئيسيين من الاتصال بالخادم؛ حيث تستخدم واجهات **REST API** لتنفيذ الطلبات واسترجاع البيانات، بينما يتم استخدام **WebSocket** لاستقبال الرسائل والأحداث الجديدة بصورة لحظية.

وقد ساعد الجمع بين **Next.js** و **React** و **TypeScript** على بناء واجهة منظمة وقابلة للصيانة، مع فصل مكونات العرض عن منطق الخادم وتوفير أساس مناسب لتطوير وظائف الواجهة بصورة تدريجية.

10.6 – Docker و Docker Compose

تم استخدام **Docker** لتشغيل مكونات النظام ضمن حاويات (**Containers**) مستقلة، بحيث تحتوي كل خدمة على بيئة التشغيل والاعتماديات اللازمة لها. ويساعد هذا الأسلوب على تقليل الاختلاف بين بيئات التطوير والتشغيل وتسهيل إعادة بناء النظام وتشغيله على أجهزة مختلفة.

تم إعداد حاويات مستقلة للمكونات الرئيسية في النظام، مثل الواجهة الأمامية المبنية باستخدام **Next.js**، والواجهة الخلفية المبنية باستخدام **FastAPI**، ومكوّن **Worker**، إضافة إلى الخدمات المساندة مثل **PostgreSQL** و **RabbitMQ** و **Redis**.

أما **Docker Compose** فقد استخدم لتعريف هذه الخدمات ضمن إعداد موحد يحدد طريقة تشغيلها والعلاقات بينها، مثل الشبكات الداخلية ومتغيرات البيئة ومنفذ الاتصال ووحدات التخزين الدائمة. ويسمح ذلك بتشغيل مجموعة الخدمات التي يتكون منها النظام وإدارتها بصورة مترابطة بدلاً من تشغيل كل حاوية وإعدادها يدوياً بصورة منفصلة.

كما تم استخدام وحدات التخزين الدائمة (**Volumes**) مع الخدمات التي تحتاج إلى المحافظة على بياناتها بعد إعادة تشغيل الحاويات، وخاصة قاعدة البيانات والبيانات الدائمة الخاصة بالنظام. أما الاتصال بين الخدمات فيتم من خلال شبكة **Docker** الداخلية وباستخدام أسماء الخدمات بدلاً من الاعتماد على عناوين ثابتة.

وقد وفر استخدام **Docker** و **Docker Compose** بيئة تشغيل قابلة للتكرار ومنظمة، وساعد على عزل الخدمات عن بعضها وتوضيح الاعتماديات بينها، إضافة إلى تسهيل اختبار تشغيل النظام كمجموعة متكاملة من المكونات.

11.6 – Nginx

تم استخدام **Nginx** كنقطة دخول أمامية إلى النظام (**Reverse Proxy**)، بحيث تستقبل من خلاله الطلبات القادمة من المستخدمين ثم يتم توجيهها إلى الخدمة المناسبة داخل بيئة التشغيل.

يتولى **Nginx** توجيه الطلبات المتعلقة بالواجهة الأمامية إلى خدمة **Next.js**، بينما يتم تمرير طلبات واجهة **REST API** إلى الواجهة الخلفية المبنية باستخدام **FastAPI**. ويساعد ذلك على تقديم مكونات النظام المختلفة من خلال نقطة دخول موحدة بدلاً من مطالبة المستخدم بالاتصال بكل خدمة بصورة مستقلة.

كما تم إعداد **Nginx** لتمرير اتصالات **WebSocket** إلى الواجهة الخلفية، إذ تختلف هذه الاتصالات عن طلبات **HTTP** التقليدية بسبب استمرار الاتصال بين العميل والخادم لفترة زمنية طويلة. ويضمن إعداد **Proxy** المناسب استمرار هذا الاتصال وإمكانية استخدامه لإيصال الأحداث اللحظية.

ويؤدي وجود **Nginx** أمام خدمات التطبيق أيضاً إلى إخفاء تفاصيل الشبكة الداخلية والمنفذ المستخدمة بين الحاويات عن المستخدم النهائي، بحيث تبقى عمليات الاتصال الداخلي بين الخدمات منفصلة عن الواجهة التي يتم الوصول إليها من خارج النظام.

وقد ساعد استخدام Nginx على تنظيم حركة الطلبات بين مكونات التطبيق وتوفير نقطة دخول واضحة إلى النظام، مع دعم كل من طلبات HTTP واتصالات WebSocket ضمن بيئة التشغيل.

12.6- أدوات الأمان والتشفير

تم استخدام مجموعة من الأدوات والخوارزميات الأمنية لحماية حسابات المستخدمين والجلسات والبيانات المخزنة والتحقق من سلامة سجل الأحداث. وتم توزيع هذه الآليات على عدة مستويات بحيث لا تعتمد حماية النظام على وسيلة أمنية واحدة فقط.

بالنسبة إلى كلمات المرور، تم استخدام خوارزمية Argon2 لتجزئتها قبل تخزينها في قاعدة البيانات. وبذلك لا يتم الاحتفاظ بكلمة المرور الأصلية، وإنما يتم تخزين قيمة ناتجة عن عملية التجزئة تستخدم لاحقاً للتحقق من صحة بيانات تسجيل الدخول.

أما إدارة المصادقة بين العميل والخادم فتعتمد على رموز JWT (JSON Web Token)، حيث تستخدم رموز الوصول لإثبات هوية المستخدم عند إرسال الطلبات المحمية. كما يتم ربط عملية المصادقة بجلسات محفوظة في النظام، مع استخدام رموز تحديث لتجديد الوصول وفق آلية إدارة الجلسات المعتمدة.

ولحماية البيانات الحساسة أثناء التخزين، يستخدم النظام آليات للتشفير على مستوى الخادم. ويتم تشفير الملفات والمرفقات باستخدام AES-256-GCM، وهي خوارزمية تشفير موثقة توفر حماية محتوى البيانات مع إمكانية التحقق من سلامتها أثناء فك التشفير.

كما تم استخدام خوارزمية SHA-256 في الآليات المتعلقة بسلامة سجل الأحداث، حيث تستخدم قيم التجزئة لبناء سلسلة تربط الأحداث المتتالية، إضافة إلى استخدامها ضمن بناء Merkle Tree وإنشاء قيمة Merkle Root التي تمثل مجموعة من الأحداث.

ويتم دعم نقاط التحقق الخاصة بسجل التدقيق باستخدام التوقيع الرقمي Ed25519، حيث يسمح المفتاح الخاص بإنشاء التوقيع، بينما يستخدم المفتاح العام للتحقق من صحته. ويساعد ذلك على اكتشاف تعديل بيانات نقطة التحقق أو استبدالها بعد توقيعها.

وقد ساعد الجمع بين تجزئة كلمات المرور وإدارة الرموز والجلسات وتشفير البيانات ودوال التجزئة والتوقيع الرقمي على توفير طبقات أمنية متعددة تغطي المصادقة وحماية البيانات والتحقق من سلامة السجلات.

13.6- أدوات الاختبار والتحقق

تم استخدام مجموعة من أدوات الاختبار والتحقق للتأكد من صحة مكونات النظام وسلامة التكامل بينها، حيث شملت عملية التحقق الواجهة الخلفية وقاعدة البيانات ومكوّن Worker والواجهة الأمامية وبيئة التشغيل.

تم الاعتماد على إطار **pytest** بصورة أساسية لتنفيذ اختبارات الواجهة الخلفية المكتوبة بلغة Python. واستخدمت الاختبارات للتحقق من الوظائف المختلفة مثل المصادقة وإدارة الجلسات والقنوات والعضويات والرسائل والصلاحيات، إضافة إلى عمليات التوزيع وسجل الأحداث وآليات الأمان.

كما تم تنفيذ اختبارات تكامل للتحقق من تفاعل عدة مكونات مع بعضها، وخصوصاً العمليات التي تعتمد على قاعدة البيانات ومسار Outbox وتوزيع الرسائل. وتم كذلك تضمين اختبارات لحالات الفشل وإعادة المحاولة والتحقق من رفض العمليات غير المصرح بها.

بالنسبة إلى الواجهة الأمامية، تم استخدام أدوات بيئة **Node.js** للتحقق من صحة المشروع، بما في ذلك فحص TypeScript وبناء نسخة الإنتاج، مما يساعد على اكتشاف الأخطاء والأنواع ومشكلات البناء قبل تشغيل التطبيق.

كما تم التحقق من ملفات ترحيل قاعدة البيانات باستخدام **Alembic** والتأكد من إمكانية تطبيقها بصورة صحيحة، إضافة إلى بناء صور Docker وتشغيل الخدمات للتحقق من صحة إعدادات بيئة التشغيل والتكامل بين المكونات.

وشملت عملية التحقق أيضاً الاختبارات الخاصة بآليات سلامة سجل الأحداث، مثل إعادة حساب قيم التجزئة والتحقق من Merkle Root والتوقيع الرقمي واكتشاف التعديل على البيانات. وقد ساعد تنوع هذه الاختبارات على تغطية الجوانب الوظيفية والأمنية والتكاملية للنظام، وسيتم عرض خطة الاختبار والنتائج بصورة أكثر تفصيلاً في الفصل العملي.

14.6 - Git و GitHub

تم استخدام نظام التحكم بالإصدارات **Git** لإدارة الشيفرة المصدرية للمشروع وتتبع التغييرات التي تطرأ عليها أثناء مراحل التطوير المختلفة. ويسمح **Git** بالاحتفاظ بسجل للتعديلات والرجوع إلى إصدارات سابقة عند الحاجة، إضافة إلى تسهيل تنظيم عملية إضافة الميزات وإصلاح الأخطاء.

تم تقسيم التغييرات البرمجية إلى مجموعة من عمليات الحفظ (**Commits**) التي تصف التطوير الذي تم تنفيذه، مما يساعد على تتبع تطور المشروع ومعرفة التعديلات التي أجريت على الملفات والمكونات المختلفة.

كما تم استخدام **GitHub** لاستضافة مستودع المشروع وحفظ الشيفرة المصدرية وملفات الإعداد والتوثيق ضمن موقع مركزي. ويحتوي المستودع على المكونات المختلفة للنظام، مثل الواجهة الخلفية والواجهة الأمامية ومكون **Worker** وملفات **Docker** وإعدادات التشغيل والاختبارات والتوثيق.

وساعد استخدام **Git** و **GitHub** على تنظيم عملية التطوير والمحافظة على سجل واضح للتغييرات، إضافة إلى تسهيل مراجعة الشيفرة ومشاركة المشروع وتشغيله اعتماداً على الملفات والتعليمات الموجودة في المستودع.

15.6 - الخلاصة

تم في هذا الفصل استعراض أهم الأدوات والتقنيات المستخدمة في تنفيذ نظام المراسلة الموزع، بدءاً من Python و FastAPI المستخدمة في بناء الواجهة الخلفية، و PostgreSQL و SQLAlchemy و Alembic المستخدمة في إدارة البيانات، وصولاً إلى RabbitMQ و Redis و WebSocket المستخدمة في التوزيع غير المتزامن والاتصال اللحظي.

كما تم توضيح التقنيات المستخدمة في بناء الواجهة الأمامية باستخدام Next.js و React و TypeScript، والأدوات المعتمدة لتشغيل النظام وإدارة خدماته مثل Docker و Docker Compose و Nginx.

وتناول الفصل أيضاً الأدوات والخوارزميات المرتبطة بالأمان وحماية البيانات وسلامة سجل الأحداث، إضافة إلى أدوات الاختبار والتحقق وإدارة الشيفرة المصدرية باستخدام Git وGitHub.

وقد شكل تكامل هذه الأدوات البيئة التقنية اللازمة لتنفيذ التصميم الموضح في الفصل السابق. وفي الفصل التالي سيتم عرض الجانب العملي للمشروع، من خلال توضيح كيفية تنفيذ المكونات الرئيسة وربطها مع بعضها، إضافة إلى استعراض الاختبارات والنتائج التي تم الحصول عليها.

الفصل السابع

القسم العملي

يتناول هذا الفصل كيفية تنفيذ النظام عملياً، من خلال عرض تنظيم المشروع، وتنفيذ الواجهة الخلفية والعامل المسؤول عن توزيع الرسائل، والواجهة الأمامية، بالإضافة إلى آليات الأمان والاختبارات المستخدمة للتحقق من صحة النظام.

1.7 - المقدمة

يتناول هذا الفصل الجانب العملي من المشروع، حيث يتم توضيح كيفية تحويل المتطلبات والتصميمات التي تم عرضها في الفصول السابقة إلى نظام مراسلة موزع قابل للتشغيل. ويركز الفصل على البنية البرمجية الفعلية للمشروع وطريقة تنفيذ المكونات الرئيسية وآلية تكاملها مع بعضها.

يبدأ الفصل بعرض تنظيم الشيفرة المصدرية وتقسيم المشروع إلى مكونات مستقلة، ثم يتم توضيح تنفيذ الواجهة الخلفية والوظائف الأساسية المتعلقة بالمصادقة والقنوات والعضويات والرسائل والمرفقات. بعد ذلك يتم استعراض تنفيذ مسار التوزيع غير المتزامن بالاعتماد على Transactional Outbox و Worker و RabbitMQ، إضافة إلى الاتصال اللحظي باستخدام Redis و WebSocket وآليات مزامنة الرسائل بعد انقطاع الاتصال.

كما يتناول الفصل الجوانب الأمنية المنفذة في النظام، بما في ذلك حماية الحسابات والبيانات وسجل الأحداث وآليات التحقق من سلامته، ثم يتم عرض الواجهة الأمامية وأهم الشاشات التي يتعامل معها المستخدم. ويوضح بعد ذلك تكامل المكونات من خلال تتبع سيناريو نشر رسالة بصورة كاملة من المرسل حتى وصولها إلى المشترك.

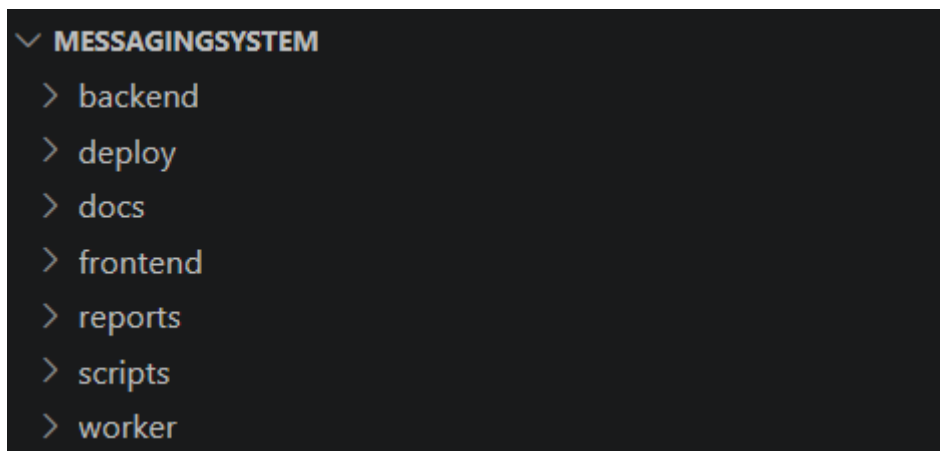
وفي الجزء الأخير من الفصل يتم عرض طريقة تشغيل النظام والاختبارات التي تم تنفيذها للتحقق من الوظائف والتكامل والأمان والموثوقية، إلى جانب النتائج التي تم الحصول عليها، بهدف تقييم مدى تحقيق النظام للمتطلبات المحددة للمشروع.

2.7 - تنظيم المشروع

1.2.7 - البنية العامة للمشروع

تم تنظيم الشيفرة المصدرية للمشروع ضمن مجموعة من المجلدات الرئيسة، بحيث يمثل كل مجلد جزءاً محدداً من النظام. ويساعد هذا التقسيم على فصل الواجهة الخلفية والواجهة الأمامية والمعالجة غير المتزامنة وملفات التشغيل والتوثيق، بدلاً

من وضع جميع مكونات النظام ضمن بنية واحدة مترابطة. ويوضح الشكل رقم (22) البنية العامة لمجلدات المشروع وملفات التشغيل الرئيسية.



الشكل رقم 22 البنية العامة لمجلدات المشروع وملفات التشغيل الرئيسية

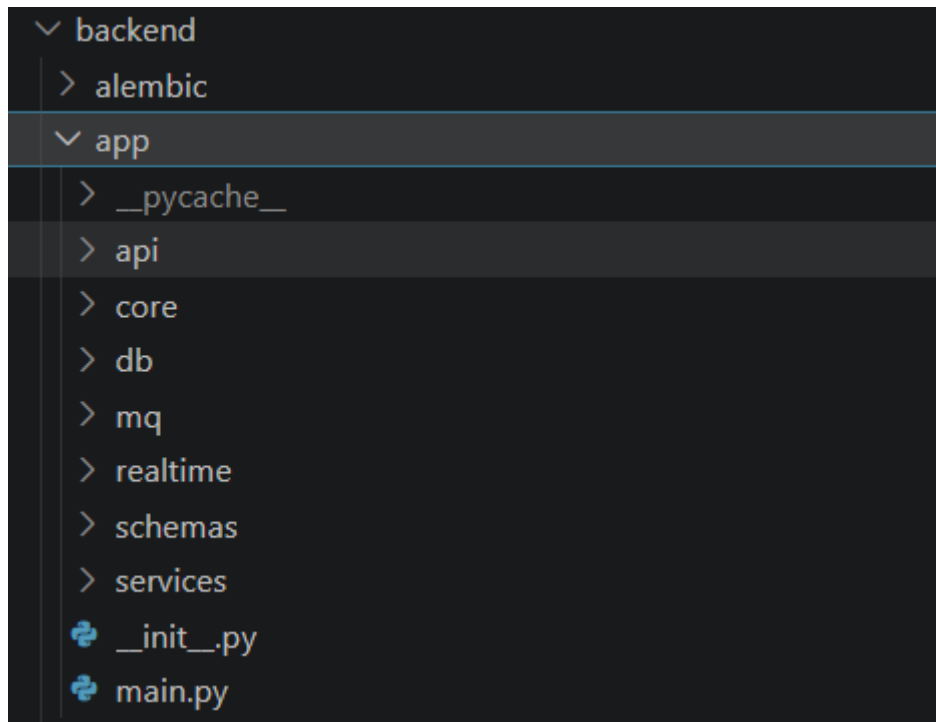
تتكون البنية العامة للمشروع من مجلد **backend** الذي يحتوي على تطبيق FastAPI ونماذج قاعدة البيانات والخدمات والاختبارات وترحيلات Alembic ، ومجلد **frontend** الخاص بتطبيق Next.js وملفات الواجهة، إضافة إلى مجلد **worker** الذي يحتوي على التطبيق المسؤول عن المعالجة الخلفية ومسار توزيع الرسائل.

كما يحتوي المشروع على مجلد **deploy** الذي يجمع الملفات المتعلقة بتهيئة خدمات التشغيل مثل Nginx و PostgreSQL، ومجلد **scripts** الذي يحتوي على الأدوات والبرامج المساعدة المستخدمة في عمليات التحقق والتشغيل، إضافة إلى مجلد **docs** الذي يضم وثائق المعمارية والاختبارات والأمان ومتطلبات المشروع وحالة التنفيذ.

وعلى مستوى الجذر توجد ملفات Docker Compose المستخدمة لتجميع وتشغيل الخدمات المختلفة، إلى جانب ملفات متغيرات البيئة النموذجية وملفات الإعدادات العامة. ويسمح هذا التنظيم بإبقاء شيفرة التطبيق منفصلة عن ملفات البنية التحتية والتوثيق، مع وجود نقطة موحدة لتشغيل النظام كمجموعة من الخدمات المتكاملة.

2.2.7 - تنظيم الواجهة الخلفية

تم تنظيم الواجهة الخلفية ضمن المسار `backend/app` بالاعتماد على فصل المسؤوليات بين مجموعة من الطبقات والمكونات البرمجية. فلا يتم وضع معالجة طلب المستخدم وقواعد العمل والوصول إلى قاعدة البيانات ضمن ملف واحد، وإنما يتم توزيعها على أجزاء متخصصة، مما يجعل الشيفرة أوضح وأسهل في الاختبار والتطوير. ويوضح الشكل رقم (23) التنظيم الداخلي لملفات الواجهة الخلفية وتقسيمها بحسب المسؤوليات.



الشكل رقم 23 التنظيم الداخلي لملفات الواجهة الخلفية

تمثل `main.py` نقطة بدء تطبيق `FastAPI`، حيث يتم فيها إنشاء التطبيق وتسجيل المسارات وإعداد معالجات الأخطاء والاتصال عبر `WebSocket`، إضافة إلى تهيئة المكونات التي يحتاج إليها التطبيق أثناء دورة حياته. ومن هذه النقطة يتم ربط بقية أجزاء الواجهة الخلفية مع بعضها.

توجد واجهات `HTTP` ضمن طبقة المسارات `api/routes`، حيث تتولى استقبال الطلبات والتحقق من البيانات المطلوبة على مستوى واجهة البرمجة ثم استدعاء الخدمة المناسبة. وقد تم فصل المسارات وفق المجال الوظيفي، مثل مسارات المصادقة والقنوات والعضويات والرسائل والمستخدمين والأحداث وفحوص الصحة. فعلى سبيل المثال تبدأ عملية نشر الرسالة من المسار الموجود في `api/routes/messages.py`، بينما تبدأ عمليات إدارة القنوات من `api/routes/channels.py`.

أما قواعد العمل الأساسية فتوجد ضمن مجلد `services` وتحتوي هذه الطبقة على خدمات متخصصة مثل `AuthService` لإدارة عمليات المصادقة والجلسات، و `ChannelService` لإدارة القنوات والعضويات والدعوات، و `MessageService` لتنفيذ عمليات نشر الرسائل واسترجاعها والتعامل مع خصائصها، بالإضافة إلى خدمات أخرى مرتبطة بالصلاحيات و `Outbox` وسجل الأحداث والتحقق من سلامته. وبهذا تبقى قواعد العمل منفصلة قدر الإمكان عن تفاصيل استقبال طلب `HTTP`.

تتولى طبقة قاعدة البيانات تمثيل البيانات الدائمة والوصول إليها، حيث يحتوي `db/models.py` على كيانات `SQLAlchemy` التي تمثل الجداول الرئيسية في `PostgreSQL` مثل المستخدمين والجلسات والقنوات والعضويات والرسائل والأحداث وسجلات `Outbox`. كما تستخدم طبقة `schemas` لتعريف نماذج البيانات التي يتم استقبالها أو إعدادها من واجهات `API` والتحقق من بنيتها قبل تمريرها إلى الخدمات.

وتوجد الوظائف المشتركة والبنية المساندة ضمن core، وتشمل الإعدادات والأمان والتشفير ومعالجة الأخطاء وعدداً من الأدوات المستخدمة في أكثر من جزء من التطبيق. كما تم فصل الوظائف الخاصة بالاتصال اللحظي ضمن realtime، حيث يوجد مدير WebSocket المسؤول عن إدارة الاتصالات والاشتراكات وتحرير الأحداث القادمة من Redis إلى المستخدمين المتصلين. أما الوظائف المتعلقة ببنية RabbitMQ والتوجيه فتوجد ضمن mq.

يسمح هذا التنظيم بأن يمر الطلب بصورة عامة من طبقة المسارات إلى الخدمة المسؤولة عن قواعد العمل، ثم إلى قاعدة البيانات أو المكونات المساندة عند الحاجة، قبل إعادة النتيجة إلى المستخدم. ويساعد هذا الفصل على تقليل الترابط بين أجزاء النظام وإتاحة اختبار منطق العمل بصورة أكثر استقلالية.

3.2.7- فصل مكونات النظام

تم تنفيذ النظام على شكل مجموعة من المكونات المستقلة، بحيث يؤدي كل مكون مجموعة محددة من المسؤوليات بدلاً من تنفيذ جميع الوظائف ضمن تطبيق واحد. ويظهر هذا الفصل بصورة واضحة بين الواجهة الأمامية والواجهة الخلفية ومكون Worker، إضافة إلى خدمات PostgreSQL و RabbitMQ و Redis.

تتولى الواجهة الأمامية المبنية باستخدام Next.js عرض البيانات وإدارة تفاعل المستخدم مع النظام، حيث تتصل بالواجهة الخلفية من خلال REST API لتنفيذ العمليات المختلفة، كما تستخدم WebSocket لاستقبال الرسائل والتحديثات الجديدة بصورة لحظية.

أما الواجهة الخلفية المبنية باستخدام FastAPI فتتولى استقبال الطلبات والتحقق من هوية المستخدم وصلاحياته وتنفيذ قواعد العمل، إضافة إلى التعامل مع قاعدة بيانات PostgreSQL. وتعد قاعدة البيانات المصدر الدائم للبيانات الرئيسة مثل المستخدمين والقنوات والعضويات والرسائل والأحداث وسجلات Outbox.

تم فصل عمليات توزيع الرسائل عن خادم FastAPI باستخدام مكون مستقل هو Worker. فبعد حفظ الرسالة وسجل Outbox، لا يحتاج طلب المستخدم إلى انتظار اكتمال عملية التوزيع إلى جميع المشتركين، وإنما يقوم Worker لاحقاً بقراءة سجلات Outbox وتنفيذ عمليات النشر وإعادة المحاولة عند الحاجة. ويساعد هذا الفصل على عدم ربط زمن استجابة الواجهة الخلفية مباشرة بزمن عمل وسيط الرسائل.

يتعامل Worker مع RabbitMQ لتنفيذ التوجيه غير المتزامن للرسائل إلى طوابير المستخدمين المشتركين، بينما يستخدم Redis كوسيط سريع لتحرير الأحداث الخاصة بالمستخدمين المتصلين إلى خادم FastAPI الذي يدير اتصالات WebSocket. وبذلك تم الفصل بين مسار التوزيع الموثوق ومسار الإيصال اللحظي للمستخدمين النشطين.

يسمح هذا الأسلوب لكل مكون بأن يعمل بصورة مستقلة وفق مسؤوليته، كما يسهل اختبار المكونات وتشغيلها وإعادة تشغيل أحدها دون دمج جميع وظائف النظام ضمن عملية واحدة. ويشكل هذا الفصل بين التخزين الدائم والتوزيع غير المتزامن والاتصال اللحظي أحد المبادئ الأساسية التي اعتمد عليها التنفيذ العملي للنظام.

3.7 - تنفيذ الواجهة الخلفية ووظائف النظام

1.3.7 - تنفيذ المصادقة وإدارة الجلسات

تم تنفيذ نظام المصادقة ضمن الواجهة الخلفية بحيث لا يعتمد التحقق من المستخدم على رمز وصول مستقل فقط، وإنما يتم ربط عملية المصادقة بجلسة محفوظة في قاعدة البيانات. ويتيح هذا الأسلوب التحكم في الجلسات وإبطالها عند تسجيل الخروج أو عند اكتشاف استخدام غير صحيح لرموز التحديث.

عند إنشاء حساب جديد، يتم التحقق من البيانات المدخلة والتأكد من عدم تعارض اسم المستخدم أو البريد الإلكتروني مع حساب موجود مسبقاً. ولا يتم تخزين كلمة المرور بصورتها الأصلية، وإنما تتم معالجتها باستخدام خوارزمية **Argon2** وحفظ قيمة التجزئة الناتجة في سجل المستخدم. كما يدعم النظام مسار التحقق من البريد الإلكتروني من خلال إنشاء بيانات تحقق مرتبطة بالحساب قبل اعتماد عملية التحقق.

عند تسجيل الدخول، تبحث خدمة المصادقة عن المستخدم اعتماداً على بيانات الدخول، ثم تتم مقارنة كلمة المرور المدخلة مع قيمة التجزئة المحفوظة. وفي حال نجاح التحقق يتم إنشاء جلسة جديدة للمستخدم في جدول الجلسات، ثم إصدار **Access Token** و **Refresh Token** مرتبطين بهذه الجلسة.

تم تنفيذ رمز الوصول باستخدام **JWT** ليتم إرساله مع الطلبات التي تحتاج إلى مصادقة. ولا يكفي التحقق من صحة توقيع الرمز فقط، بل ترتبط بياناته بجلسة المستخدم، مما يسمح للخادم برفض رمز صادر بصورة صحيحة في حال كانت الجلسة المرتبطة به قد تم إبطالها أو لم تعد صالحة.

أما **Refresh Token** فيستخدم للحصول على رمز وصول جديد دون مطالبة المستخدم بإعادة تسجيل الدخول في كل مرة تنتهي فيها صلاحية **Access Token**. وعند تنفيذ عملية التحديث يتم تطبيق **تدوير رموز التحديث (Refresh Token Rotation)** بحيث يتم استبدال رمز التحديث المستخدم برمز جديد بدلاً من السماح باستخدام الرمز نفسه بصورة متكررة. ويساعد الاحتفاظ بحالة الجلسة على اكتشاف محاولة إعادة استخدام رمز تحديث قديم ورفضها.

عند تسجيل الخروج لا يقتصر التنفيذ على حذف بيانات المصادقة من الواجهة الأمامية، وإنما يتم إبطال الجلسة المقابلة لها في الخادم. ونتيجة لذلك لا يمكن الاستمرار في استخدام الجلسة نفسها للوصول إلى العمليات المحمية أو للحصول على رموز وصول جديدة.

أما المسارات المحمية في الواجهة الخلفية، فتقوم أولاً باستخراج بيانات المصادقة والتحقق من رمز الوصول والجلسة المرتبطة به، ثم تحديد المستخدم الحالي قبل السماح بتنفيذ قواعد العمل الخاصة بالطلب. وبهذا أصبحت آلية المصادقة موحدة بين الوظائف المختلفة مثل إدارة القنوات والعضويات والرسائل والملفات.

وقد وفر هذا التنفيذ فصلاً بين هوية المستخدم وحالة الجلسة، مع دعم إبطال الجلسات وتدوير رموز التحديث والكشف عن إعادة استخدامها، بدلاً من الاعتماد على JWT غير مرتبط بحالة مخزنة في الخادم.

2.3.7- تنفيذ القنوات والعضويات والصلاحيات

تم تنفيذ إدارة القنوات والعضويات ضمن الواجهة الخلفية من خلال مجموعة من المسارات والخدمات التي تتحكم في إنشاء القناة وإعداداتها وأعضائها والصلاحيات المرتبطة بكل مستخدم. وتبدأ العمليات من واجهات API الخاصة بالقنوات والعضويات، ثم يتم تمريرها إلى ChannelService لتنفيذ قواعد العمل وتحديث البيانات الدائمة في PostgreSQL.

عند إنشاء قناة جديدة، يقوم النظام بإنشاء سجل القناة وسجل العداد المرتبط بها، ثم ينشئ عضوية لمالك القناة ويمنحه الدور الإداري الأعلى فيها. كما يتم تسجيل حدث channel.created ضمن سجل الأحداث. وبعد تثبيت التغييرات في قاعدة البيانات تتم معالجة الحالة اللازمة لربط مالك القناة بمسار التوزيع الخاص بها.

يوفر النظام مجموعة متكاملة من العمليات لإدارة العضويات، تشمل الانضمام إلى القناة ومغادرتها، واستعراض الأعضاء وطلبات الانضمام المعلقة، وإرسال الدعوات وقبولها، والموافقة على طلبات العضوية، وإضافة مستخدم بصورة مباشرة، وإزالة عضو من القناة. وتوجد هذه العمليات ضمن واجهات العضويات وتنفذ قواعدها الأساسية داخل خدمة القنوات.

تختلف نتيجة عملية الانضمام بحسب سياسة القناة وحالة المستخدم؛ ففي الحالات التي تتطلب موافقة إدارية يتم الاحتفاظ بالطلب كعضوية معلقة إلى أن يقوم مستخدم مخول بالموافقة عليه. وعند اعتماد العضوية يصبح المستخدم مشتركاً فعلياً في القناة، ويتم تحديث حالة التوزيع الخاصة به بحيث يمكن توجيه رسائل القناة إليه. ويطبق المسار نفسه عند قبول دعوة أو عند إضافة عضو بصورة مباشرة.

كما يدعم النظام إدارة الأدوار داخل القناة، بما في ذلك ترقية الأعضاء وخفض صلاحياتهم وتعديل الصلاحيات الإدارية وإزالة الأعضاء. ولا تعتمد الحماية على إظهار أو إخفاء الأزرار في الواجهة الأمامية، وإنما تتم إعادة التحقق من الدور والصلاحيات في الواجهة الخلفية قبل تنفيذ العمليات الحساسة. وتتركز قواعد التحكم بالوصول في طبقة rbac.py إلى جانب فحوص إضافية ضمن خدمات القنوات والرسائل.

وترتبط العضوية أيضاً بعملية توزيع الرسائل، لأن كون المستخدم عضواً مصرحاً له في القناة يحدد ما إذا كان من الممكن تسجيله ضمن مسارات التوزيع واستقبال أحداثها. لذلك يتم تحديث حالة الربط الخاصة بالمستخدم عند عمليات مثل الانضمام وقبول الدعوة والإضافة المباشرة والموافقة على طلب العضوية، بينما تؤدي إزالة العضوية إلى إنهاء حق المستخدم في استقبال الأحداث اللاحقة الخاصة بالقناة.

أما الدعوات الموجهة لمستخدم محدد، فقد تم تعزيز تنفيذها بحيث يتم ربط الدعوة بحساب المستخدم بصورة ثابتة عند توفره، بدلاً من الاعتماد فقط على قيمة بريد إلكتروني قابلة للتغيير. أما الدعوات الموجهة إلى بريد لم يسجل بعد فتتطلب عند القبول تطابق البريد الحالي بعد تطبيعته وأن يكون البريد قد تم التحقق منه، وذلك لمنع انتقال الدعوة إلى حساب آخر بصورة غير صحيحة.

وقد سمح هذا التنفيذ بربط إدارة القنوات والعضويات بنظام الصلاحيات ومسار توزيع الرسائل، بحيث لا تعتبر العضوية مجرد بيانات معروضة في الواجهة، وإنما تمثل جزءاً من قواعد الوصول والتوجيه التي يعتمد عليها النظام أثناء التشغيل.

3.3.7- تنفيذ الرسائل

تم تنفيذ وظائف الرسائل ضمن الواجهة الخلفية من خلال مسارات API مخصصة وخدمة MessageService التي تحتوي على قواعد العمل المرتبطة بإنشاء الرسالة واسترجاعها والتأكد من صلاحية المستخدم للتعامل معها. وتبقى PostgreSQL المصدر الدائم للرسائل، بينما تستخدم مكونات التوزيع والاتصال اللحظي لإيصال الرسالة بعد حفظها.

عند إرسال المستخدم رسالة إلى قناة، تتحقق الواجهة الخلفية أولاً من هوية المستخدم ووجود القناة ومن امتلاكه عضوية وصلاحيات تسمح له بالنشر فيها. كما يتم التحقق من صحة محتوى الطلب والمرفقات المرتبطة به قبل البدء بعملية الحفظ، بحيث يتم رفض العملية قبل إنشاء الرسالة في حال عدم تحقق الشروط المطلوبة.

بعد نجاح عمليات التحقق، يتم إنشاء سجل Message في قاعدة البيانات وربطه بالقناة والمرسل. ويحتفظ النظام بالبيانات اللازمة لترتيب الرسائل داخل القناة وتمكين المستخدمين من استرجاعها لاحقاً بصورة منتظمة. كما يتم تطبيق الحماية المعتمدة للبيانات الحساسة أثناء التخزين قبل حفظها في قاعدة البيانات.

ولا يتم حفظ الرسالة بصورة منفصلة عن عملية التوزيع، وإنما يتم إنشاء سجل Outbox يمثل العمل المطلوب لتنفيذ توزيعها. ويتم حفظ الرسالة وسجل Outbox ضمن نفس المعاملة في قاعدة البيانات، بحيث لا تظهر حالة تم فيها حفظ الرسالة دون تسجيل مهمة التوزيع المقابلة لها أو العكس.

كما يتم إنشاء الحدث المناسب في سجل الأحداث للعمليات التي تتطلب التدقيق، مما يسمح للنظام بتسجيل العمليات المهمة المرتبطة بالرسائل والمحافظة على أثر يمكن التحقق منه لاحقاً. وبعد نجاح المعاملة يعيد الخادم بيانات الرسالة المحفوظة إلى الواجهة الأمامية، بينما يستكمل مكوّن Worker عملية توزيعها بصورة غير متزامنة.

أما عند استعراض رسائل قناة، فتتحقق الواجهة الخلفية أولاً من صلاحية المستخدم للوصول إلى القناة، ثم تسترجع الرسائل المحفوظة من PostgreSQL وفق ترتيبها. ويسمح الاعتماد على التخزين الدائم للمستخدم باستعراض السجل السابق واسترجاع الرسائل التي لم تصله أثناء انقطاع الاتصال اللحظي.

وبذلك تم فصل عملية قبول الرسالة وحفظها بصورة موثوقة عن عملية توزيعها إلى المشتركين؛ حيث تكون مسؤولية الواجهة الخلفية هي التحقق من الطلب وتثبيت الرسالة وحالة التوزيع المطلوبة في قاعدة البيانات، بينما تتولى المكونات الخلفية تنفيذ عملية التوزيع بعد ذلك.

4.3.7- تنفيذ الملفات والمرفقات

تم تنفيذ دعم الملفات والمرفقات بحيث تكون عملية رفع الملف منفصلة عن عملية نشر الرسالة، ثم يتم ربط الملف بالرسالة بعد التحقق من ملكيته وصلاحيته للاستخدام. ويحتفظ النظام بسجل Upload للبيانات الوصفية الخاصة بالملف، بينما تستخدم سجلات MessageAttachment لربط الملفات بالرسائل والقنوات التي تنتمي إليها.

عند رفع ملف، لا تقوم الواجهة الخلفية بتحميل محتواه كاملاً إلى الذاكرة، وإنما تتم معالجة جسم الطلب بصورة متدفقة (Streaming) مع فرض حدود على الحجم والتحقق من الحجم الفعلي وقيمة التجزئة الخاصة بالمحتوى. ويساعد ذلك على التعامل مع الملفات دون تخصيص ذاكرة بحجم الملف الكامل.

قبل حفظ الملف بصورة نهائية، يتم تشفير محتواه على مستوى الخادم باستخدام AES-256-GCM. وتتم معالجة الملف على شكل أجزاء بحجم محدود، حيث يتم تشفير كل جزء بصورة مستقلة مع بيانات تحقق مرتبطة به. ولا يتم إنشاء نسخة كاملة غير مشفرة من الملف بجانب النسخة النهائية أثناء عملية الرفع، وإنما تكتب البيانات المشفرة إلى ملف مؤقت ثم يتم تثبيتها في المسار النهائي بصورة ذرية بعد نجاح العملية.

تحتفظ قاعدة البيانات كذلك بمعلومات إصدار التشفير ومعرّف المفتاح المستخدم للملف، مما يسمح للنظام بمعرفة المفتاح المطلوب عند القراءة ودعم تغيير المفاتيح مستقبلاً دون الحاجة إلى تجربة جميع المفاتيح المتوفرة. كما تبقى معلومات الحجم وقيمة التجزئة مرتبطة بالمحتوى المنطقي الأصلي للملف وليس بحجم البيانات المشفرة على القرص.

عند إرفاق ملف برسالة، يتحقق النظام من أن الملف صالح للاستخدام وأن المستخدم مخول لربطه بالرسالة، ثم يتم إنشاء العلاقة بين Message و Upload من خلال MessageAttachment. كما تم فرض قيد على مستوى قاعدة البيانات يضمن أن القناة المسجلة للمرفق تتوافق مع القناة التي تنتمي إليها الرسالة، بحيث لا يعتمد هذا الشرط على الشيفرة البرمجية وحدها.

أما عملية تحميل الملف فلا تعتمد على معرفة معرف الملف فقط، وإنما تبدأ بالمصادقة ثم البحث عن سجل الملف والتحقق من صلاحية المستخدم للوصول إليه. فإذا كان الملف مرفقاً برسالة داخل قناة، يتم الاعتماد على العلاقة بين الرسالة والقناة والعضوية لتحديد إمكانية الوصول، وبذلك لا يستطيع مستخدم خارج القناة الحصول على محتوى مرفق خاص بمجرد معرفة معرفه.

بعد نجاح التحقق من الصلاحية، يتم التحقق من سلامة معلومات الملف المشفر ثم فك تشفير البيانات بصورة متدفقة وإرسالها إلى العميل. وتبقى عملية فك التشفير محدودة الأجزاء بدلاً من تحميل الملف المشفر أو غير المشفر كاملاً في الذاكرة قبل بدء الإرسال.

وبذلك يجمع تنفيذ المرفقات بين التحكم في الوصول والحماية أثناء التخزين، حيث تستخدم PostgreSQL للاحتفاظ بالبيانات الوصفية والعلاقات، بينما تحفظ بيانات الملفات بصورة مشفرة، ولا يتم كشف المحتوى غير المشفر إلا بعد نجاح عمليات المصادقة والتحقق من الصلاحية.

4.7- تنفيذ التوزيع غير المتزامن والاتصال اللحظي

1.4.7- تنفيذ Transactional Outbox

تم تطبيق نمط **Transactional Outbox** لربط عملية حفظ الرسالة بعملية توزيعها دون الحاجة إلى إرسال الرسالة مباشرة إلى RabbitMQ أثناء معالجة طلب المستخدم. ويعتمد التنفيذ على PostgreSQL بوصفها المصدر الدائم للحقيقة، بحيث يتم تسجيل نية التوزيع داخل قاعدة البيانات نفسها قبل أن تبدأ عملية النشر إلى وسيط الرسائل.

عند نجاح التحقق من طلب نشر رسالة، يتم ضمن المعاملة نفسها إنشاء سجل الرسالة وسجل Outbox المرتبط بعملية توزيعها، إضافة إلى الحدث المطلوب في سجل التدقيق. ولا يتم اعتبار العملية ناجحة بصورة جزئية؛ فإذا فشل تثبيت أحد هذه التغييرات يتم التراجع عن المعاملة، أما عند نجاحها فتكون الرسالة وطلب توزيعها محفوظين بصورة متوافقة في PostgreSQL.

بعد تثبيت المعاملة، لا تقوم الواجهة الخلفية بانتظار اكتمال التوزيع إلى المشتركين قبل إعادة الاستجابة إلى المستخدم. وبدلاً من ذلك يبقى سجل Outbox محفوظاً ليقوم مكوّن **Worker** بمعالجته بصورة مستقلة في الخلفية. ويؤدي ذلك إلى فصل زمن تنفيذ طلب النشر عن زمن الاتصال بـ RabbitMQ وتوزيع الرسالة.

يقوم **Worker** دورياً بالبحث عن سجلات Outbox التي تحتاج إلى معالجة، ثم يحجز السجل المناسب قبل البدء بالنشر لمنع عدة عمليات معالجة من التعامل مع السجل نفسه في الوقت ذاته. وبعد ذلك يستخدم البيانات الموجودة في السجل لبناء الحدث المطلوب وإرساله إلى RabbitMQ عبر مسار التوجيه المناسب.

إذا نجحت عملية النشر، يتم تحديث حالة سجل Outbox لتعكس نجاح المعالجة. أما إذا حدث خطأ أثناء الاتصال بوسيط الرسائل أو أثناء النشر، فلا تفقد الرسالة الأصلية ولا نية توزيعها، لأن السجل يبقى محفوظاً في PostgreSQL ويمكن جدولته لإعادة المحاولة لاحقاً. وتسمح هذه الآلية بتتبع حالة التوزيع بدلاً من الاعتماد على السجلات النصية فقط.

كما أن تصميم التسليم في المشروع لا يدعي تحقيق **Exactly-Once Delivery**، وإنما يعتمد على تسليم قابل لإعادة المحاولة وموجه نحو **At-Least-Once Delivery**. ولهذا يجب أن تكون عمليات المعالجة قادرة على التعامل بصورة صحيحة مع احتمال إعادة معالجة الحدث عند حدوث بعض حالات الفشل.

وبذلك أصبح سجل Outbox يمثل الحد الفاصل بين عملية الحفظ الموثوقة داخل PostgreSQL وعملية التوزيع غير المتزامنة عبر RabbitMQ، وهو ما يسمح باستمرار عملية التسليم حتى عند حدوث توقف مؤقت في **Worker** أو وسيط الرسائل دون فقدان الرسالة المحفوظة. ويبين الشكل رقم (24) حالة الحاويات المختلفة أثناء تشغيل مكونات النظام.

```
abdalmajeed@abdalmajeed: /mnt/d/Users/abdalmajeed/Desktop/FYP/MessagingSystem$ docker compose ps -a
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
messaging-system-backend-1	messaging-system-backend	"sh -c 'alembic upgr..."	backend	11 seconds ago	Up 8 seconds (healthy)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
messaging-system-frontend-1	messaging-system-frontend	"docker-entrypoint.s..."	frontend	10 seconds ago	Up 2 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
messaging-system-postgres-1	postgres:16	"docker-entrypoint.s..."	postgres	2 days ago	Up 27 seconds (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
messaging-system-rabbitmq-1	rabbitmq:3.13-management	"docker-entrypoint.s..."	rabbitmq	2 days ago	Up 27 seconds (healthy)	0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
messaging-system-redis-1	redis:7	"docker-entrypoint.s..."	redis	2 days ago	Up 27 seconds (healthy)	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
messaging-system-worker-1	messaging-system-worker	"python -m worker ap..."	worker	10 seconds ago	Up 2 seconds	

الشكل رقم 24 حالة الحاويات وهي تعمل

2.4.7- تنفيذ Worker و RabbitMQ

تم تنفيذ مكوّن **Worker** كتطبيق مستقل عن الواجهة الخلفية، وتمثل إحدى مسؤولياته الرئيسة في نقل عمليات التوزيع المسجلة في PostgreSQL إلى RabbitMQ. ويعمل بصورة غير متزامنة بحيث يمكنه معالجة عمليات التوزيع دون إيقاف خادم FastAPI أو ربط زمن استجابة المستخدم بزمن عمل وسيط الرسائل.

يبدأ مسار النشر بقراءة Worker لسجلات Outbox المستحقة للمعالجة. وبعد حجز السجل المطلوب، يقوم ببناء بيانات الحدث ونشرها إلى **Topic Exchange** في RabbitMQ. ويستخدم النظام Exchange دائم باسم `ex.channels`، كما تنشر الرسائل في مسار الوسيط بوضع تخزين دائم حتى لا يكون توجيه الرسالة معتمداً فقط على ذاكرة وسيط الرسائل. تعتمد عملية التوزيع على ربط العضويات في PostgreSQL بحالة الاشتراك في RabbitMQ. فعند امتلاك المستخدم عضوية فعالة في قناة، يتم إنشاء أو المحافظة على الربط المناسب بين طابوره ومسار القناة. وعندما ينشر حدث خاص بهذه القناة، يقوم RabbitMQ بتطبيق قواعد التوجيه وإرسال نسخة منه إلى طوابير المستخدمين المرتبطين بها، وبذلك ينشر المرسل الحدث مرة واحدة بينما يتولى الوسيط عملية توزيعه على المشتركين.

تم استخدام **طوابير دائمة لكل مستخدم** بدلاً من الاعتماد على الاتصال اللحظي وحده. لذلك يستطيع RabbitMQ الاحتفاظ بالرسائل الموجهة إلى الطابور حتى عندما لا يكون المستخدم متصلاً بواجهة WebSocket في لحظة النشر. ويظل PostgreSQL مع ذلك المصدر الأساسي للحقيقة، كما تتوفر واجهات `History` و `Sync` كمسار آخر لاسترجاع البيانات المحفوظة عند إعادة الاتصال.

ولا تنتهي مسؤولية Worker عند نشر سجل Outbox. فهو يتعامل أيضاً مع استهلاك طوابير المستخدمين النشطين، وعند وجود مستخدم متصل يتم تمرير الحدث المستهلك إلى Redis، ليصبح متاحاً لخادم FastAPI المسؤول عن اتصال WebSocket الخاص بذلك المستخدم. وبذلك يشكل Worker حلقة الوصل بين مسار التوزيع الدائم في RabbitMQ ومسار الإيصال اللحظي عبر Redis و WebSocket.

كما يستخدم التنفيذ آليات التأكيد التي يوفرها RabbitMQ أثناء النشر والمعالجة، بحيث يمكن لـ Worker التمييز بين العملية الناجحة والفشل الذي يحتاج إلى معالجة لاحقة. ولا يعني ذلك ضمان وصول الحدث مرة واحدة فقط، وإنما يعمل النظام وفق تصميم قابل لإعادة المحاولة وموجه نحو التسليم من نوع **At-Least-Once**.

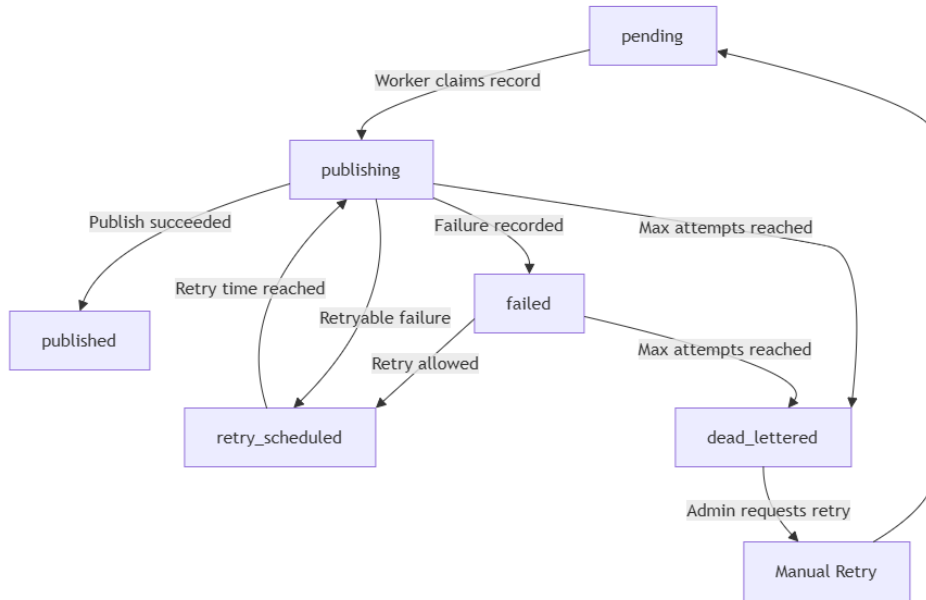
أدى هذا التنفيذ إلى فصل الناشر عن المشتركين بصورة فعلية؛ فالواجهة الخلفية لا تحتاج إلى معرفة جميع المستخدمين الذين يجب إرسال الرسالة إليهم لحظة معالجة الطلب، وإنما تنشر عملية التوزيع بصورة غير متزامنة، بينما يعتمد RabbitMQ

على الطوابير والروابط الحالية لتحديد المستلمين. ويمثل ذلك التطبيق العملي لنموذج النشر والاشتراك الذي يقوم عليه المشروع.

3.4.7- معالجة الفشل وإعادة المحاولة

تم تنفيذ آلية لمعالجة حالات فشل توزيع الرسائل بحيث يتم الاحتفاظ بحالة كل عملية نشر داخل سجل Outbox في PostgreSQL. ولا يؤدي فشل الاتصال بـ RabbitMQ أو فشل عملية النشر إلى حذف العملية أو فقدان الرسالة، وإنما يتم تحديث حالة سجل Outbox والاحتفاظ بالمعلومات اللازمة لإعادة المحاولة لاحقاً.

يمكن أن تمر عملية التوزيع بعدة حالات وتظهر هذه الحالات ومسارات الانتقال بينها في الشكل رقم (25) هي pending و publishing و published و retry_scheduled و failed و dead_lettered. كما يحتفظ سجل Outbox بعدد المحاولات والحد الأقصى المسموح لها وموعد المحاولة التالية وآخر خطأ مسجل، إضافة إلى وقت نجاح النشر أو وقت الانتقال إلى حالة الفشل النهائي.



الشكل رقم 25 حالات معالجة سجل Outbox وإعادة المحاولة عند الفشل

عندما يختار Worker سجلاً جاهزاً للمعالجة، تنتقل حالته إلى publishing أثناء تنفيذ عملية النشر. فإذا نجح RabbitMQ في قبول الرسالة يتم تحديث الحالة إلى published، وبذلك يصبح واضحاً أن عملية النشر إلى وسيط الرسائل قد تمت بنجاح.

أما عند فشل عملية النشر، فيتم زيادة عدد المحاولات وتحديد موعد جديد للمحاولة التالية. ويستخدم النظام **Exponential Backoff** لزيادة فترة الانتظار بين المحاولات المتتالية، بدلاً من إعادة تنفيذ العملية بصورة مستمرة وسريعة عند استمرار سبب الفشل. ويمكن ضبط عدد المحاولات وفترة الانتظار ومعامل الزيادة والحد الأعلى من خلال إعدادات بيئة التشغيل.

لا يقوم Worker بإعادة قراءة جميع السجلات الفاشلة في كل دورة، وإنما يختار سجلات pending والسجلات الموجودة في حالة retry_scheduled التي حان وقت محاولتها التالية. ويساعد ذلك على منع حدوث حلقة محاولات مكثفة قد تستهلك موارد النظام عندما يكون RabbitMQ غير متاح بصورة مؤقتة.

عند استنفاد العدد الأقصى من المحاولات، تنتقل عملية التوزيع إلى الحالة dead_lettered. وتبقى هذه الحالة مسجلة في PostgreSQL بوصفها المرجع الأساسي لحالة عملية التسليم. ويحاول Worker كذلك نسخ بيانات العملية الفاشلة إلى Dead-Letter Exchange في RabbitMQ باسم ex.channels.dlx والطابور q.dead.messages لتوفير وسيلة إضافية للمراقبة التشغيلية، إلا أن نجاح هذه النسخة ليس شرطاً لحفظ حالة الفشل، لأن PostgreSQL تبقى مصدر الحقيقة.

تم كذلك توفير واجهات إدارية وشاشة **Delivery Monitor** تسمح لمالك القناة أو المستخدم الإداري المخول باستعراض إحصائيات التوزيع والعمليات الفاشلة أو التي وصلت إلى حالة dead_lettered. ويمكن من خلالها طلب إعادة محاولة عملية محددة، وعند ذلك تتم إعادة السجل إلى الحالة pending. ليتمكن Worker من معالجته من جديد، كما يتم تسجيل حدث broker.manual_retry_requested ضمن سجل الأحداث.

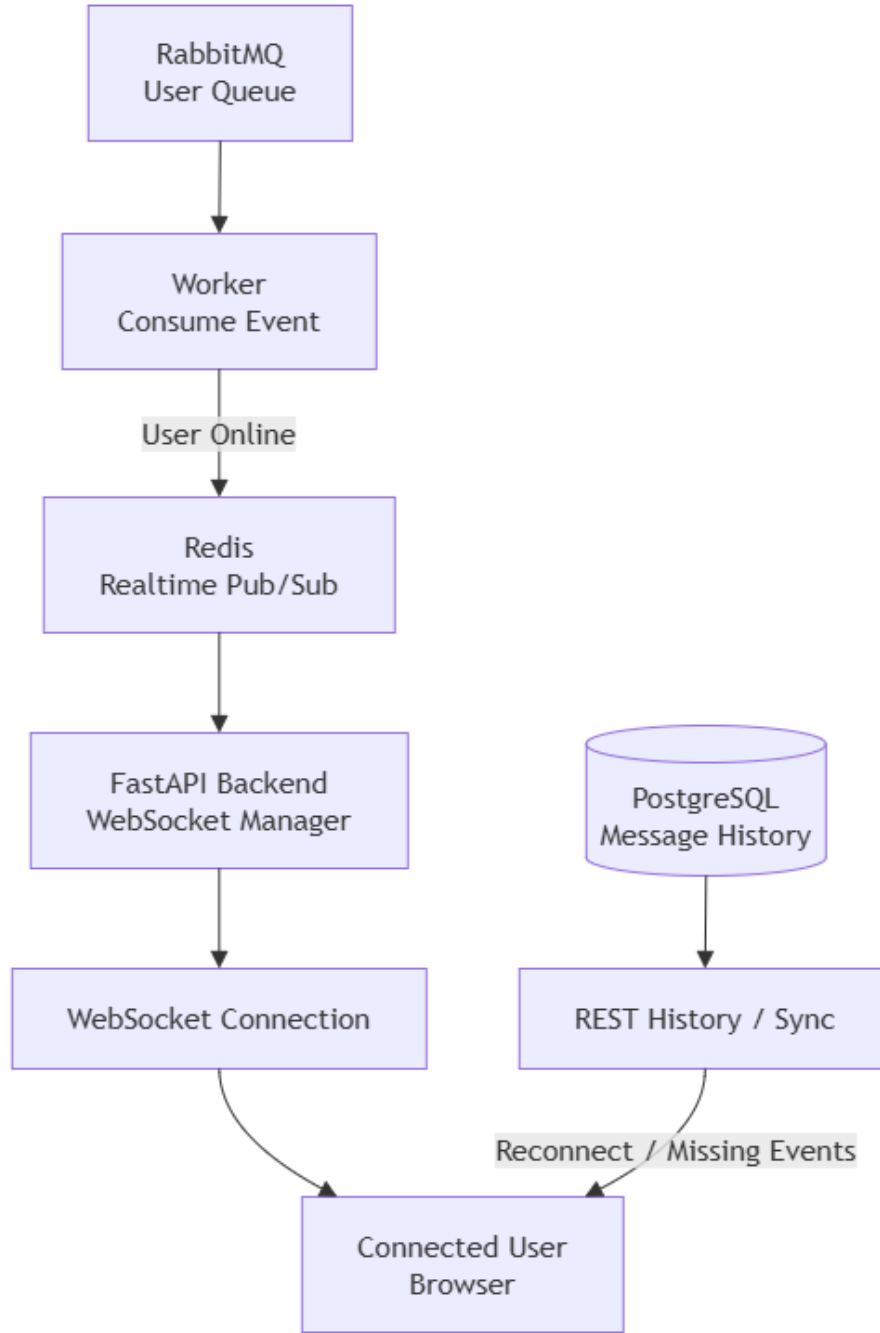
وبذلك تصبح حالات فشل التوزيع قابلة للملاحظة والتتبع وإعادة المعالجة، بدلاً من أن تكون أخطاء غير مرئية داخل سجلات التشغيل فقط. ويظل نموذج التسليم المستخدم موجهاً نحو **At-Least-Once Delivery**، لذلك لا يدعي النظام ضمان تنفيذ عملية التسليم مرة واحدة فقط.

4.4.7- الاتصال اللحظي والمزامنة

تم تنفيذ الاتصال اللحظي بصورة مستقلة عن التخزين الدائم للرسائل، بحيث لا يعتمد النظام على WebSocket بوصفه الوسيلة الوحيدة لمعرفة الرسائل الجديدة. وتبقى PostgreSQL المصدر الأساسي للبيانات، بينما يستخدم Redis وWebSocket لتسريع إيصال الأحداث إلى المستخدمين المتصلين.

بعد وصول حدث الرسالة إلى طابور المستخدم في RabbitMQ، يقوم مكوّن Worker بمعالجة الحدث. وعندما يكون المستخدم متصلاً بالنظام يتم تمرير الحدث إلى Redis، الذي يستخدم كقناة تنسيق بين Worker وخوادم الواجهة الخلفية المسؤولة عن اتصالات WebSocket.

تحتفظ الواجهة الخلفية باتصالات WebSocket الفعالة للمستخدمين من خلال مكوّن مخصص لإدارة الاتصال اللحظي. وعند وصول حدث خاص بمستخدم متصل من Redis، يتم تحديد الاتصال المناسب وإرسال الحدث إلى المتصفح، فتمكن الواجهة الأمامية من تحديث المحادثة دون تنفيذ طلب HTTP جديد للحصول على كل رسالة. ويوضح الشكل رقم (26) تكامل المسار اللحظي مع آلية المزامنة بعد انقطاع الاتصال.



الشكل رقم 26 مسار الاتصال اللحظي ومزامنة الرسائل بعد انقطاع الاتصال

لا يتم السماح بإنشاء اتصال WebSocket اعتماداً على معرف المستخدم فقط، وإنما تخضع عملية الاتصال للمصادقة والتحقق من الجلسة. ويستخدم النظام آلية رموز قصيرة العمر ومخصصة للاتصال اللحظي، بحيث لا يتم الاعتماد على ترميز بيانات المصادقة طويلة العمر بصورة مباشرة داخل عنوان اتصال WebSocket.

ومع ذلك لا يعتبر وصول حدث WebSocket دليلاً وحيداً على وجود الرسالة. فقد ينقطع الاتصال بالإنترنت أو يعاد تشغيل خادم الواجهة الخلفية أو يفقد المستخدم الاتصال اللحظي خلال فترة قصيرة. وفي هذه الحالات تبقى الرسائل محفوظة في PostgreSQL ولا تعتبر مفقودة.

عند فتح قناة أو إعادة الاتصال، تستطيع الواجهة الأمامية استرجاع سجل الرسائل من الواجهة الخلفية من خلال واجهات REST. كما يوفر النظام آلية للمزامنة تسمح للعميل بطلب البيانات التي حدثت بعد آخر حالة معروفة لديه، وبذلك يمكن تعويض الرسائل أو الأحداث التي لم تصل أثناء فترة انقطاع الاتصال.

يتم الاحتفاظ بحالة مرتبطة بالمستخدم والقناة للمساعدة على معرفة نقطة التقدم التي وصل إليها العميل، واستخدامها في عمليات المزامنة. وعند استرجاع البيانات تعتمد الواجهة الخلفية على السجلات الدائمة وليس على محتوى Redis، لأن Redis يستخدم للتنسيق اللحظي ولا يمثل قاعدة البيانات المرجعية للرسائل.

وبذلك يعمل المساران بصورة متكاملة؛ يوفر WebSocket تجربة استخدام لحظية عندما يكون المستخدم متصلاً، بينما توفر واجهات History و Sync وسيلة لاستعادة الحالة الصحيحة عند إعادة الاتصال. ويمنع هذا التصميم تحويل الاتصال اللحظي إلى نقطة فشل تؤدي إلى فقدان البيانات.

5.7- تنفيذ الأمان وسلامة سجل الأحداث

1.5.7- حماية الحسابات والبيانات

تم تنفيذ الحماية الأمنية في النظام على عدة مستويات تشمل كلمات المرور والجلسات وواجهات البرمجة والصلاحيات واتصالات WebSocket والملفات والبيانات المخزنة. ولا يعتمد النظام على آلية حماية واحدة، وإنما يتم تطبيق مجموعة من الفحوص والقيود في نقاط مختلفة من مسار الطلب. بالنسبة إلى كلمات المرور، لا يتم تخزين القيمة الأصلية للمستخدم في قاعدة البيانات، وإنما يتم تحويلها إلى قيمة تجزئة باستخدام خوارزمية Argon2. وعند تسجيل الدخول تتم مقارنة كلمة المرور المدخلة بقيمة التجزئة المحفوظة بدلاً من استرجاع كلمة المرور الأصلية.

أما المصادقة بعد تسجيل الدخول فتستخدم JWT Access Tokens مرتبطة بجلسات محفوظة في PostgreSQL. ويسمح هذا الربط للخادم بالتحقق من أن الرمز لم يصدر فقط بصورة صحيحة، وإنما أن الجلسة المرتبطة به لا تزال فعالة. كما تم تنفيذ تدوير رموز التحديث Refresh Token Rotation وإبطال الجلسات عند تسجيل الخروج، إضافة إلى التعامل مع محاولات إعادة استخدام رمز تحديث قديم.

تم تطبيق التحكم في الوصول ضمن الواجهة الخلفية نفسها، حيث يتم التحقق من هوية المستخدم وعضويته ودوره قبل تنفيذ العمليات الحساسة. فعلى سبيل المثال، لا يكفي أن تظهر للمستخدم واجهة إدارة القناة حتى يستطيع تعديلها، وإنما يعاد التحقق في الخادم من امتلاكه الدور المطلوب قبل تغيير الإعدادات أو إدارة الأعضاء أو تنفيذ العمليات الإدارية.

ويستخدم النظام مبدأ الأدوار والصلاحيات RBAC لتحديد العمليات المتاحة للمستخدم ضمن القناة. ويتم تطبيق هذه الفحوص في الخدمات والمسارات التي تتعامل مع الموارد المحمية مثل القنوات والعضويات والرسائل والمرفقات، بحيث يتم رفض الطلب غير المصرح به حتى لو تم إرساله مباشرة إلى واجهة API دون استخدام الواجهة الأمامية.

كما تمت حماية اتصال WebSocket بحيث لا يتم إنشاء الاتصال اعتماداً على معرف مستخدم يرسله العميل فقط. ويستخدم النظام رمز اتصال قصير العمر ومخصص لإنشاء WebSocket، ويتم التحقق منه ومن حالة الجلسة قبل السماح بإنشاء الاتصال. ويحد ذلك من الحاجة إلى استخدام رموز المصادقة طويلة العمر مباشرة داخل عنوان الاتصال اللحظي.

أما الملفات والمرفقات فتخضع للمصادقة والتحقق من الصلاحية قبل السماح بتحميلها. كما يتم تشفير الملفات الجديدة أثناء التخزين باستخدام AES-256-GCM، مع الاحتفاظ بمعلومات إصدار التشفير ومعرف المفتاح المستخدم. وعند التحميل يتم التحقق من الصلاحية أولاً ثم فك تشفير البيانات وإرسالها إلى المستخدم المصرح له.

تم كذلك تطبيق حماية للبيانات الحساسة المخزنة على مستوى الخادم، بحيث لا تقتصر الحماية على التحكم في الوصول إلى قاعدة البيانات فقط، وإنما تستخدم آليات تشفير للبيانات التي تحتاج إلى حماية أثناء التخزين. ولا يمثل هذا النظام تشفيراً من طرف إلى طرف **End-to-End Encryption**، إذ تتم عمليات التشفير وفك التشفير ضمن مكونات الخادم.

إضافة إلى ذلك، تطبق الواجهة الخلفية قيوداً على بعض العمليات والطلبات لتقليل إساءة الاستخدام، مثل الحدود المتعلقة بالمصادقة والاتصال ورفع الملفات. كما يتم التحقق من حجم البيانات وصحة المدخلات قبل معالجتها، ويتم إرجاع أخطاء مناسبة عند تجاوز القيود أو عدم امتلاك الصلاحية المطلوبة.

وقد أدى الجمع بين تجزئة كلمات المرور وإدارة الجلسات والتحقق من الصلاحيات وحماية **WebSocket** والتشفير أثناء التخزين والقيود التشغيلية إلى توفير عدة طبقات دفاعية، بحيث يؤدي تجاوز طبقة واحدة إلى استمرار وجود طبقات أخرى تتحقق من هوية المستخدم وصلاحيته العملية وسلامة البيانات. ويلخص الجدول رقم (13) أهم آليات الحماية الأمنية المطبقة والأجزاء التي تقوم بحمايتها.

الجزء المحمي	الآلية المنفذة
كلمات المرور	Argon2
المصادقة	JWT مرتبط بالجلسة
استمرار الجلسة	Refresh Token Rotation
تسجيل الخروج	إبطال الجلسة
صلاحيات القنوات	RBAC وفحص العضوية
WebSocket	رمز قصير العمر مرتبط بالجلسة
المرفقات	فحص الصلاحية + AES-256-GCM
البيانات الحساسة	تشفير أثناء التخزين
إساءة استخدام الواجهات	حدود للطلبات والتحقق من المدخلات

الجدول رقم 11 آليات الحماية الأمنية المطبقة في النظام

2.5.7- تنفيذ سجل الأحداث

تم تنفيذ سجل أحداث دائم لتوثيق العمليات المهمة التي تحدث داخل النظام، بحيث لا تقتصر مراقبة النشاط على ملفات السجل النصية الخاصة بالخادم. وتُخزن الأحداث ضمن جدول **Event** في **PostgreSQL**، مما يسمح باسترجاعها وعرضها والتحقق من سلامتها لاحقاً. وتتم عملية إنشاء الأحداث بصورة مركزية من خلال خدمة **EventService** والدالة **log_event**.

يحتوي كل حدث على معلومات تصف العملية التي حدثت والسياق المرتبط بها، مثل نوع الحدث، والمستخدم الذي نفذ العملية عند وجوده، والقناة المرتبطة بالحدث، وتاريخ إنشائه، إضافة إلى بيانات تفصيلية ضمن الحمولة **payload**. ويسمح هذا الشكل المنظم بالتمييز بين أنواع الأحداث ومعرفة من قام بالعملية وعلى أي مورد تمت. كما تدخل هذه البيانات نفسها لاحقاً ضمن حساب البصمة المشفرة الخاصة بالحدث.

لا يقتصر التسجيل على نوع واحد من العمليات، وإنما تستدعي خدمات النظام سجل الأحداث عند تنفيذ عدد من الوظائف المهمة. فعلى سبيل المثال يتم تسجيل إنشاء القنوات وتغييرات العضويات ونشر الرسائل، كما يتم تسجيل بعض محاولات الوصول أو النشر غير المصرح بها وأحداث مرتبطة بفشل عملية التوزيع وإعادة المحاولة. وبهذا يقدم السجل صورة عن النشاط الوظيفي والأمني والتشغيلي للنظام.

عند نشر رسالة مثلاً، يتم إنشاء حدث `message.published` إلى جانب الرسالة وسجل `Outbox` ضمن مسار الحفظ، بينما يؤدي إنشاء قناة إلى تسجيل حدث `channel.created`. كما تنتج بعض العمليات الإدارية أحداثاً خاصة بها، مثل طلب إعادة محاولة توزيع فاشل، مما يسمح بربط الإجراءات الإدارية اللاحقة بسجل واضح يمكن مراجعته.

كما يمكن استعراض أحداث القناة من خلال واجهات `API` الخاصة بالأحداث، وتعرض الواجهة الأمامية هذه البيانات ضمن قسم **Event Log** في تفاصيل القناة للمستخدمين المخولين. وقد تم دمج التحقق من سلامة السجل مع هذه الواجهة أيضاً، بحيث يمكن للمستخدم الإداري الانتقال من مشاهدة النشاط إلى التحقق من سلامة السلسلة المرتبطة به.

ولا يُعامل سجل الأحداث على أنه مجرد نص حر، بل تعتمد الأحداث الجديدة على تمثيل منظم وثابت يسمح بحساب قيمة `SHA-256` تغطي البيانات المهمة للحدث وسياقه وعلاقته بالحدث السابق ضمن النطاق نفسه. وقد شكل هذا التصميم الأساس الذي بنيت عليه لاحقاً طبقات التحقق من سلامة السجل باستخدام سلسلة التجزئة وشجرة `Merkle`.

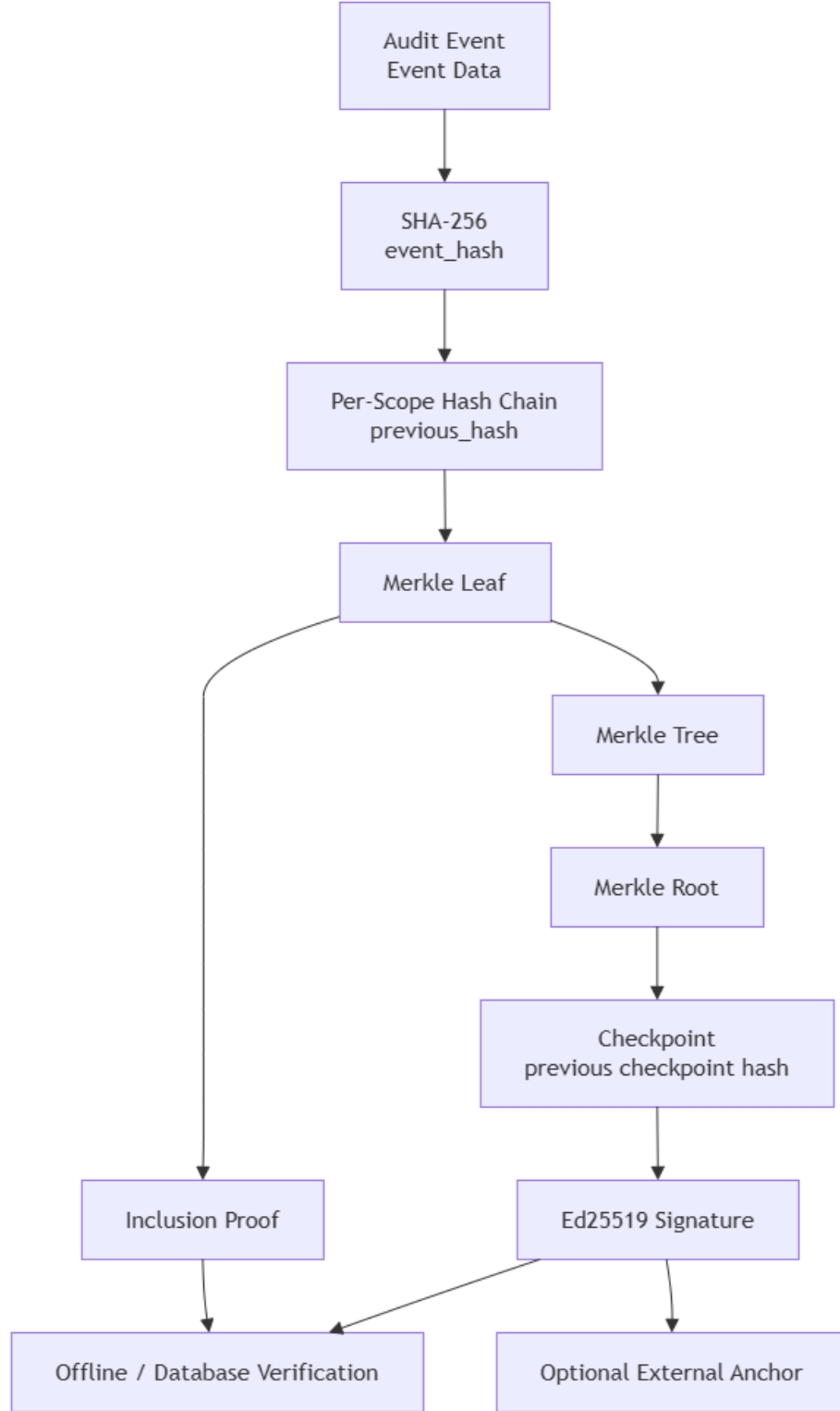
وبذلك يوفر سجل الأحداث أثراً دائماً للعمليات المهمة داخل النظام، ويساعد في متابعة النشاط والتحقيق في العمليات الإدارية ومحاولات الوصول غير المصرح بها وحالات فشل التوزيع، إضافة إلى كونه المصدر الذي تعتمد عليه آليات التحقق المشفر من سلامة سجل التدقيق. ويعرض الجدول رقم (14) أمثلة عن الأحداث التي يسجلها النظام والغرض من تسجيل كل منها.

مثال عن الحدث	الغرض من التسجيل
channel.created	توثيق إنشاء قناة جديدة
message.published	توثيق نشر رسالة
تغييرات العضوية	توثيق الانضمام والموافقة والتغييرات الإدارية
محاولات الوصول غير المصرح	توثيق بعض الأحداث الأمنية
أحداث فشل التوزيع	تتبع مشاكل التسليم وإعادة المحاولة
broker.manual_retry_requested	توثيق طلب إداري لإعادة التسليم

الجدول رقم 12 أمثلة عن الأحداث المسجلة وأغراض تسجيلها

3.5.7- تنفيذ التحقق من سلامة السجل

تم تنفيذ آلية متعددة الطبقات للتحقق من سلامة سجل الأحداث، بهدف اكتشاف التعديل أو الحذف أو كسر تسلسل الأحداث بعد تسجيلها. ويعتمد التنفيذ على سلسلة تجزئة باستخدام `SHA-256` لكل نطاق من الأحداث، ثم يضيف فوقها نقاط تحقق مبنية باستخدام `Merkle Tree` وموقعة رقمياً باستخدام `Ed25519`. ويوضح الشكل رقم (27) تكامل سلسلة التجزئة مع شجرة `Merkle` في آلية التحقق من سلامة السجل.



الشكل رقم 27 آلية التحقق من سلامة سجل الأحداث باستخدام سلسلة التجزئة وشجرة Merkle

يحصل كل حدث جديد على قيمة `event_hash` محسوبة من تمثيل ثابت للبيانات المهمة الخاصة بالحدث، مثل المعرفات ونوع الحدث ووقت الإنشاء والحمولة والسياق المرتبط بالمستخدم أو القناة، إضافة إلى قيمة `previous_hash` وبهذا يرتبط كل حدث بالحدث السابق ضمن نطاقه، سواء كان نطاقاً عاماً للنظام أو خاصاً بقناة معينة. فإذا تم تعديل حدث قديم أو تغيير ترتيبه تنكسر العلاقة بين قيم التجزئة ويمكن اكتشاف ذلك أثناء عملية التحقق.

ولمنع تعارض عدة عمليات كتابة متزامنة أثناء بناء السلسلة، يستخدم التنفيذ أفعال PostgreSQL استشارية **Advisory Locks** على مستوى نطاق سجل الأحداث. ويسمح ذلك بالمحافظة على تسلسل صحيح للأحداث التي تنتمي إلى النطاق نفسه دون الحاجة إلى قفل جدول الأحداث كاملاً.

أضيفت بعد ذلك طبقة **Merkle Checkpoints**، حيث يتم اختيار مجموعة محدودة من الأحداث التي لم تدخل في نقطة تحقق سابقة، وإعادة التحقق من قيم تجزئتها، ثم أخذ نسخة ثابتة من قيم **event_hash** الخاصة بها وتحويلها إلى أوراق في شجرة **Merkle**. ومن هذه الأوراق يتم حساب قيمة واحدة هي **Merkle Root** تمثل التزاماً مشفراً بالمجموعة كاملة.

لا يتم تخزين الجذر وحده، وإنما ينشأ سجل مستقل لنقطة التحقق يحتوي على عدد الأوراق ومعلومات المجموعة والجذر وقيمة نقطة التحقق السابقة. وبهذا ترتبط نقاط التحقق نفسها على شكل سلسلة، مما يسمح باكتشاف التعديل في نقطة قديمة أو محاولة استبدال جزء من تسلسل نقاط التحقق. ويتم حفظ نقطة التحقق وأوراقها ضمن معاملة واحدة في PostgreSQL.

قبل تثبيت نقطة التحقق يتم حساب **checkpoint_hash**. ثم توقيعه باستخدام **Ed25519**. ويستخدم التحقق المفتاح العام الموافق لمعرف المفتاح المخزن مع نقطة التحقق، بينما لا يحتاج خادم التطبيق العادي إلى امتلاك المفتاح الخاص. وفي إعداد التشغيل الإنتاجي تم عزل مفتاح التوقيع الخاص داخل خدمة مخصصة لإنشاء نقاط التحقق، في حين تستخدم بقية المكونات المفاتيح العامة عند الحاجة إلى التحقق.

يوفر النظام أيضاً **Merkle Inclusion Proof**. لإثبات أن حدثاً معيناً كان جزءاً من نقطة تحقق دون الحاجة إلى نقل أو إعادة حساب جميع أحداث المجموعة. ويحتوي الإثبات على قيمة ورقة الحدث وعددها وموقعها والعقد المجاورة لإعادة حساب الجذر. وبعد إعادة بناء المسار تتم مقارنة الجذر الناتج بالجذر الموجود في نقطة التحقق، ثم التحقق من قيمة نقطة التحقق وتوقيع **Ed25519**. ويمكن تنفيذ هذا التحقق خارج قاعدة البيانات باستخدام حزمة الإثبات والمفتاح العام فقط.

تتميز هذه الآلية بأن حجم إثبات الانتماء ينمو تقريباً وفق $O(\log n)$ بدلاً من الحاجة إلى معالجة جميع العناصر $O(n)$. فعلى سبيل المثال يحتاج إثبات حدث ضمن مجموعة متوازنة تحتوي على 1024 حدثاً إلى نحو عشر قيم تجزئة مجاورة فقط. وقد تحقق الاختبار العملي في المشروع من إنشاء نقطة تحقق تضم 1024 حدثاً وتوليد إثبات مدمج والتحقق منه بصورة مستقلة.

كما يدعم النظام تصدير **External Anchor** يحتوي على أحدث التزام موقع لنقطة التحقق. ويمكن حفظ هذا الالتزام بصورة مستقلة عن قاعدة البيانات، بحيث يصبح من الممكن كشف محاولة حذف أحدث نقاط التحقق وإظهار نسخة أقدم صحيحة التوقيع. إلا أن هذه الحماية من التراجع تعتمد على قيام المشغل فعلياً بحفظ **Anchor** في مكان مستقل، ولا يدعي النظام وجود خدمة خارجية آلية لتنفيذ ذلك.

وقد تم اختبار الآلية من خلال تعديل نسخ من بيانات الأحداث وقيم التجزئة وأوراق **Merkle** والجذر والتوقيع ومسار الإثبات، وتمكنت عمليات التحقق من اكتشاف حالات العبث المختلفة. لذلك توصف الآلية بأنها **Tamper-Evident** وقابلة للتحقق تشفيرياً، ولا توصف بأنها سجل غير قابل للتعديل بصورة مطلقة أو **Blockchain**.

6.7 - تنفيذ الواجهة الأمامية ودليل الاستخدام

1.6.7 - واجهة المصادقة والتنقل

تم تنفيذ الواجهة الأمامية باستخدام **Next.js** و **React** لتوفير نقطة التفاعل الأساسية بين المستخدم ووظائف النظام. وتتولى الواجهة عرض النماذج والصفحات وإدارة حالة المستخدم، بينما تبقى عمليات التحقق من الهوية والصلاحيات وقواعد العمل الفعلية ضمن الواجهة الخلفية.

عند فتح النظام دون وجود جلسة مستخدم صالحة تظهر واجهة المصادقة، التي تتيح للمستخدم إنشاء حساب جديد أو تسجيل الدخول باستخدام بيانات حساب موجود. وتقوم الواجهة بإرسال البيانات إلى واجهات **API** الخاصة بالمصادقة، ثم يتم تحديث حالة التطبيق بعد نجاح العملية ونقل المستخدم إلى الواجهة الرئيسية.

تم تجميع منطق المصادقة في الواجهة الأمامية ضمن `use-auth.ts`، حيث يتولى دورة المصادقة الخاصة بالمستخدم، بما في ذلك تسجيل الدخول وإنشاء الحساب وتسجيل الخروج واستعادة الجلسة عند إعادة فتح التطبيق. ويساعد ذلك على إبقاء منطق المصادقة بعيداً عن المكونات المرئية التي تقتصر مهمتها على عرض النماذج والتفاعل مع المستخدم.

بعد نجاح المصادقة يستطيع المستخدم الانتقال إلى الجزء الرئيس من التطبيق والتعامل مع القنوات المتاحة له. وتعمل واجهة التنقل كنقطة وصول إلى الوظائف الرئيسة للنظام، فيستطيع المستخدم اختيار القناة التي يريد التعامل معها والانتقال إلى تفاصيلها، إضافة إلى الوصول إلى الوظائف المتاحة له بحسب صلاحيات حسابه وعضويته في القنوات.

يتغير محتوى الواجهة بحسب حالة المستخدم والصلاحيات التي يعيدها النظام. فبعض الخيارات الإدارية لا تكون مطلوبة للمستخدم العادي، بينما يستطيع مالك القناة أو المستخدم المخول الوصول إلى وظائف إضافية متعلقة بإدارة القناة وأعضائها ومراقبة حالة التوزيع. ومع ذلك لا يعتمد النظام أمنياً على إخفاء هذه الخيارات، لأن الصلاحيات يعاد التحقق منها داخل الواجهة الخلفية قبل تنفيذ العملية.

عند تسجيل الخروج تطلب الواجهة إنهاء جلسة المستخدم ثم تعيد حالة التطبيق إلى الوضع غير المصادق عليه. كما تدعم الواجهة استعادة حالة المصادقة عند إعادة تحميل التطبيق، مما يسمح باستمرار تجربة الاستخدام دون الحاجة إلى إعادة تنفيذ خطوات تسجيل الدخول في كل انتقال داخل التطبيق ما دامت الجلسة صالحة.

وبذلك تشكل واجهة المصادقة والتنقل الطبقة الأولى التي يتعامل معها المستخدم، فهي تربط بين إدارة الجلسة من جهة والوصول المنظم إلى القنوات ووظائف النظام من جهة أخرى، مع إبقاء التحقق الأمني وقواعد الصلاحيات ضمن الواجهة الخلفية.

2.6.7 - واجهات القنوات والعضويات

توفر الواجهة الأمامية مجموعة من الصفحات والمكونات التي تسمح للمستخدم بالتعامل مع القنوات والعضويات دون الحاجة إلى استخدام واجهات API بصورة مباشرة. وتشمل هذه الوظائف إنشاء القنوات وعرض القنوات المتاحة والانضمام إليها ومغادرتها، إضافة إلى إدارة الأعضاء والدعوات وطلبات الانضمام بالنسبة إلى المستخدمين الذين يمتلكون الصلاحيات المناسبة.

يمكن للمستخدم إنشاء قناة جديدة من خلال نموذج مخصص في الواجهة، حيث يدخل البيانات المطلوبة للقناة ثم ترسل الواجهة الطلب إلى الواجهة الخلفية. وبعد نجاح العملية يتم تحديث قائمة القنوات وإتاحة القناة الجديدة للمستخدم الذي أنشأها باعتباره مالِكاً لها.

عند اختيار قناة، يستطيع المستخدم الانتقال إلى واجهتها الرئيسة أو إلى صفحة التفاصيل الخاصة بها. وتمثل `channel-view.tsx` الواجهة الرئيسة للتعامل مع القناة، بينما تتولى `channel-details.tsx` عرض معلومات القناة والوظائف الإدارية وسجل الأحداث. ويساعد هذا الفصل على إبقاء واجهة المراسلة منفصلة عن الوظائف الإدارية الخاصة بالقناة.

تدعم الواجهة عمليات الانضمام إلى القنوات ومغادرتها، كما تعرض حالة العضوية للمستخدم. فإذا كانت القناة تتطلب موافقة إدارية يمكن إرسال طلب انضمام، ثم يظهر الطلب ضمن قائمة الطلبات المعلقة للمستخدمين المخولين بإدارة العضويات. وبعد الموافقة يتم تحديث حالة العضوية وإتاحة الوظائف المرتبطة بالقناة للمستخدم.

بالنسبة إلى مالك القناة والمستخدمين الذين يمتلكون صلاحيات إدارية، توفر صفحة التفاصيل أدوات لإدارة الأعضاء، ومنها عرض قائمة الأعضاء والطلبات المعلقة والدعوات، إضافة إلى ترقية الأعضاء أو خفض أدوارهم وإزالة عضو أو تعديل بعض الصلاحيات الإدارية. كما يدعم النظام إرسال الدعوات وقبولها لإضافة مستخدم إلى القناة.

ولا تعرض الواجهة جميع الخيارات لجميع المستخدمين بصورة متطابقة، وإنما تتكيف الوظائف المتاحة مع دور المستخدم داخل القناة. فالمستخدم العادي يركز على الوصول إلى القناة والمحتوى المسموح له به، بينما تظهر أدوات الإدارة للمستخدم الذي يمتلك الدور المطلوب. وتبقى هذه الاختلافات في الواجهة وسيلة لتحسين تجربة الاستخدام فقط، لأن التحقق النهائي من الصلاحيات يتم دائماً في الواجهة الخلفية.

وتحتوي صفحة تفاصيل القناة كذلك على وظائف إضافية مرتبطة بالإدارة والمراقبة، من بينها سجل الأحداث، وفي الإصدارات التي تتضمن مراقبة التسليم تظهر أيضاً معلومات مرتبطة بحالات التوزيع وإعادة المحاولة للمستخدمين المخولين. وبذلك تجمع صفحة التفاصيل بين إدارة القناة وإدارة العضوية والوصول إلى المعلومات التشغيلية المرتبطة بها.

وبذلك توفر واجهات القنوات والعضويات طبقة استخدام متكاملة فوق الوظائف الموجودة في الواجهة الخلفية، بحيث يستطيع المستخدم تنفيذ دورة العمل الأساسية للقناة من الإنشاء والانضمام وحتى إدارة الأعضاء والصلاحيات من خلال واجهات رسومية واضحة.

3.6.7- واجهة المراسلة

تمثل واجهة المراسلة الجزء الرئيس الذي يتعامل معه المستخدم بعد اختيار إحدى القنوات. وقد تم تنفيذها ضمن المكون `channel-view.tsx` بحيث تجمع بين عرض سجل الرسائل وكتابة الرسائل الجديدة والتعامل مع الخصائص الإضافية المرتبطة بها، بينما تتولى الواجهة الخلفية التحقق من العضوية والصلاحيات وحفظ البيانات بصورة دائمة.

عند فتح القناة تقوم الواجهة بتحميل الرسائل المحفوظة وعرضها بترتيبها داخل القناة، مع المعلومات المرتبطة بكل رسالة مثل المرسل ووقت النشر والمحتوى. وبذلك يستطيع المستخدم استعراض الرسائل السابقة حتى لو لم يكن متصلاً بالنظام وقت نشرها، لأن السجل المعروض يعتمد في النهاية على البيانات الدائمة الموجودة في PostgreSQL .

يوجد في أسفل واجهة القناة حقل مخصص لكتابة الرسائل الجديدة. وعند الإرسال يتم تمرير المحتوى إلى واجهة `API` الخاصة بالنشر، ثم تظهر الرسالة في واجهة القناة بعد نجاح العملية. ولا تتولى الواجهة عملية توزيع الرسالة على بقية المشتركين، وإنما يبدأ بعد ذلك المسار الخلفي الذي يعتمد على `RabbitMQ` والذي تم شرحه في الأقسام السابقة.

أما الرسائل الواردة أثناء بقاء المستخدم داخل التطبيق فتصل من خلال اتصال `WebSocket` ويقوم `use-websocket.tsx` باستقبال الأحداث اللحظية وتحديث البيانات المستخدمة من قبل واجهة `React`، مما يسمح بظهور الرسالة الجديدة داخل القناة دون الحاجة إلى إعادة تحميل الصفحة. وعند فقدان الاتصال يمكن الرجوع إلى مسارات `History` و `Sync` لاستعادة الحالة الصحيحة.

لا تقتصر الواجهة على إرسال النصوص فقط، بل تدعم عدداً من الوظائف التفاعلية المرتبطة بالرسائل، ومنها الرد على رسالة سابقة، وإضافة التفاعلات `(Reactions)`، وتثبيت الرسائل `(Pins)`، إضافة إلى تحديث حالة المشاهدة. كما يدعم منطق الرسائل في النظام تعديل الرسالة وحذفها وفق الصلاحيات المطبقة في الواجهة الخلفية.

يدعم النظام أيضاً إرفاق الملفات بالرسائل. حيث يتم رفع الملف أولاً من خلال المسار المخصص للرفع، ثم يتم ربطه بالرسالة عند النشر. وبعد ظهور المرفق يستطيع المستخدم المخول طلب تنزيله، فتقوم الواجهة الخلفية بالتحقق من صلاحية الوصول قبل إعادة محتوى الملف. وبذلك لا تتعامل واجهة المراسلة مع المرفق كرابط عام مستقل عن القناة، وإنما كمورد محمي مرتبط بالرسالة.

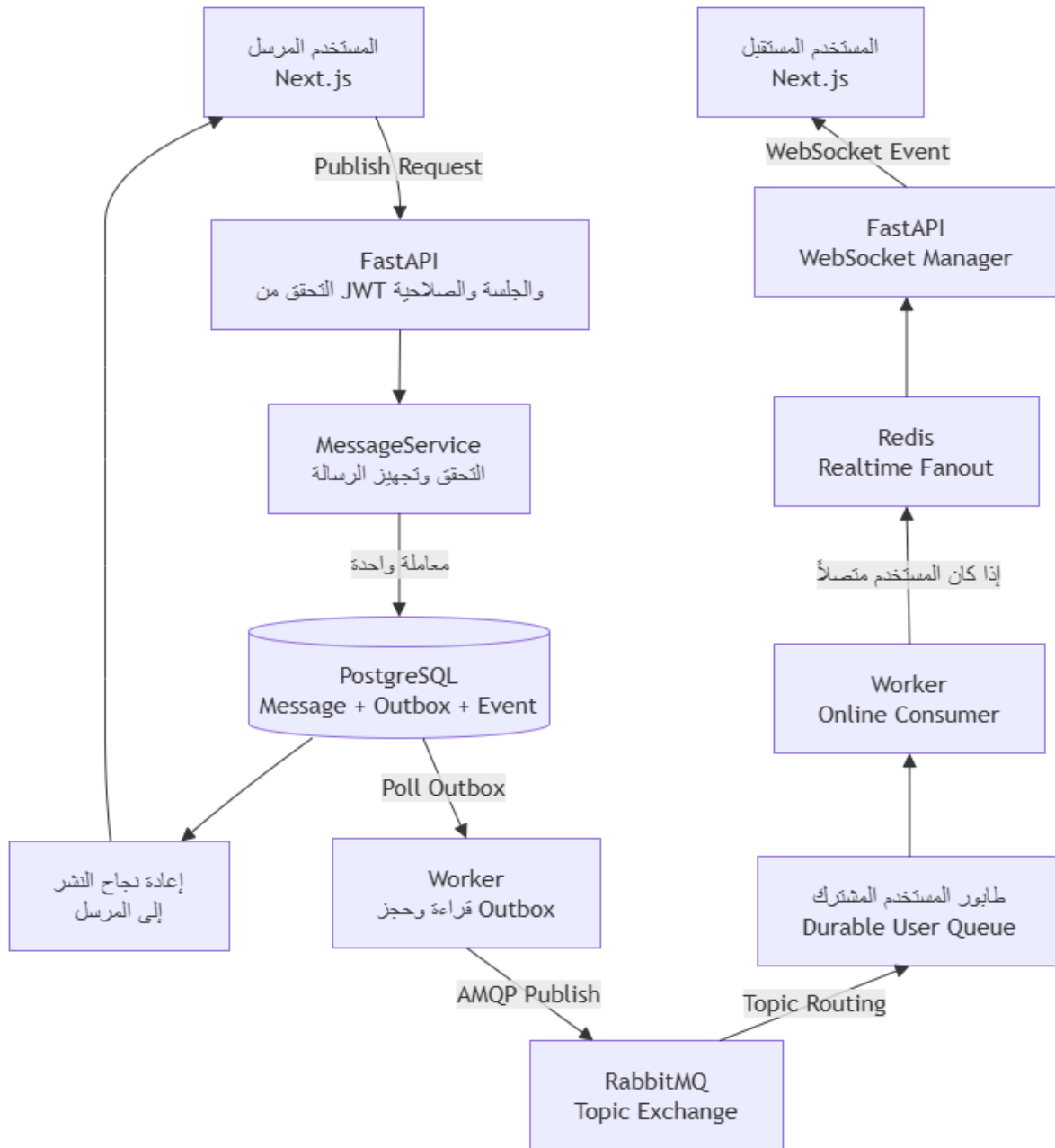
وتتغير بعض الإجراءات الممكن تنفيذها على الرسالة بحسب صاحب الرسالة ودور المستخدم في القناة. فالمستخدم العادي لا يمتلك بالضرورة العمليات الإدارية نفسها التي يستطيع مالِك القناة أو المستخدم المخول تنفيذها، بينما يبقى التحقق النهائي من تلك الصلاحيات في الخادم وليس في عناصر الواجهة فقط.

وقد أدى دمج سجل الرسائل مع `WebSocket` وخصائص الرد والتفاعل والتثبيت والمرفقات إلى توفير واجهة مراسلة متكاملة، في حين بقيت PostgreSQL المرجع الدائم للبيانات واستُخدم الاتصال اللحظي لتحسين سرعة ظهور التحديثات وليس بديلاً عن التخزين والمزامنة.

7.7- تكامل مكونات النظام وتشغيله

1.7.7- سيناريو نشر رسالة متكامل

يمكن توضيح تكامل مكونات النظام من خلال تتبع عملية نشر رسالة من مستخدم إلى إحدى القنوات التي تضم مستخدماً آخر مشتركاً فيها. وبمر هذا السيناريو بعدة مكونات مستقلة، تبدأ من الواجهة الأمامية وتنتهي بإظهار الرسالة لحظياً لدى المشترك، مع بقاء PostgreSQL المرجع الدائم للرسالة طوال المسار. ويوضح الشكل رقم (28) هذا المسار من المرسل وحتى وصول الرسالة إلى المستقبل.



الشكل رقم 28 مسار نشر الرسالة وتوزيعها من المرسل إلى المستقبل عبر مكونات النظام

يبدأ السيناريو عندما يكتب المستخدم الرسالة في واجهة القناة ويضغط زر الإرسال. تقوم واجهة Next.js بإرسال طلب نشر إلى FastAPI مع بيانات المصادقة المطلوبة ومحتوى الرسالة والمعلومات الإضافية المرتبطة بها عند وجودها.

تتحقق الواجهة الخلفية أولاً من جلسة المستخدم وهويته، ثم تتحقق من أن القناة موجودة وأن المستخدم يمتلك عضوية وصلاحيات تسمح له بالنشر فيها. ويتم كذلك التحقق من بنية الطلب والحدود المطبقة على المحتوى والمرفقات قبل تنفيذ عملية الحفظ.

بعد نجاح التحقق تتولى MessageService إنشاء الرسالة وتجهيز محتواها للحفظ المحلي. وضمن معاملة واحدة في PostgreSQL يتم تثبيت سجل Message وسجل Outbox الذي يمثل نية توزيع الرسالة، إضافة إلى الحدث المرتبط بعملية النشر في سجل الأحداث. وبهذا لا توجد نافذة يكون فيها محتوى الرسالة محفوظاً بينما تكون مهمة توزيعها قد فُقدت.

بعد نجاح المعاملة تعيد FastAPI نتيجة عملية النشر إلى المستخدم المرسل. ومن المهم أن هذه الاستجابة لا تنتظر وصول الرسالة إلى جميع المشتركين، لأن عملية القبول والتخزين منفصلة عن عملية التوزيع الخلفية. وبالتالي تكون الرسالة قد أصبحت محفوظة بصورة دائمة قبل بدء مسار التوزيع غير المتزامن.

يقوم Worker بعد ذلك بقراءة سجلات Outbox المستحقة للمعالجة وحجز سجل الرسالة، ثم ينشر الحدث إلى Topic Exchange في RabbitMQ. ويقوم RabbitMQ باستخدام روابط الاشتراك الحالية لتوجيه الحدث إلى طوابير المستخدمين الذين يمتلكون عضوية تسمح لهم باستقبال أحداث القناة.

إذا كان أحد المشتركين متصلاً بالنظام، يقوم الجزء المسؤول عن استهلاك طابوره في Worker باستقبال الحدث ثم تمريره إلى Redis. ويستخدم Redis هنا كوسيط سريع بين مكوّن Worker وخادم FastAPI الذي يحتفظ باتصال WebSocket الخاص بالمستخدم.

قبل تسليم الحدث النهائي عبر WebSocket يحافظ النظام على التحقق من صلاحية المستخدم الحالية للوصول إلى القناة، بحيث لا يكون وجود حدث قديم داخل مسار التوزيع كافياً لإرسال محتوى الرسالة إلى مستخدم تمت إزالة عضويته. ويستخدم التنفيذ الحالي معلومات إصدار حالة العضوية لإعادة التحقق من PostgreSQL عند الحاجة قبل فك محتوى الرسالة وإرساله.

بعد نجاح التحقق، يرسل مدير WebSocket الحدث إلى متصفح المستخدم المشترك، فتقوم واجهة React بتحديث قائمة الرسائل وتظهر الرسالة الجديدة دون الحاجة إلى إعادة تحميل الصفحة. وبذلك تكون الرسالة قد انتقلت من عملية المستخدم الواحدة إلى التخزين الدائم ثم التوزيع غير المتزامن وأخيراً إلى التحديث اللحظي للمشارك.

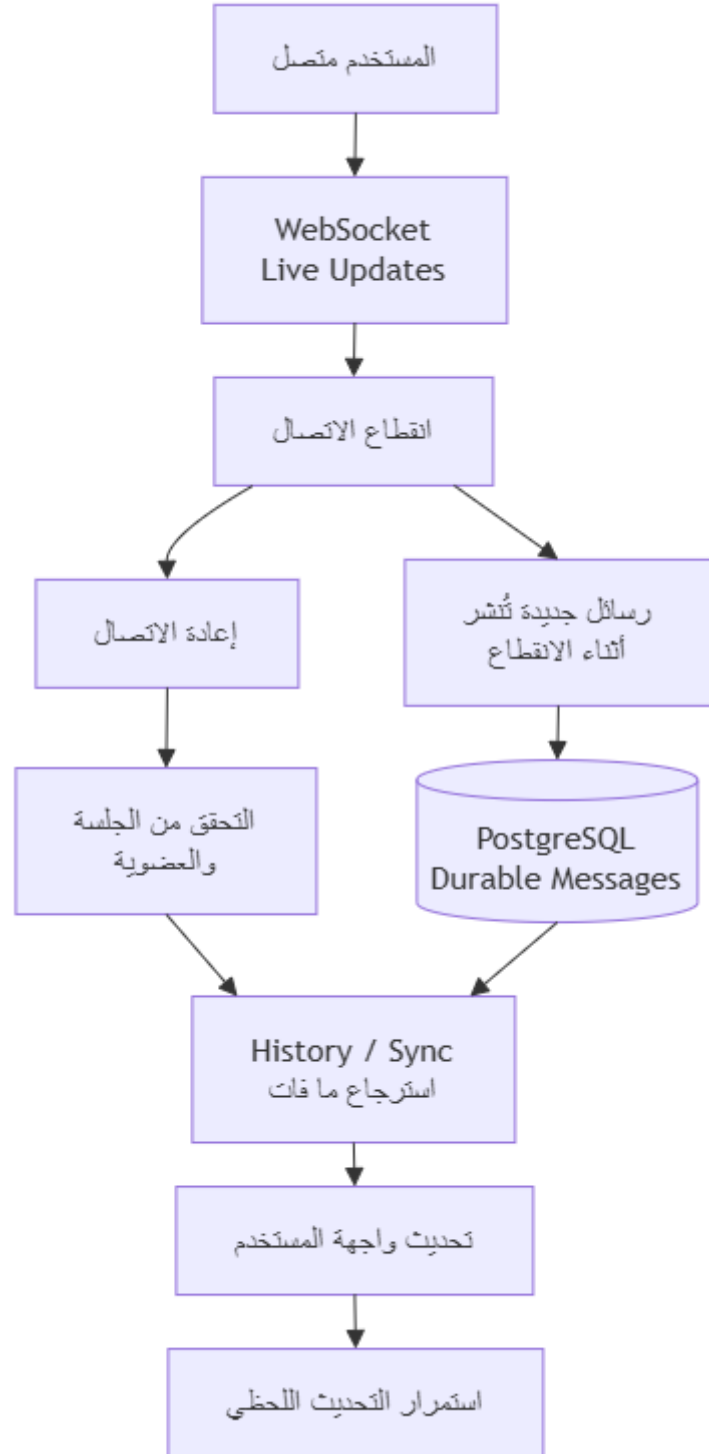
يوضح هذا السيناريو أن المرسل لا يتعامل مباشرة مع المستخدمين المستقبلين، كما أن FastAPI لا تقوم بإرسال الرسالة بصورة فردية إلى كل مشترك. فالواجهة الخلفية مسؤولة عن قبول الرسالة وتثبيتها، وWorker مسؤول عن المعالجة الخلفية، وRabbitMQ مسؤول عن التوجيه، وRedis وWebSocket مسؤولان عن تسريع إيصال الأحداث للمستخدمين المتصلين. وهذا الفصل هو التطبيق العملي لفكرة النشر والاشتراك في المشروع.

2.7.7 - انقطاع الاتصال وإعادة المزامنة

تم تصميم النظام بحيث لا يؤدي انقطاع اتصال WebSocket إلى فقدان الرسائل من وجهة نظر المستخدم. فالرسائل تثبت أولاً في PostgreSQL قبل بدء عملية التوزيع، بينما يمثل WebSocket مساراً سريعاً لإظهار التحديثات للمستخدمين المتصلين. لذلك يمكن استعادة الحالة الصحيحة بعد عودة الاتصال حتى لو لم تصل بعض الأحداث اللحظية أثناء فترة الانقطاع.

عندما ينقطع اتصال المستخدم، يتوقف المسار اللحظي بين Redis وخادم WebSocket والمتصفح، إلا أن بقية النظام لا تعتمد على بقاء هذا الاتصال. ويمكن أن تستمر الرسائل الجديدة في الحفظ والتوزيع بصورة مستقلة، بينما تبقى النسخة الدائمة منها موجودة في PostgreSQL.

عند عودة المستخدم إلى التطبيق أو فتح القناة من جديد، تقوم الواجهة بإعادة إنشاء الاتصال المطلوب ثم تسترجع البيانات التي تحتاج إليها من الواجهة الخلفية. ويوفر النظام لهذا الغرض مسارات REST لاستعراض الرسائل والمزامنة، إضافة إلى دعم استئناف واسترجاع السجل عبر اتصال WebSocket. ويوضح الشكل رقم (29) مسار استعادة الرسائل وإعادة المزامنة بعد عودة اتصال المستخدم.



الشكل رقم 29 مسار استعادة الرسائل والمزامنة بعد انقطاع الاتصال

تتم عملية الاسترجاع بالاعتماد على الحالة المعروفة لدى العميل وحالة الرسائل الدائمة في الخادم، بحيث يمكن إعادة الرسائل التي لم تظهر أثناء فترة الانقطاع بدلاً من افتراض أن كل ما أرسل عبر الاتصال اللحظي قد وصل إلى العميل. وتتولى `MessageService` عمليات `list_messages` و `sync`، بينما تدعم طبقة `WebSocket` وظائف الاستئناف وإرسال السجل السابق وتحديث حالة المشاهدة. بعد استرجاع الرسائل الناقصة تقوم الواجهة بدمجها مع الحالة الموجودة لديها، ثم يستمر استقبال التحديثات الجديدة عبر `WebSocket`. وبهذا تكون عملية المزامنة مسؤولة عن سد الفجوة بين آخر حالة معروفة قبل الانقطاع والحالة الحالية الموجودة في `PostgreSQL`.

ترتبط عملية المزامنة كذلك بقواعد الوصول الحالية، فلا يعني كون المستخدم قد امتلك اتصالاً سابقاً بالقناة أنه يستطيع استرجاع محتواها دائماً. إذ تعيد الواجهة الخلفية التحقق من العضوية والصلاحيات قبل السماح بقراءة السجل أو تنفيذ المزامنة. وقد شملت اختبارات الأمان حالات تمنع المستخدم ذي العضوية المعلقة أو المستخدم الخارجي من استرجاع رسائل القنوات الخاصة، كما تحافظ على وصول المستخدم إلى التحديثات الموجهة إليه مباشرة مثل حدث إزالة عضويته عند الحاجة.

كما أن Redis لا يستخدم مخزناً لتعويض الرسائل المفقودة بعد الانقطاع، لأنه يمثل طبقة تنسيق مؤقتة للمستخدمين النشطين. وتبقى PostgreSQL المصدر الذي تعتمد عليه عمليات الاسترجاع، بينما يؤدي RabbitMQ و Redis و WebSocket أدواراً مختلفة في التوزيع والإيصال اللحظي.

وبذلك يسمح الجمع بين الاتصال اللحظي والمزامنة للمستخدم بالحصول على تجربة سريعة أثناء الاتصال، مع المحافظة في الوقت نفسه على إمكانية استعادة الحالة بعد انقطاع الشبكة أو إعادة فتح التطبيق. ويمنع هذا التصميم الاعتماد على اتصال WebSocket طويل العمر كشرط أساسي لسلامة سجل الرسائل.

3.7.7- تشغيل النظام

تم إعداد المشروع ليعمل كمجموعة من الخدمات المنفصلة التي يتم تجميعها وتشغيلها باستخدام Docker Compose. ويحتوي المستودع على ثلاثة ملفات تشغيل رئيسية هي docker-compose.yml للتطوير المحلي، و docker-compose.hardened.yml للتشغيل المحلي المحسن والتحقق الأمني، و docker-compose.production.yml لتشغيل موجه إلى بيئة إنتاجية على خادم واحد.

يشغل مسار التطوير الخدمات الرئيسية التي يحتاج إليها النظام، وهي PostgreSQL و RabbitMQ و Redis والواجهة الخلفية و Worker والواجهة الأمامية. ويتيح ذلك تشغيل البنية الموزعة كاملة من خلال أمر واحد بدلاً من تشغيل كل خدمة وإعداد اتصالاتها بصورة منفصلة. وقد تم اعتماد الأمر docker compose up -d --build بوصفه المسار الأساسي لبناء الصور وتشغيل الخدمات محلياً.

قبل التشغيل يتم إنشاء ملف env. انطلاقاً من env.example. ثم ضبط متغيرات البيئة المطلوبة، مثل بيئة التشغيل ومفاتيح المصادقة والتشفير وبيانات الاتصال بالخدمات. وقد أصبح تحديد بيئة التشغيل صريحاً، بحيث لا يعتمد التطبيق على اختيار وضع التطوير بصورة ضمنية عند غياب متغير ENVIRONMENT. أما بيئة الإنتاج فتستخدم env.production.example. كقالب فقط، وتحتاج إلى قيم حقيقية يتم تمريرها أثناء النشر ولا يتم تخزينها داخل المستودع.

بعد إعداد البيئة يمكن التحقق أولاً من صحة ملف Compose باستخدام docker compose config، ثم بناء وتشغيل الخدمات. وبعد الإقلاع يستخدم docker compose ps -a للتأكد من أن الخدمات أصبحت في الحالة المطلوبة وأن الخدمات التي تحتوي على فحوص صحة قد أصبحت جاهزة. ويعرض Backend مسار صحة محدوداً يمكن استخدامه للتحقق من أن التطبيق يعمل دون كشف تفاصيل داخلية غير ضرورية. أما مخطط قاعدة البيانات فيتم إدارته بواسطة Alembic. وفي مسار التشغيل الإنتاجي توجد خدمة migrate مستقلة تستخدم حساب قاعدة بيانات إداري لتنفيذ alembic upgrade head وتحديث المخطط ومنح الصلاحيات للحساب التشغيلي، ثم يبدأ Backend و Worker باستخدام حساب أقل صلاحية ولا يقومون بتنفيذ الترحيلات أثناء التشغيل العادي.

يتضمن الإصدار الحالي ترحيلات قاعدة البيانات وصولاً إلى الترحيل 0024_phase11_merkle_audit، وقد تم التحقق من إنشاء قاعدة جديدة حتى هذا الإصدار ومن الترقية من الإصدار السابق إليه. ويسمح استخدام Alembic بالمحافظة على تطور مخطط PostgreSQL بصورة قابلة للتتبع بدلاً من تعديل الجداول يدوياً بين الإصدارات.

بعد تشغيل الخدمات، يتصل Backend ب PostgreSQL و Redis والمكونات المطلوبة لتقديم REST و WebSocket، بينما يتصل Worker ب PostgreSQL و RabbitMQ و Redis لتنفيذ معالجة Outbox والتوزيع وإعادة المحاولة والجسر اللحظي. وتشغل الواجهة الأمامية تطبيق Next.js الذي يتصل بالمستخدم من جهة وبواجهات Backend من جهة أخرى.

في ملف التشغيل الإنتاجي يتم وضع Nginx أمام التطبيق بوصفه نقطة الدخول الخارجية، بينما تبقى خدمات PostgreSQL و RabbitMQ و Redis و Backend و Worker غير مكشوفة مباشرة إلى الشبكة الخارجية. وقد أظهرت اختبارات بيئة الإنتاج أن المنافذ الخارجية كانت مقصورة على HTTP و HTTPS من خلال Proxy، بينما بقيت بقية الخدمات ضمن شبكة Docker الداخلية.

ولا تقتصر عملية التأكد من تشغيل النظام على ظهور الحاويات فقط، إذ يوفر المشروع سكريبتات تحقق تقوم بتنفيذ دورة استخدام متكاملة. ويستخدم scripts/verify_demo_flow.py للتحقق من التسجيل وتسجيل الدخول وإنشاء القناة والانضمام والنشر وحفظ الرسالة وتوزيعها عبر Worker و RabbitMQ و Redis و WebSocket واسترجاعها أيضاً عبر REST. وقد نجح هذا المسار في بقاء التحقق الموثقة للمشروع.

وبذلك تم توفير مسار تشغيل موحد يسمح ببدء جميع مكونات النظام والتحقق من جاهزيتها وتشغيل ترحيلات قاعدة البيانات وتنفيذ سيناريو تحقق عملي، مع فصل واضح بين إعدادات التطوير المحلي والتشغيل المحسن وبيئة الإنتاج ذات المتطلبات الأمنية الأعلى.

8.7 – الاختبارات والنتائج

1.8.7 – منهجية الاختبار

تم اعتماد منهجية اختبار متعددة المستويات للتحقق من النظام، لأن اختبار كل مكون بصورة منفصلة لا يكفي لإثبات صحة نظام موزع يعتمد على قاعدة بيانات ووسيط رسائل ومكون Worker واتصال لحظي. لذلك جُمعت اختبارات الشيفرة الآلية مع اختبارات التكامل والتحقق من بيئة التشغيل واختبارات السيناريو الكامل.

تمثل اختبارات الواجهة الخلفية الجزء الأكبر من الاختبارات الآلية، وقد تم تنفيذها باستخدام pytest. وتشمل اختبارات للوظائف الأساسية والخدمات وقواعد الصلاحيات والمصادقة والجلسات والقنوات والعضويات والرسائل والمرفقات والتوزيع وسجل الأحداث، إضافة إلى مجموعات مستقلة للتحقق من التحسينات الأمنية التي أضيفت إلى النظام.

لم تعتمد جميع الاختبارات على كائنات وهمية فقط، بل استخدمت المراحل المتقدمة من التحقق قواعد PostgreSQL مؤقتة ومعزولة عن بيانات المشروع، كما استخدمت اختبارات معينة Redis فعلياً للتحقق من السلوك الذي يعتمد على التنسيق المشترك والحضور والاتصال اللحظي. ويساعد استخدام خدمات حقيقية في هذه الحالات على اختبار خصائص لا يمكن التحقق منها بصورة كافية من خلال اختبار الدوال منفردة.

تم كذلك تخصيص مجموعات **Regression Tests** بحيث يعاد تشغيل الاختبارات السابقة بعد كل مرحلة من مراحل التقوية الأمنية أو إضافة وظائف جديدة. والهدف من ذلك هو التأكد من أن إصلاح مشكلة في المصادقة أو WebSocket أو التشفير أو المرفقات أو سجل التدقيق لا يؤدي إلى تعطيل وظائف سبق أن تم التحقق منها. وقد شملت عمليات التحقق الأخيرة تشغيل المجموعة الكاملة للواجهة الخلفية إلى جانب مجموعات مركزة للمراحل الجديدة.

أما الوظائف الأمنية فقد اختبرت باستخدام حالات نجاح وحالات رفض متعددة، مثل محاولة استخدام جلسة ملغاة، وإعادة استخدام Refresh Token، والوصول إلى مورد دون عضوية، والتلاعب بالبيانات المشفرة، ومحاولات تغيير بيانات سجل الأحداث أو Merkle Proof أو التوقيع. والهدف من هذه الاختبارات ليس فقط إثبات أن الاستخدام الصحيح يعمل، وإنما إثبات أن النظام يرفض الحالات غير الصحيحة بالطريقة المتوقعة.

وللتحقق من تكامل المكونات بصورة فعلية، تم استخدام سكريبت verify_demo_flow.py على بيئة Docker تعمل فيها PostgreSQL و RabbitMQ و Redis و Backend و Worker والواجهة. ويقوم هذا الاختبار بتنفيذ سيناريو متكامل يشمل التسجيل وتسجيل الدخول وإنشاء القناة والانضمام والنشر وحفظ الرسالة ومعالجتها عبر Worker و RabbitMQ و Outbox، ثم استقبالها عبر Redis و WebSocket والتحقق من إمكانية استرجاعها بواسطة REST.

تم أيضاً اختبار تطور قاعدة البيانات من خلال Alembic، بحيث لا يقتصر التحقق على قاعدة جديدة فقط، وإنما يتم عند الحاجة اختبار الانتقال من إصدار سابق إلى الإصدار الجديد. وفي أحدث مرحلة تم التحقق من إنشاء مخطط جديد وصولاً إلى الترحيل 0024_phase11_merkle_audit، إضافة إلى مسار ترقية تمثيلي من 0023 إلى 0024.

أما الواجهة الأمامية فقد خضعت لفحوص TypeScript باستخدام npm run typecheck، إضافة إلى بناء نسخة الإنتاج باستخدام npm run build. كما تم التحقق من ملفات الترجمة العربية والإنكليزية ومن إمكانية بناء واجهة Next.js ضمن مسارات التشغيل المعتمدة. ولا تتضمن النسخة الحالية مجموعة آلية كاملة لاختبارات المتصفح من طرف إلى طرف، لذلك تم الاعتماد في سيناريو التكامل الكامل على سكريبتات التحقق والخدمات الفعلية بدلاً من الادعاء بوجود Browser E2E Suite غير منفذة.

وشملت منهجية التحقق كذلك ملفات Docker Compose، حيث تم فحص إمكانية توليد إعدادات بيئات التطوير والتشغيل المحسن والإنتاج، إضافة إلى اختبارات بناء وتشغيل الخدمات والتحقق من فحوص الصحة وعزل بعض الخدمات في بيئة الإنتاج. وبهذا لم تقتصر عملية التحقق على الشيفرة، بل شملت أيضاً الطريقة التي يتم من خلالها تجميع النظام وتشغيل مكوناته.

وبذلك تعتمد منهجية الاختبار في المشروع على الجمع بين الاختبارات المركزة لكل وظيفة، واختبارات الانحدار بعد التعديلات، واختبارات التكامل مع الخدمات الفعلية، والتحقق من الترحيلات وواجهة المستخدم وبيئة Docker، إضافة إلى تنفيذ سيناريو كامل يمر عبر المكونات الموزعة الرئيسة للنظام. ويلخص الجدول رقم (15) مستويات الاختبار المختلفة وما يتم التحقق منه في كل مستوى.

مستوى الاختبار	ما يتم التحقق منه
اختبارات Backend	قواعد العمل، التحقق من المدخلات، الصلاحيات والخدمات
اختبارات الأمان	المصادقة، الجلسات، التشفير، الوصول غير المصرح، كشف العبث
اختبارات التكامل	PostgreSQL و Redis والمكونات التي تحتاج إلى خدمة فعلية
Regression Tests	التأكد من عدم كسر الوظائف السابقة بعد التعديلات
Full-flow Verification	Outbox → RabbitMQ → Worker → Redis → WebSocket → REST Sync
اختبارات الواجهة	Next.js production build, TypeScript typecheck
اختبارات قاعدة البيانات	upgrade paths, Alembic fresh migration
اختبارات التشغيل	Docker Compose وبناء وتشغيل الخدمات

الجدول رقم 13 مستويات الاختبار وعمليات التحقق

2.8.7- الاختبارات الوظيفية والتكاملية

تم تنفيذ مجموعة من الاختبارات الوظيفية للتحقق من أن العمليات الأساسية التي يحتاج إليها المستخدم تعمل وفق المتطلبات، ثم تم دعمها باختبارات تكامل تتأكد من أن المكونات المنفصلة تتعاون بصورة صحيحة عند تنفيذ سيناريو حقيقي. وشملت هذه الاختبارات المصادقة والقنوات والعضويات والرسائل والمرفقات والمزامنة، إضافة إلى مسار النشر والاشتراك الكامل.

في مجال المصادقة تم التحقق من إنشاء المستخدمين وتسجيل الدخول واستعادة بيانات المستخدم الحالي وتجديد الجلسة وتسجيل الخروج. كما تم اختبار المسارات التي تحتاج إلى مصادقة للتأكد من عدم السماح بتنفيذها عند غياب هوية صحيحة، إضافة إلى التحقق من أن حالة الجلسة تنعكس بصورة صحيحة على العمليات المحمية.

أما اختبارات القنوات والعضويات فقد شملت إنشاء القناة وتوليد بياناتها المطلوبة وإنشاء عضوية المالك، ثم الانضمام إلى القنوات ومعالجة العضويات التي تتطلب موافقة إدارية. كما تم اختبار السيناريو الذي يكون فيه المستخدم في حالة انتظار قبل الموافقة ثم يصبح عضواً فعالاً بعد اعتماد الطلب، مع التأكد من تغيير إمكانية الوصول إلى وظائف القناة تبعاً لحالة العضوية. ويحتوي المشروع أيضاً على verifier مخصص لمسار العضوية التي تتطلب الموافقة، ويتحقق من طلب الانضمام والموافقة والمزامنة ووصول الرسالة بعد اعتماد العضوية.

تم اختبار نشر الرسائل من خلال التحقق من صلاحية المرسل ثم إنشاء الرسالة وحفظها واسترجاعها لاحقاً. ومن الاختبارات الأساسية الموجودة في المشروع اختبار test_smoke_flow_channel_join_publish_sync_and_events، الذي يجمع ضمن سيناريو واحد إنشاء القناة والانضمام والنشر والمزامنة والتحقق من الأحداث المسجلة. كما توجد اختبارات مستقلة لإنشاء القناة وتشغيل الرسالة والتحقق من الوصول إليها. وشملت اختبارات المرفقات رفع الملفات وربطها بالرسائل واسترجاعها من قبل مستخدم مخول، إلى جانب التحقق من أن العلاقة بين المرفق والرسالة والقناة تبقى متوافقة. وفي الاختبارات النهائية تم تشغيل مسار رفع فعلي أدى إلى إنشاء ملف مشفر، ثم تم تنزيله ومقارنة محتواه الأصلي بعد فك التشفير للتأكد من تطابقه، مع بقاء الوصول مرتبطاً بقواعد العضوية.

أما المزامنة فقد اختبرت من خلال إرسال رسائل أثناء عدم اعتماد العميل على الاتصال اللحظي ثم استرجاعها باستخدام واجهة /sync/ وقد تحقق السيناريو النهائي من أن الرسالة التي لم تصل لحظياً يمكن استعادتها بعد ذلك من التخزين الدائم، وهو ما يثبت أن WebSocket ليس المصدر الوحيد لسجل الرسائل. ويلخص الجدول رقم (16) أهم الاختبارات الوظيفية والتكاملية والنتائج التي تم الحصول عليها.

السيناريو المختبر	ما تم التحقق منه	النتيجة
التسجيل وتسجيل الدخول	إنشاء الحساب والجلسة والوصول إلى المسارات المحمية	نجح
إنشاء قناة	إنشاء القناة وعضوية المالك وتسجيل الحدث	نجح
الانضمام للقناة	العضوية المباشرة وحالة انتظار الموافقة	نجح
نشر رسالة	الحفظ والاسترجاع وإنشاء بيانات التوزيع	نجح
المرفقات	الرفع والربط والتنزيل للمستخدم المخول	نجح
المزامنة	استرجاع الرسائل التي فاتت المستخدم عبر /sync/	نجح
النشر والاشتراك الكامل	PostgreSQL → Outbox → RabbitMQ → Worker → Redis → WebSocket	نجح
ترحيلات قاعدة البيانات	إنشاء قاعدة جديدة والترقية من إصدار سابق	نجح

الجدول رقم 14 نتائج الاختبارات الوظيفية والتكاملية

وللتحقق من سلسلة النشر والاشتراك بصورة متكاملة تم استخدام scripts/verify_demo_flow.py. ينشئ هذا السكريبت عدة مستخدمين، ويسجل دخولهم، وينشئ قناة، وينفذ الانضمام إليها، ثم ينشر رسالة ويتابع انتقالها عبر PostgreSQL وسجل Outbox و RabbitMQ و Worker و Redis وصولاً إلى WebSocket، إضافة إلى التحقق من REST backfill عند الحاجة. ولذلك يمثل هذا السكريبت أقوى اختبار عملي على مستوى المستودع للمسار الموزع الكامل. وتظهر نتيجة تنفيذ هذا السيناريو المتكامل في الشكل رقم (30).

```

D:\Users\abdal\Desktop\FYP\MessagingSystem>python scripts/verify_demo_flow.py --base-url http://localhost:8000/v1
[STEP] 0) Waiting for API health
[INFO] health timeout=60s, poll interval=1s
[PASS] API health check passed
[STEP] 1) Register/login User A, User B, and User C
[PASS] Registered and logged in demo_a_7f951e7b, demo_b_7b54f4de, and demo_c_211e4ddd
[STEP] 2) User A creates public/open channel 'news'
[PASS] Created channel news (36b4a247-c01b-4edf-ac12-fc2912f41e0a)
[STEP] 3) User B opens WebSocket before joining the channel
[INFO] hello timeout=10s, subscribe acknowledgement timeout=10s, live delivery timeout=30s
[PASS] WebSocket hello received
[STEP] 4) User B joins channel while the WebSocket is already open
[PASS] User B joined and refreshed the open WebSocket subscription
[STEP] 5) User A publishes a message
[PASS] Published message b46583bc-a136-4cf3-888d-0c1c6f1a6ea9
[PASS] User B received the message through WebSocket
[STEP] 6) User B verifies REST backfill
[PASS] REST backfill verified alongside live WebSocket delivery
[STEP] 7) User A checks event log has core events
[PASS] Event log contains channel and message activity
[STEP] 8) User A verifies event integrity for the channel
[PASS] Event integrity verified for 3 events
[STEP] 9) User C is blocked from private upload content
[FAIL] Demo verification failed (RuntimeError): PUT /uploads/83442e57-abb0-4e7b-b4b1-f9d03dd74c45/content failed (500):
{"code":"INTERNAL_ERROR","message":"internal server error","details":null}

```

الشكل رقم 30 نتيجة ال full Pub/Sub flow

وفي التحقق النهائي تم تشغيل هذا السيناريو داخل بيئة معزولة تعمل فيها الخدمات الحقيقية للنظام، ونجح مسار التسجيل وتسجيل الدخول وإنشاء القناة والانضمام والنشر وتثبيت الرسالة في PostgreSQL وانتقال سجل Outbox إلى حالة النشر ووصول الرسالة عبر RabbitMQ وWorker وRedis وWebSocket، إضافة إلى استرجاعها باستخدام REST.

كما تم التحقق من تكامل قاعدة البيانات مع الإصدارات الجديدة عن طريق تشغيل Alembic على قاعدة فارغة حتى الترحيل 0024_phase11_merkle_audit، إضافة إلى اختبار ترقية قاعدة ممثلة من الإصدار 0023 إلى 0024 مع الاحتفاظ بالمستخدمين والقنوات والرسائل والمرفقات والبيانات الموجودة مسبقاً دون تغيير غير مقصود.

توضح هذه الاختبارات أن التحقق لم يقتصر على اختبار كل دالة بصورة منفصلة، بل شمل دورة استخدام متكاملة تمر عبر المكونات الأساسية للنظام. ومع ذلك يبقى verify_demo_flow.py سكرت تحقّق عملي وليس بديلاً عن مجموعة Browser E2E أو بيئة اختبار موزعة واسعة متعددة العقد، وهي من الحدود التي يتم توضيحها ضمن نتائج المشروع.

3.8.7- اختبارات الأمان والموثوقية

تم تنفيذ مجموعة واسعة من الاختبارات للتحقق من أن النظام لا يعمل فقط في الحالات الصحيحة، وإنما يرفض الاستخدام غير المصرح ويتعامل بصورة متوقعة مع حالات الفشل والتلاعب. وشملت هذه الاختبارات المصادقة والجلسات والصلاحيات وتشفير البيانات وموثوقية التوزيع والاتصال اللحظي وسلامة سجل الأحداث. ويلخص الجدول رقم (17) أهم حالات الأمان والموثوقية التي تم اختبارها والنتائج المتوقعة منها.

مجال الاختبار	مثال عن الحالة المختبرة	النتيجة المتوقعة
الجلسات	إعادة استخدام Refresh Token قديم	اكتشاف Replay ورفض الجلسة
الصلاحيات	مستخدم خارجي يطلب سجل قناة خاصة	رفض الوصول
إزالة العضوية	عضو سابق يحاول قراءة السجل	رفض الوصول
تشفير الرسائل	فحص التخزين في PostgreSQL/Outbox	عدم وجود النص الصريح
تشفير المرفقات	تعديل Ciphertext أو GCM Tag	رفض فك التشفير
Broker Generation	وصول Bind/Unbind قديم	تجاهله كحالة Stale
Outbox	فشل النشر المتكرر	Retry ثم Dead Letter عند الاستنفاد
Redis Presence	إغلاق أحد اتصاليين	يبقى المستخدم Online
Audit Hash Chain	تغيير بيانات حدث	اكتشاف عدم تطابق التجزئة
Merkle Proof	تعديل عقدة أو توقيع	فشل التحقق

الجدول رقم 15 نتائج اختبارات الأمان والموثوقية

في مجال المصادقة تم اختبار تدوير Refresh Token ومحاولة إعادة استخدام رمز تحديث قديم بعد استبداله. وتحقق السيناريو النهائي من أن إعادة استخدام الرمز القديم تؤدي إلى اكتشاف حالة Replay وإبطال عائلة الجلسة المرتبطة بها، كما تم التحقق من إمكانية تسجيل الدخول من جديد ومن أن تسجيل الخروج يبطل الجلسة السابقة.

أما اختبارات الصلاحيات فقد استخدمت مستخدمين بأدوار وحالات عضوية مختلفة، مثل المالك والعضو والمستخدم الخارجي. وتم التحقق من رفض الوصول إلى سجل قناة خاصة أو مرفقاتها للمستخدم غير العضو، كما تم التحقق من أن المستخدم يفقد القدرة على استرجاع السجل المحمي بعد إزالة عضويته من القناة. وفي سيناريو الإنتاج التجريبي أعادت محاولات المستخدم الخارجي للوصول إلى الموارد الخاصة استجابة رفض 403.

تمت كذلك تغطية حماية البيانات أثناء التخزين باختبارات تتحقق من أن نص الرسالة المحفوظ في PostgreSQL وسجل Outbox يبقى ضمن الغلاف المشفر بالإصدار الحالي، وأن محتوى الملف المرفوع لا يحفظ كنص صريح على القرص. كما تم اختبار تغيير أجزاء متعددة من تنسيق الملف المشفر، مثل الرأس ومعرف المفتاح والNonce وأطوال الأجزاء والبيانات المشفرة ووسم GCM، وقد أدت حالات التلاعب إلى فشل مغلق دون الرجوع إلى نسخة غير مشفرة.

وفي مجال موثوقية التوزيع تم اختبار الحالات المختلفة لسجل Outbox، بما في ذلك نجاح النشر وجدولة إعادة المحاولة وحالة الفشل النهائي dead_lettered كما تتحقق الاختبارات من حفظ معلومات الخطأ بصورة محدودة وآمنة، ومن إمكانية إعادة العملية إلى مسار المعالجة من خلال إعادة المحاولة الإدارية. ويجب التمييز هنا بين هذه الاختبارات الآلية وبين اختبار انقطاع RabbitMQ الفعلي؛ فقد تم التحقق من منطق الفشل وإعادة المحاولة، لكن النسخة النهائية لا تدعي تنفيذ اختبار Live كامل لانقطاع RabbitMQ على مستوى المنظومة.

كما تم اختبار ترتيب تحديث روابط RabbitMQ باستخدام أجيال حالة العضوية، بحيث لا تستطيع عملية Bind أو Unbind قديمة الوصول متأخرة وتجاوز حالة أحدث. وقد أثبتت اختبارات RabbitMQ أن الأجيال القديمة تعامل كحالة stale_generation، بينما تبقى أحدث حالة مخزنة في PostgreSQL هي التي تحدد الربط المطلوب فعلياً.

أما الاتصال اللحظي والحضور فقد خضع لاختبارات باستخدام Redis حقيقي مؤقت. وشملت الحالات وجود أكثر من WebSocket للمستخدم نفسه، والاتصالات الموجودة على أكثر من Backend، وإغلاق اتصال مع بقاء اتصال آخر، وانتهاء Lease بعد توقف خادم، وإعادة الاتصال السريعة وإبطال الجلسات. وقد تحقق التنفيذ من أن إغلاق اتصال واحد لا يجعل المستخدم Offline طالما بقي اتصال آخر فعالاً.

تم أيضاً اختبار سلوك النظام عند عدم توفر Redis في منطق الحضور، بحيث لا يؤدي فقدان Redis إلى استنتاج خاطئ بأن المستخدم أصبح غير متصل. وبدلاً من ذلك تعامل الحالة على أنها غير مؤكدة إلى حين استعادة الخدمة. ولا تمثل حالة الحضور أساساً للمصادقة أو منح الصلاحيات، ولذلك لا يؤدي فقدانها إلى منح وصول غير مصرح. ولم يتم الادعاء بتنفيذ اختبار انقطاع كامل لكل خدمات Redis ضمن Stack أو اختبار تحميل إنتاجي طويل المدة.

وبالنسبة إلى سجل الأحداث، تم اختبار سلسلة SHA-256 وطبقة Merkle والتوقيع الرقمي من خلال إدخال تعديلات مقصودة على نسخ من البيانات. وشملت الاختبارات تغيير حمولة الحدث وقيمة تجزئته وأوراق Merkle والجذر ومعلومات Checkpoint والتوقيع ومسار Inclusion Proof ومفاتيح التحقق. وتمكنت آليات التحقق من اكتشاف هذه الحالات ورفض الأدلة المعدلة.

كما أجري اختبار عملي مباشر على بيانات PostgreSQL، حيث أعطى السجل نتيجة صحيحة قبل العبث، ثم أدى تغيير محتوى حدث إلى EVENT_HASH_MISMATCH، وأدى تغيير Merkle Root إلى CHECKPOINT_HASH_MISMATCH. وفي العرض النهائي تم كذلك تعديل نسخة من Inclusion Proof، وفشل التحقق منها كما هو متوقع، بينما نجح التحقق من النسخة الأصلية والتوقيع وسلسلة نقاط التحقق. توضح هذه الاختبارات أن الأمان والموثوقية لم يقيما فقط من خلال مراجعة الشيفرة، وإنما من خلال حالات رفض وتلاعب وفشل محددة. ومع ذلك بقيت بعض الجوانب خارج نطاق التحقق الحالي، مثل اختبارات الانقطاع الكامل المطول لوسيط الرسائل، واختبارات Slow Clients واسعة النطاق، واختبارات التحميل الإنتاجية متعددة العقد، ولذلك لا يتم تقديم هذه الجوانب على أنها معتمدة إنتاجياً.

4.8.7- نتائج الاختبارات

أظهرت نتائج التحقق النهائي نجاح الوظائف الرئيسة للنظام ومسارات التكامل والأمان التي تم تنفيذها ضمن نطاق المشروع. وقد أجريت عملية التحقق الأخيرة بعد اكتمال مراحل التنفيذ والتقوية، وشملت الاختبارات الآلية وبناء مكونات النظام وترجيحات قاعدة البيانات وبيئة التشغيل الإنتاجية وسيناريو متكامل للاستخدام. ويلخص الجدول رقم (18) النتائج النهائية لأهم الاختبارات وعمليات التحقق المنفذة على النظام.

الاختبار أو عملية التحقق	النتيجة
المجموعة الكاملة لاختبارات Backend	352 passed, 2 warnings
مجموعة الاختبارات المركزة	213 passed, 2 warnings
Frontend TypeScript Typecheck	نجح
Frontend Production Build	نجح
تطابق ملفات الترجمة	845 مفتاحاً لكل لغة - نجح
Development Compose	نجح
Hardened Compose	نجح
Production Compose	نجح
Nginx Configuration	نجح
Fresh Alembic 0001 → 0024	نجح
Upgrade 0023 → 0024	نجح
Production End-to-End Scenario	نجح
Merkle Integrity Demo	نجح مع كشف التلاعب

الجدول رقم 16 النتائج النهائية للاختبارات وعمليات التحقق

بلغ عدد اختبارات الواجهة الخلفية في المجموعة الكاملة **352** اختباراً ناجحاً دون وجود أي اختبار فاشل، مع ظهور تحذيرين مرتبطين بإهمالات في مكتبات خارجية مستخدمة من Python و Passlib، وليس بأخطاء وظيفية في المشروع. كما نجحت مجموعة مركزة تضم **213** اختباراً تغطي مراحل التطوير والأمان الرئيسة التي سبقت التحقق النهائي.

أما الواجهة الأمامية فقد اجتازت فحص TypeScript باستخدام `npm run typecheck`، كما نجح إنشاء النسخة المحسنة للإنتاج باستخدام `npm run build`. وتم كذلك التحقق من ملفات الترجمة العربية والإنكليزية، حيث تطابق **845** مفتاح ترجمة في كل ملف مع التحقق من البنية والعناصر النائية المستخدمة داخل النصوص.

نجحت أيضاً عمليات توليد إعدادات Docker Compose لبيئات التطوير والتشغيل المحسن والإنتاج، إضافة إلى التحقق من صحة إعدادات Nginx في المسارين المحسن والإنتاجي. وفي بيئة الإنتاج التجريبية كانت المنافذ الخارجية مقتصرة على Nginx عبر HTTP و HTTPS، بينما بقيت PostgreSQL و RabbitMQ و Redis و Backend و Worker والخدمات الداخلية دون منافذ مباشرة إلى المضيف.

وفيما يتعلق بقاعدة البيانات، أكد alembic heads وجود رأس ترحيل واحد هو 0024_phase11_merkle_audit. كما نجحت ترقية قاعدة PostgreSQL فارغة ابتداءً من الترحيل الأول وحتى الإصدار 0024، ونجح سيناريو ترقية قاعدة تحتوي على بيانات فعلية من 0023 إلى 0024 دون تغيير غير مقصود في المستخدمين والقنوات والرسائل والمرفقات والبيانات الموجودة مسبقاً.

أظهرت اختبارات البناء كذلك إمكانية بناء الصور المستخدمة في النظام من الملفات الموجودة في المستودع. ونجح بناء Backend البارد بعد التحسين خلال نحو 252 ثانية، بينما استغرق إعادة البناء الدافئة مع الاستفادة من الذاكرة المخبأة نحو 8.8 ثوانٍ. كما انخفض الحجم الخام لصورة Backend من نحو 152.4 MB إلى 84.5 MB، ونجح بناء Worker والواجهة الأمامية وبيئة Production Compose الكاملة.

أما سيناريو التحقق النهائي المتكامل فقد نجح في تنفيذ دورة استخدام تشمل إنشاء عدة حسابات وتسجيل الدخول وإنشاء قناة خاصة وإدارة العضوية والتحقق من الصلاحيات، ثم نشر رسالة ووصولها إلى مستخدم متصل عبر Redis → Worker → RabbitMQ → Outbox. كما تم التأكد من تخزين الرسالة بصورة مشفرة وإمكانية استرجاع رسالة فائتة باستخدام /sync ومنع المستخدم الخارجي من قراءة سجل القناة أو تنزيل ملفاتها.

نجحت كذلك اختبارات المرفقات في التأكد من تخزين الملفات الجديدة بصيغة مشفرة وعدم وجود المحتوى الأصلي بصورة صريحة، ثم تنزيل الملف بصورة مطابقة للمحتوى الأصلي للمستخدم المصرح له، مع رفض الطلب نفسه للمستخدم غير المخول. كما نجحت اختبارات تدوير Refresh Token واكتشاف إعادة الاستخدام وإبطال الجلسة وتسجيل الخروج.

وفي اختبار سلامة سجل الأحداث تم إنشاء Merkle Checkpoints فعلية من أحداث موجودة في PostgreSQL. ونجح التحقق من تجزئة الحدث ومركلة إدراج Proof وCheckpoint Hash وتوقيع Ed25519 وسلسلة نقاط التحقق، بينما فشل التحقق من نسخة تم تعديل أحد عناصر الإثبات فيها عمداً، وهو السلوك المتوقع من آلية كشف العبث.

تشير هذه النتائج إلى أن النظام حقق المتطلبات الوظيفية الأساسية للمشروع، إضافة إلى تنفيذ والتحقق من مسار النشر والاشتراك غير المتزامن والاتصال اللحظي والحماية أثناء التخزين وسلامة سجل الأحداث. ومع ذلك لا تمثل هذه النتائج اعتماداً إنتاجياً شاملاً؛ إذ لم يتم تنفيذ اختبارات تحميل مستمرة واسعة النطاق أو اختبارات متصفح آلية كاملة أو اختبارات انقطاع موزعة متعددة العقد، ولذلك تبقى هذه الجوانب ضمن الأعمال المستقبلية.

9.7 - الخلاصة

تم في هذا الفصل عرض الجانب العملي من المشروع، بدءاً من تنظيم مكونات النظام وتنفيذ الوظائف الأساسية، وصولاً إلى التوزيع غير المتزامن والاتصال اللحظي وآليات الأمان وسلامة سجل الأحداث.

كما تم توضيح تكامل الواجهة الخلفية وقاعدة البيانات وRedis وWebsocket مع الواجهة الأمامية، وعرض طريقة تشغيل النظام واختباره.

وأظهرت النتائج نجاح الوظائف الأساسية ومسار نشر الرسائل وتوزيعها واستعادتها، مما يؤكد تحقيق المتطلبات العملية الرئيسية للمشروع.